VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 11: Packages, Advanced Concepts and UVM Introduction

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Module 11 Overview 3

Part 1: Packages and Scope 5
- 1.1 Packages 5
- 1.2 Importing Packages 11
- 1.3 The Scope Resolution Operator (::) 16

Part 2: Generic Code 21
- 2.1 Parameterized Classes 21
- 2.2 Generic Reusable Classes 27
- 2.3 Virtual Interfaces: Advanced Recap 33
- 2.4 Clocking Blocks: Advanced Recap 35

Part 3: Process Control 39
- 3.1 Process Control 39
- 3.2 Named Blocks and the disable Statement 41
- 3.3 Named Fork Blocks, disable fork and wait fork 44
- 3.4 The Process Class and Process Handles 48
- 3.5 Watchdog and Timeout Patterns 55

Part 4: Random Sequences, C Code and System Tasks 59
- 4.1 randcase 59
- 4.2 randsequence 61
- 4.3 DPI Introduction 64
- 4.4 Simulator System Tasks and Functions 70

Part 5: Introduction to UVM 81
- 5.1 What UVM Is and Why It Exists 81
- 5.2 How SystemVerilog Features Map to UVM 83
- 5.3 The Class Hierarchy: uvm_object and uvm_component 84
- 5.4 Phases 85
- 5.5 The Factory 87
- 5.6 The Configuration Database 87
- 5.7 UVM Components at a High Level 88
- 5.8 A Minimal UVM Skeleton for the ALU 91
- 5.9 SystemVerilog versus UVM 97

Part 6: Module 11 Revision and Practice 99
- 6.1 Revision Summary 99
- 6.2 Interview Questions and Answers 101
- 6.3 Output-Prediction Questions 103
- 6.4 Find-the-Error and Debugging Questions 107
- 6.5 Coding Exercises with Solutions 111
- 6.6 Concepts You Must Understand Before Moving Forward 118
- 6.7 What Comes Next 118

Module 11 Overview

Modules 1 to 10 taught the language features and used them to build a complete layered testbench for the ALU, with coverage and assertions. This last teaching module does three things:

1. It covers the features that make a testbench **organised and reusable**: packages, the scope resolution operator, and parameterized (generic) classes.
2. It covers the remaining **advanced language features** that interviewers ask about: fine-grained process control, `randcase` and `randsequence`, the Direct Programming Interface (DPI) to C, and the system tasks and functions you use every day.
3. It introduces **UVM** at a high level, by showing that every UVM concept is something you have already built by hand in Module 9.

UVM is deliberately not the focus. The aim is that when an interviewer asks "how does a UVM driver get its transactions?" you can answer by relating it to the mailbox between `alu_generator` and `alu_driver`, which you fully understand.

How this module is organised

Part	Sections	Syllabus topics
1. Packages and scope	1.1 to 1.3	Packages, importing packages, scope resolution operator
2. Generic code	2.1 to 2.4	Parameterized classes, generic reusable classes, virtual interfaces (advanced recap), clocking blocks (advanced recap)
3. Process control	3.1 to 3.5	Process control, named blocks, named fork blocks, process class and process handles, watchdog patterns
4. Random sequences, C and system tasks	4.1 to 4.4	`randcase`, `randsequence`, DPI introduction, simulator system tasks
5. UVM introduction	5.1 to 5.9	Introduction to UVM, SystemVerilog OOP and UVM, UVM components, SystemVerilog versus UVM
6. Revision and practice	6.1 to 6.7	Summary, questions, exercises, checklist

What you need from earlier modules

Topic in this module	Builds on
Packages, `typedef`, enums	M1 (the `alu_pkg` package)
Parameterized classes, static members, `extern`	M4 (classes, static properties, inheritance)
Process control	M7 (`fork`...`join`, `join_any`, `join_none`, `disable fork`, `wait fork`)
Virtual interfaces and clocking blocks	M8
`randcase`, `randsequence`	M5 (randomization and `dist`)
UVM mapping	M9 (the layered testbench) and M10 (coverage)

Naming used in this module

The same names are used as in the rest of the course: the DUT is `alu` with the package `alu_pkg` and the opcode type `alu_op_e`; the interface is `alu_if` with clocking blocks `drv_cb` and `mon_cb`; the transaction class is `alu_txn`; the testbench components are `alu_generator`, `alu_driver`, `alu_monitor`, `alu_scoreboard`, `alu_env` and `alu_test`, collected in the package `alu_tb_pkg`; the mailboxes are `gen2drv_mbx` and `mon2scb_mbx`.

> **Note:** As in the other modules, the "Expected output" blocks were derived by hand from the rules of IEEE 1800-2017 and were not captured from a simulator run. Examples that use random constructs show a "Sample output" whose values differ for every seed and simulator. A few constructs in this module (`randsequence`, `process` suspend and resume, DPI) are supported unevenly by tools; the text says so where it applies.

Part 1: Packages and Scope

1.1 Packages

In plain English

A testbench is made of many files: the DUT, the interface, a dozen classes, the top module. Several of them need the same definitions. The ALU opcode type `alu_op_e` is needed by the DUT, by the interface, by the transaction class and by the scoreboard. If each file wrote its own copy of the definition, the copies would drift apart, and (as Module 1 showed) two identical-looking `typedef`s are not even the same type.

A **package** is a named box in which you put shared definitions **once**. Any file that needs them opens the box by name.

Technical definition

A `package` is a top-level SystemVerilog construct that provides a **declarative scope** (a namespace) for sharing declarations among modules, interfaces, programs, classes and other packages. A package is not instantiated. It exists exactly once in a simulation, and everything declared in it has a single copy.

Why it is needed

Problem without packages	How a package solves it
The same `typedef`, enum or parameter is repeated in many files	Declared once, imported everywhere
Two identical `typedef`s in two files are different types and cannot be connected	One declaration means one type
`` `define `` macros are global text substitutions with no scope, so names collide	Package names are scoped: `alu_pkg::WIDTH` and `fifo_pkg::WIDTH` can coexist
Classes declared outside any scope land in the compilation unit scope (`$unit`), whose contents depend on how the tool was invoked	Classes in a package have a fixed, portable home
Hard to deliver a verification component to another team	The component is delivered as one package (for example the whole of UVM is `uvm_pkg`)

What may and may not be placed in a package

Allowed in a package	Not allowed in a package
`typedef`, `enum`, `struct`, `union` types	Module, interface or program **definitions**
`parameter`, `localparam`, `const`	**Instances** of modules or interfaces
Functions and tasks	`always`, `always_ff`, `always_comb`, `initial`, `final` blocks
Class definitions	Continuous assignments (`assign`)
Variables (one shared static copy) and nets	Hierarchical references to anything outside the package (`tb_top.clk`)
`import` of other packages, `export`	Another package definition (packages cannot be nested)

Allowed in a package	Not allowed in a package
DPI `import` and `export` declarations (Section 4.3)	
Covergroup, `property`, `sequence` and `let` declarations	

The reasoning behind the right-hand column: a package is a collection of *declarations*. It contains no hardware and no processes of its own, and it must be compilable by itself, before and independently of the modules that use it. So it cannot refer to those modules.

How it works internally

```
   Compilation order                        What the compiler builds
   -----------------                        ------------------------
   1. alu_pkg.sv          ----------->      scope "alu_pkg"   { alu_op_e, ALU_ADD, ... }
   2. alu.sv              (import alu_pkg)  module alu        looks names up in alu_pkg
   3. alu_if.sv           (import alu_pkg)  interface alu_if  looks names up in alu_pkg
   4. alu_tb_pkg.sv       (import alu_pkg)  scope "alu_tb_pkg" { alu_txn, alu_driver, ... }
   5. tb_top.sv           (import both)     module tb_top

   - A package must be compiled BEFORE every file that refers to it.
   - There is ONE alu_pkg in the simulation. A variable declared in it is shared by all.
   - Importing copies nothing. It only makes a name visible without the "pkg::" prefix.
```

Two consequences are worth stating explicitly:

- A package is **not** like a C header that is pasted into every file. It is a real scope that is compiled once.
- A variable in a package is effectively a global variable. Every module, interface and class that refers to it sees the same storage.

Syntax

```
package <package_name>;
  // declarations: types, parameters, functions, tasks, classes ...
endpackage : <package_name>        // the closing label is optional
```

There are three ways to use an item from a package. They are explained fully in Section 1.2.

```
alu_pkg::alu_op_e op;          // 1. explicit reference with the scope resolution operator
import alu_pkg::alu_op_e;      // 2. explicit import of one name
import alu_pkg::*;             // 3. wildcard import of all names
```

Complete example

tb_package_basic.sv

```
package util_pkg;

  parameter int DATA_W = 8;

  typedef logic [DATA_W-1:0]       data_t;
  typedef enum {LOW, MEDIUM, HIGH} level_e;

  int unsigned error_count = 0;            // ONE copy, shared by every user of the package

  function automatic data_t swap_nibbles(data_t d);
    return {d[3:0], d[7:4]};
  endfunction
```

```
  function automatic void report_error(string msg);
    error_count++;
    $display("ERROR %0d: %s", error_count, msg);
  endfunction

  class counter;
    static int created = 0;
    int        id;
    function new();
      created++;
      id = created;
    endfunction
  endclass

endpackage : util_pkg

module tb_package_basic;
  import util_pkg::*;

  data_t  d;
  level_e lvl;
  counter c1, c2;

  initial begin
    d = swap_nibbles(8'hA5);
    $display("1. swap_nibbles(A5) = %h, DATA_W = %0d", d, DATA_W);

    lvl = HIGH;
    $display("2. lvl = %s", lvl.name());

    report_error("first problem");
    util_pkg::report_error("second problem");
    $display("3. error_count = %0d", util_pkg::error_count);

    c1 = new();
    c2 = new();
    $display("4. c2.id = %0d, counter::created = %0d", c2.id, counter::created);
  end

endmodule
```

EXPECTED OUTPUT

```
1. swap_nibbles(A5) = 5a, DATA_W = 8
2. lvl = HIGH
ERROR 1: first problem
ERROR 2: second problem
3. error_count = 2
4. c2.id = 2, counter::created = 2
```

Line by line

Code	Explanation
`package util_pkg;`	Opens the package scope. Everything up to `endpackage` belongs to `util_pkg`.
`parameter int DATA_W = 8;`	A constant visible to every user. A parameter in a package cannot be overridden per instance (there are no instances); it behaves like a `localparam`.
`typedef logic [DATA_W-1:0] data_t;`	A shared type, built from the shared parameter.
`typedef enum {LOW, MEDIUM, HIGH} level_e;`	A shared enum. The labels `LOW`, `MEDIUM`, `HIGH` are names in the package scope too.
`int unsigned error_count = 0;`	A package variable. There is one copy in the whole simulation.

Code	Explanation
`function automatic ...`	Package functions should be declared `automatic` so that each call gets its own storage (M3). Synthesis tools require this for functions used in RTL.
`class counter`	A class may be defined in a package. This is how testbench classes are normally organised.
`import util_pkg::*;`	Wildcard import inside the module: all the names above may now be used without a prefix.
`d = swap_nibbles(8'hA5);`	Calls the package function; `A5` becomes `5A`.
`lvl = HIGH;`	The enum label is visible through the wildcard import.
`report_error(...)` and `util_pkg::report_error(...)`	The same function called through the import and through an explicit prefix. Both increment the same `error_count`, which is why the second message is numbered 2.
`util_pkg::error_count`	The explicit prefix always works, whether or not the package was imported.
`counter::created`	`::` applied to a class name reaches a static member (Section 1.3).

How a real testbench package is organised

Each class is kept in its own file, and the files are pulled into the package with `` `include ``. This gives one class per file for readability and version control, and one package for scoping.

```
// file: alu_tb_pkg.sv
package alu_tb_pkg;

  import alu_pkg::*;                // the DUT's types (alu_op_e)

  typedef class alu_scoreboard;     // forward declarations, if any are needed

  `include "alu_txn.sv"             // order matters: declare a class before it is used
  `include "alu_generator.sv"
  `include "alu_driver.sv"
  `include "alu_monitor.sv"
  `include "alu_scoreboard.sv"
  `include "alu_env.sv"
  `include "alu_test.sv"

endpackage : alu_tb_pkg
```

```
  Compile list (file list, in order):

     alu_pkg.sv        the design package
     alu.sv            the DUT
     alu_if.sv         the interface (not in a package: interfaces cannot be)
     alu_tb_pkg.sv     the testbench package; it `includes the class files itself
     tb_top.sv         the top module; imports alu_pkg and alu_tb_pkg

  The individual class files (alu_txn.sv ...) are NOT put on the compile list.
  They are compiled only as part of alu_tb_pkg, through `include.
```

`include versus import

These two are confused constantly, and the difference is a standard interview question.

Property	`` `include "file" ``	`import pkg::*;`
What it is	A compiler directive (preprocessor)	A language statement

Property	`` `include "file" ``	`import pkg::*;`
What it does	Pastes the text of the file at that point, before parsing	Makes names of an already-compiled package visible in the current scope
Creates declarations?	Yes: whatever the pasted text declares is declared again, here	No: it creates nothing, it only gives access
Used twice	The declarations exist twice (two different types, or a redefinition error)	Harmless; the package still exists once
Typical use	Pulling class files into a package; macro definition files	Using a package from a module, interface, class file or another package

> **Interview trap:** "If `alu_txn.sv` is `` `include ``d in package A and also in package B, can an `A::alu_txn` handle be assigned to a `B::alu_txn` handle?" No. The text was compiled twice, in two scopes, so there are two unrelated classes that merely have the same name. The simulator reports a type mismatch. A class file must be included in exactly one package, and everybody else must import that package.

The compilation unit scope: $unit

Anything declared outside every module, interface, package and class is in the **compilation unit scope**, called `$unit`.

```
typedef logic [7:0] byte_t;        // declared outside everything: lives in $unit

module m;
  byte_t        a;                 // visible here without any import
  $unit::byte_t b;                 // explicit reference to the compilation unit scope
endmodule
```

It looks convenient, but it should be avoided:

Problem with `$unit`	Explanation
Its contents depend on the tool's command line	A simulator may treat all files on the command line as one compilation unit, or each file as a separate one. The same code can compile in one mode and fail in the other
No name to refer to from another unit	A `$unit` declaration in file A is invisible in file B when files are separate compilation units
Name pollution	Every `$unit` name is visible in every following scope of that unit; collisions are easy and hard to trace
Wildcard imports at `$unit` level affect everything after them	`import pkg::*;` written above a module, outside it, is an import into `$unit`

> **Key point:** Put shared declarations in a named package. Put `import` statements inside the module, interface or package that needs them (or in the module header), not at file level.

The short examples in this course sometimes declare a class above the test module for brevity. That places the class in `$unit`. It is acceptable in a single-file demonstration and wrong in a project.

Packages and synthesis

Packages are synthesizable when their content is: types, parameters, and `automatic` functions without timing. That is how RTL teams share bus structs, opcode enums and width parameters between blocks. Classes, package variables used as globals, and tasks with delays are simulation-only.

> **Common mistake:** Declaring a variable in a package to pass information between testbench components "because it is easy". It works, but it is a global variable: any code can change it, the order of updates is hard to follow, and two instances of the same environment cannot have different values. Use it for genuinely global things only (an error counter, a verbosity level); pass everything else through constructors or configuration objects.

> **Common mistake:** Leaving a package off the compile list or listing it after the files that use it. The error message is usually "package not found" or "undeclared identifier" at the first `import`. Packages are compiled first, in dependency order.

> **Verification application:** A verification IP block for a protocol is delivered as one package, for example `apb_agent_pkg`, containing the transaction, driver, monitor and coverage classes. An environment for a chip imports the packages of all the agents it uses. The design delivers `*_pkg.sv` files with its types and parameters, which the testbench packages import, so that DUT and testbench can never disagree about an encoding.

Interview questions

Q1. What is a package? A top-level, named declarative scope that holds declarations to be shared between modules, interfaces, programs, classes and other packages: types, parameters, functions, tasks, classes and so on. It is compiled once and exists once.

Q2. What can a package not contain? Module, interface or program definitions or instances; procedural blocks such as `always` and `initial`; continuous assignments; hierarchical references to objects outside the package; and nested packages.

Q3. Is a package synthesizable? Yes, if its contents are: typedefs, enums, structs, parameters and automatic functions without timing control. Classes and other verification constructs in a package are simulation-only.

Q4. What is the difference between a package and a module? A module describes hardware or testbench structure: it has ports, can be instantiated many times, and contains processes. A package only holds declarations: it has no ports, cannot be instantiated, contains no processes, and exists as a single copy.

Q5. If a package contains a variable and two modules import the package, how many copies of the variable exist? One. Both modules read and write the same variable.

Q6. What is the difference between `` `include `` and `import`? `` `include `` is a preprocessor directive that pastes the text of a file into the current position, so whatever the file declares is declared there. `import` makes the names of a compiled package visible in the current scope without creating anything. Classes are normally `` `include ``d into one package, and that package is then imported wherever the classes are used.

Q7. What is `$unit` and why is it discouraged? `$unit` is the compilation unit scope: the anonymous scope that holds declarations written outside any module, interface or package. It is discouraged because what belongs to

one compilation unit depends on the tool and its options, so code relying on it is not portable, and because it pollutes the name space of every scope that follows.

Q8. Why do packages have to be compiled before the modules that use them? Because an `import` or a `pkg::name` reference is resolved at compile time. The compiler must already know the package scope and the declarations inside it.

1.2 Importing Packages

In plain English

Declaring something in a package does not make it visible anywhere else. A user of the package has three choices:

1. Write the full name every time: `alu_pkg::alu_op_e`.
2. Ask for one specific name to be made visible: `import alu_pkg::alu_op_e;`.
3. Ask for every name in the package to be made available when needed: `import alu_pkg::*;`.

The rules for what happens when names collide are different for choices 2 and 3, and those rules are the main interview topic here.

Technical definition

Form	Name	Effect
`pkg::name`	Explicit (qualified) reference	Uses the item directly. Nothing is added to the local scope. Always unambiguous
`import pkg::name;`	Explicit import	**Declares** `name` in the current scope as an alias of the package item. It is an error if `name` is already declared or explicitly imported in that scope
`import pkg::*;`	Wildcard import	Makes every name in the package a **candidate**. A candidate is actually imported only at the moment it is referenced and only if no local declaration or explicit import of that name is visible

How name lookup works

When the compiler meets an identifier `x` inside a scope, it searches in this order:

```
1. Declarations in the current scope that appear before this point
   (this includes names brought in by an EXPLICIT import in this scope)
2. Names available through a WILDCARD import in the current scope
3. Repeat steps 1 and 2 in the enclosing scope, then the next enclosing scope ...
4. The compilation unit scope ($unit), including its imports
5. Not found -> error (or, for a net in a port connection, an implicit wire; see M1)
```

From this order follow all the collision rules:

Situation	Result
Local declaration of `x` and a wildcard import that also offers `x`	Legal. The local `x` wins (step 1 before step 2). The package's `x` is still reachable as `pkg::x`

Situation	Result
Local declaration of `x` and an **explicit** import of `x` in the same scope	Error: `x` is declared twice in one scope
Two explicit imports of the same name from different packages	Error
Two wildcard imports whose packages both contain `x`, and `x` is **never used**	Legal. A candidate that is never referenced causes no problem
Two wildcard imports whose packages both contain `x`, and `x` **is used** without a prefix	Error: the reference is ambiguous. Fix with `pkg_a::x`
`x` is used through a wildcard import, and **later in the same scope** `x` is declared locally	Error: the use already imported `x` into the scope, so the later declaration collides
Explicit import of `x` and a wildcard import from another package that also offers `x`	Legal. The explicit import wins

Where an import may be placed

```
// (a) In the module header: visible to the parameter list, the port list and the body.
module alu
  import alu_pkg::*;
(
  input  alu_op_e op,              // the port list can use the imported type
  ...
);

// (b) Inside the module body: visible to the body only, NOT to the port list.
module tb_top;
  import alu_pkg::*;
  import alu_tb_pkg::*;
  ...
endmodule

// (c) Inside an interface, a program, a package, a task, a function or a begin-end block.
package alu_tb_pkg;
  import alu_pkg::*;
  ...
endpackage

// (d) Outside any design unit: an import into $unit. Avoid.
import alu_pkg::*;
module bad_style;  ...  endmodule
```

Placement	Scope of the import	Recommendation
Module / interface header	Whole design unit, including ports and parameters	Use when a port or parameter needs a package type
Inside the body	The body from that point on	The normal choice
Inside a package	That package only	Normal for a testbench package using a design package
At file level (`$unit`)	Everything that follows in the compilation unit	Avoid

Imports are not passed on: export

Importing package A into package B makes A's names visible **inside B only**. A module that imports B does not thereby see A's names.

```
package alu_pkg;
  typedef enum logic [2:0] {ALU_ADD, ALU_SUB /* ... */} alu_op_e;
endpackage

package alu_tb_pkg;
  import alu_pkg::*;
  class alu_txn;
    rand alu_op_e op;                // fine: alu_op_e is visible inside alu_tb_pkg
  endclass
endpackage

module tb_top;
  import alu_tb_pkg::*;
  alu_txn  t;                        // fine
  // alu_op_e op;                    // ERROR: alu_op_e is not a name of alu_tb_pkg
  alu_pkg::alu_op_e op;              // fine: explicit reference
endmodule
```

If a package wants to pass imported names on to its own users, it uses `export`:

```
package alu_tb_pkg;
  import alu_pkg::*;
  export alu_pkg::*;                 // re-export the alu_pkg names that this package uses
  // export *::*;                    // re-export everything imported from every package
  ...
endpackage
```

`export pkg::*` re-exports only those names of `pkg` that were actually imported, that is, referenced inside the exporting package. In practice most teams simply import both packages where both are needed.

The enum label trap

An explicit import of an enum **type** does not import its **labels**. The labels are separate names in the package.

```
module m1;
  import alu_pkg::alu_op_e;          // imports only the type name
  alu_op_e op;
  // initial op = ALU_ADD;           // ERROR: ALU_ADD is not visible
  initial op = alu_pkg::ALU_ADD;     // fine
endmodule

module m2;
  import alu_pkg::*;                 // type and labels are all candidates
  alu_op_e op;
  initial op = ALU_ADD;              // fine
endmodule
```

This is the main practical reason wildcard imports are the norm for packages that contain enums.

Complete example

tb_import_rules.sv

```
package a_pkg;
  int x = 1;
  int y = 10;
  typedef enum {RED, GREEN} colour_e;
endpackage

package b_pkg;
  int x = 2;
  int z = 30;
endpackage

module tb_import_rules;
  import a_pkg::*;
  import b_pkg::*;

  int y = 99;       // a local declaration hides the wildcard candidate a_pkg::y

  initial begin
    // $display("%0d", x);   // ILLEGAL: x is offered by two wildcard imports (ambiguous)
    $display("1. a_pkg::x = %0d, b_pkg::x = %0d", a_pkg::x, b_pkg::x);
    $display("2. y = %0d (local), a_pkg::y = %0d", y, a_pkg::y);
    $display("3. z = %0d", z);
    $display("4. GREEN = %0d", GREEN);
  end

endmodule
```

EXPECTED OUTPUT

```
1. a_pkg::x = 1, b_pkg::x = 2
2. y = 99 (local), a_pkg::y = 10
3. z = 30
4. GREEN = 1
```

Line by line

Code	Explanation
`import a_pkg::*; import b_pkg::*;`	Two wildcard imports. Both packages contain `x`. That alone is not an error.
`int y = 99;`	A local `y`. It was declared before any use of `y`, so it simply takes precedence over the candidate `a_pkg::y`.
The commented `$display` of `x`	Using the bare name `x` would be an error: the compiler finds it in two wildcard-imported packages and cannot choose.
Line 1	The explicit prefix resolves the ambiguity. Both values remain accessible.
Line 2	The bare `y` is the local one (99). The package's `y` is still reachable with the prefix (10).
Line 3	`z` exists only in `b_pkg`, so the wildcard import supplies it.
Line 4	The enum label `GREEN` is a name in `a_pkg` and comes in through the wildcard. Its value is 1.

Comparison of the three ways to use a package item

Property	`pkg::name`	`import pkg::name;`	`import pkg::*;`
Typing effort	Most	Medium	Least

Property	`pkg::name`	`import pkg::name;`	`import pkg::*;`
Adds a name to the local scope	No	Yes, immediately	Only when the name is referenced
Collision with a local declaration	Impossible	Compile error	Local declaration wins silently
Collision between two packages	Impossible	Compile error at the import	Error only if the name is used
Brings in enum labels	Each label must be prefixed	Only the names listed	Yes
Reader can see where a name comes from	Yes	Yes	Not directly
Typical use	One-off references, resolving ambiguity	Strict coding styles, RTL	Testbenches, packages with enums

Interview trap: "Module `m` does `import p::*;` and also declares its own `int count;`. Package `p` also has `count`. Is that an error?" No. The local declaration takes precedence over a wildcard import. It **would** be an error with `import p::count;`, because an explicit import is itself a declaration in the local scope.

Interview trap: "Does `import pkg::*;` import everything?" It imports nothing by itself. It makes all names *candidates*; each name is imported at the point where it is first referenced, provided nothing local has that name.

Common mistake: Placing `import alu_pkg::*;` inside the module body and then using `alu_op_e` in the port list. The port list comes before the body, so the type is unknown there. Put the import in the module header, between the module name and the port list.

Common mistake: Expecting names to pass through a chain of imports. `tb_top` importing `alu_tb_pkg` does not see `alu_op_e` unless it also imports `alu_pkg` or `alu_tb_pkg` re-exports it.

Interview questions

Q1. What are the ways of accessing an item in a package? Three: an explicit reference `pkg::item`; an explicit import `import pkg::item;`; and a wildcard import `import pkg::*;`.

Q2. What is the difference between an explicit import and a wildcard import? An explicit import declares that one name in the current scope immediately, so a clash with a local declaration or another explicit import is a compile error. A wildcard import only makes the package's names candidates; a name is imported when it is first referenced, and a local declaration of the same name takes precedence.

Q3. Two packages contain a function with the same name and both are wildcard-imported. What happens? Nothing, until the function is called without a prefix. Then the reference is ambiguous and compilation fails. Calling it as `pkg_a::func()` resolves it.

Q4. Where can an import statement be written? In a module, interface or program header (before the port list), in the body of a module, interface, program or package, inside tasks, functions and blocks, and at compilation-unit level. The header form is needed when ports or parameters use package types.

Q5. Package B imports package A. Module M imports B. Can M use A's types without a prefix? No. Imports are not transitive. M must import A as well, or B must re-export the names with `export A::*;`.

Q6. Why does `import alu_pkg::alu_op_e;` not let me write `ALU_ADD`? The explicit import brings in only the type name. Enum labels are separate identifiers in the package and must be imported explicitly, referenced as `alu_pkg::ALU_ADD`, or made available by a wildcard import.

Q7. What does `export` do? It makes names that a package has imported from another package available to importers of the first package, as if they were declared there. `export pkg::*;` re-exports the names taken from `pkg`; `export *::*;` re-exports all imported names.

1.3 The Scope Resolution Operator (::)

In plain English

Many things in SystemVerilog are containers of names: a package contains types and functions, a class contains properties, methods and types. The operator `::` means "the name on the right, **as declared inside** the container on the left". It is how you reach into a named scope from outside.

Technical definition

`<scope>::<name>` is the **scope resolution operator**. The left side is a package name, a class name (or a class type, including a specialised parameterized class or a `typedef` of one), `$unit`, or the keyword `local`. The right side is an identifier declared in that scope. It is resolved at compile time and refers to the **declaration**, not to any particular object.

The dot operator `.` is different: its left side is an **instance** (an object handle, a struct variable, a module or interface instance), and it selects a member of that instance.

Every use of :: in one table

Use	Example	Covered in
Package item	`alu_pkg::alu_op_e`, `alu_pkg::ALU_ADD`, `util_pkg::report_error()`	Sections 1.1, 1.2
Wildcard and explicit import	`import alu_pkg::*;`	Section 1.2
Static property of a class	`alu_txn::count`	M4
Static method of a class	`alu_txn::get_count()`	M4
Type, enum label, parameter or constant declared inside a class	`packet::kind_e`, `packet::LONG`, `packet::HDR_LEN`	Below
Defining a method outside its class (`extern`)	`function void alu_txn::print();`	M4
Defining a constraint outside its class	`constraint alu_txn::c_small { a < 16; }`	M5
Reaching a hidden base-class member by class name	`base_txn::print()` from inside a derived class	M4
Static member of a parameterized class specialisation	`stack #(int)::instances`	Section 2.1

Use	Example	Covered in
Compilation unit scope	`$unit::byte_t`	Section 1.1
Built-in `std` package	`std::randomize(v)`, `std::process`	M5, Section 3.4
Caller's scope inside `randomize() with {}`	`obj.randomize() with { a == local::a; }`	M5
Covergroup type options and static coverage methods	`alu_cg::type_option.goal = 90;`, `alu_cg::get_coverage()`	M10
UVM factory	`alu_seq_item::type_id::create("req")`	Section 5.5

The last row is the reason UVM code is full of `::`. `type_id` is a type declared inside the class (by a macro), and `create` is a static method of that type. So `alu_seq_item::type_id::create(...)` is simply "class scope, then nested class scope, then static method".

How it works internally

```
   p.len                       packet::count
   -----                       -------------
   1. take the handle p        1. find the class scope "packet" (compile time)
   2. follow it to the object  2. find the declaration "count" in that scope
   3. read member len of       3. it is static, so it has one storage location that belongs
      THAT object                 to the class itself: read it. No object is needed.

   Run-time operation;         Compile-time name resolution;
   fails if p is null.         works even if no object has ever been created.
```

Only things that exist without an object can be reached through a class name with `::` from outside the class: static properties, static methods, types, enum labels, parameters and constants. A non-static property has no storage until an object is constructed, so `packet::len` from a module is an error.

Complete example

tb_scope_resolution.sv

```
package cfg_pkg;
  parameter int MAX_LEN = 16;
  function automatic int twice(int v);
    return 2 * v;
  endfunction
endpackage

class packet;
  typedef enum {SHORT, LONG} kind_e;      // a type declared inside the class
  localparam int HDR_LEN = 4;             // a constant declared inside the class
  static int count = 0;                   // one copy for the whole class

  kind_e kind;
  int    len;

  extern function new(kind_e kind, int len);
  extern function void print();

  static function int get_count();
    return count;
  endfunction
endclass

function packet::new(kind_e kind, int len);      // constructor defined outside the class
  this.kind = kind;
```

```
  this.len  = len;
  count++;
endfunction

function void packet::print();                    // method defined outside the class
  $display("   packet kind=%s len=%0d", kind.name(), len);
endfunction

module tb_scope_resolution;

  packet         p;
  packet::kind_e k;       // class-scoped type
  int            v;

  initial begin
    $display("1. cfg_pkg::MAX_LEN = %0d, cfg_pkg::twice(5) = %0d",
             cfg_pkg::MAX_LEN, cfg_pkg::twice(5));
    $display("2. packet::count = %0d, packet::HDR_LEN = %0d",
             packet::count, packet::HDR_LEN);

    k = packet::LONG;                   // class-scoped enum label
    p = new(k, 12);
    p = new(packet::SHORT, 3);
    p.print();
    $display("3. packet::get_count() = %0d", packet::get_count());

    void'(std::randomize(v) with { v inside {[1:3]}; });
    $display("4. v is in the range 1 to 3: %b", v inside {[1:3]});
  end

endmodule
```

EXPECTED OUTPUT

```
1. cfg_pkg::MAX_LEN = 16, cfg_pkg::twice(5) = 10
2. packet::count = 0, packet::HDR_LEN = 4
   packet kind=SHORT len=3
3. packet::get_count() = 2
4. v is in the range 1 to 3: 1
```

Line by line

Code	Explanation
`cfg_pkg::MAX_LEN`, `cfg_pkg::twice(5)`	Package scope. No import was written, and none is needed with the prefix.
`typedef enum {SHORT, LONG} kind_e;` inside the class	The type and its labels belong to the class scope. Outside the class they must be written `packet::kind_e`, `packet::SHORT`.
`extern function new(...)`, `extern function void print();`	Prototypes only. The bodies are written outside the class.
`function packet::new(...)`	`packet::` tells the compiler which class this body belongs to. Inside the body the class scope applies, so `kind_e`, `count` and `this` are used without a prefix.
`packet::count`, `packet::HDR_LEN` on line 2	A static property and a class constant read through the class name. No `packet` object exists yet, and `count` is 0.
`k = packet::LONG;`	The enum label, qualified by the class.
`p = new(k, 12); p = new(packet::SHORT, 3);`	Two objects are constructed; each constructor call increments the shared `count`. The first object is no longer referenced after the second assignment.
`p.print();`	The dot operator: `p` is a handle, `print` runs on the object it refers to (the second one).
`packet::get_count()`	A static method called through the class name. It returns 2.

Code	Explanation
`std::randomize(v) with {...}`	`std` is the built-in package that every SystemVerilog scope sees. `std::randomize` is the scope-level randomize function from M5. The value differs per run, but it always satisfies the constraint, so line 4 always prints 1.

:: versus .

Property	`::` scope resolution	`.` member / hierarchy selection
Left operand	A scope name: package, class, `$unit`, `local`	An instance: object handle, struct or union variable, interface or module instance, virtual interface
Resolved	At compile time	Member selection on a handle happens at run time
Needs an object	No	Yes for class members (a null handle causes a run-time error)
Reaches	Static properties and methods, types, enum labels, parameters, package items	Any accessible member of the instance, static or not
Example	`alu_txn::count`, `alu_pkg::ALU_ADD`	`txn.a`, `vif.drv_cb.valid_in`, `tb_top.dut.result`

A static member can be reached both ways: `packet::count` and `p.count` refer to the same storage. The `::` form is preferred because it shows the reader that the member is static and it does not depend on a handle being valid.

> **Interview trap:** "Can you call a non-static method with `::` ?" From outside the class, no: `packet::print()` in a module is an error, because `print` needs an object. The one legitimate non-static use is *inside* a derived class, where `base_class::method()` selects the base-class version of a member that the derived class has hidden; it behaves like `super.method()` but can name any ancestor.

> **Common mistake:** Writing `packet.count` or `alu_pkg.ALU_ADD`. A class name or a package name is not an instance, so the dot operator does not apply to it.

> **Common mistake:** Forgetting the class prefix on the *return type* of an out-of-block method. In `function kind_e packet::get_kind();` the return type is read before the compiler sees `packet::`, so `kind_e` is unknown at that point. Write `function packet::kind_e packet::get_kind();`. Argument types come after `packet::` and do not need the prefix.

Interview questions

Q1. What is the scope resolution operator and where is it used? `::` selects a name from a named scope. It is used to access package items (`pkg::item`), static properties and methods of a class (`cls::member`), types, enum labels and parameters declared inside a class, to define `extern` methods and constraints outside the class body (`function void cls::method();`), to access `$unit` and the `std` package, and to name a specialisation's static members in a parameterized class.

Q2. What is the difference between `::` and `.`? `::` works on a scope name and is resolved at compile time; it needs no object. `.` works on an instance (a handle, a struct, an interface instance) and selects a member of that instance.

Q3. How do you access a static variable of a class without creating an object? `class_name::variable_name`.

Q4. How do you define a class method outside the class? Declare the prototype inside the class with `extern`, then write the body outside as `function <return_type> <class_name>::<method_name>(<arguments>); ... endfunction`.

Q5. A class declares `typedef enum {RD, WR} dir_e;`. How does a module declare a variable of that type and assign `WR`? `my_class::dir_e d; d = my_class::WR;`

Q6. Explain `alu_seq_item::type_id::create("req")`. `alu_seq_item` is a class. `type_id` is a type declared inside that class by the UVM registration macro. `create` is a static method of `type_id`. The two `::` operators walk from the class scope into the nested type's scope and call its static method, which asks the UVM factory to construct the object.

Q7. What does `local::` mean in `randomize() with { addr == local::addr; }`? Inside the `with` block, names are looked up first in the object being randomized. `local::addr` forces the name to be resolved in the scope that contains the `randomize()` call, so the constraint compares the object's `addr` with the caller's `addr`.

Part 2: Generic Code

2.1 Parameterized Classes

In plain English

Suppose you write a class that is a stack of integers. Next week you need a stack of strings, then a stack of transactions. The logic (push, pop, full, empty) is identical every time; only the **type of the thing stored** changes. Copying the class and editing the type by hand gives three classes to maintain.

A **parameterized class** is a class written with a blank in it. The blank is filled in when the class is used: "a stack of `int`", "a stack of `string`, at most 16 deep". You write the logic once.

You have already used one: `mailbox #(alu_txn)` from M7 is the built-in `mailbox` class with its type blank filled in by `alu_txn`.

Technical definition

A parameterized class is a class declared with a **parameter port list** `#( ... )`. Parameters are of two kinds:

- a **value parameter**, a compile-time constant such as a width or a depth: `int DEPTH = 8`;
- a **type parameter**, introduced by the keyword `type`, which stands for a data type: `type T = int`.

Supplying actual values for the parameters is called **specialising** the class. Each distinct combination of parameter values produces a distinct class type, called a **specialisation**. The parameterized class itself is called a **generic class**; it is a template, not a type.

Why it is needed

Need	Without parameters	With parameters
Same container for different element types	One copy of the class per type	`stack #(int)`, `stack #(string)`, `stack #(alu_txn)`
Same component for different bus widths	One copy per width, or the widest width everywhere	`bus_driver #(32)`, `bus_driver #(64)`
Type safety	A typeless container accepts anything; mistakes appear at run time	The compiler rejects a wrong type at compile time
Reusable verification libraries	Impossible without knowing the user's types	The user's transaction type is passed as a parameter. All of UVM's TLM ports, `uvm_driver`, `uvm_sequencer` and `uvm_config_db` are parameterized classes

Syntax

```
// Declaration
class stack #(type T = int, int DEPTH = 8);
  T items [$];
  ...
endclass

// Use (specialisation)
stack #(byte, 16)            s1;    // parameters by position
stack #(.T(string))          s2;    // parameters by name; DEPTH keeps its default (8)
stack #()                    s3;    // all defaults: T = int, DEPTH = 8
stack                        s4;    // also the default specialisation, when every
                                    // parameter has a default
typedef stack #(alu_txn, 32) txn_stack_t;      // a short name for a specialisation
txn_stack_t                  s5;
```

Rules:

- Parameters are constants fixed at compile (elaboration) time. They cannot be changed at run time and cannot depend on run-time values.
- A parameter without a default value must be supplied in every specialisation.
- Inside the class, a type parameter is used exactly like a type name, and a value parameter exactly like a `localparam`.
- The keyword `parameter` is optional in the list: `#(parameter type T = int)` and `#(type T = int)` are the same.

How it works internally

```
  Source                                  What the compiler creates
  ------                                  -------------------------
  class stack #(type T = int,             (a template: no code is generated for it)
                int DEPTH = 8); ...

  stack #(int, 8)     a;   ----+
  stack               b;   ----+----->   class stack#(int,8)      one static "instances"
  stack #()           c;   ----+

  stack #(string, 2)  d;   ---------->   class stack#(string,2)   its own "instances"

  stack #(int, 16)    e;   ---------->   class stack#(int,16)     its own "instances"

  - Same parameter values  -> the SAME class type  -> handles are assignment compatible.
  - Different values       -> DIFFERENT, unrelated class types -> handles NOT compatible.
  - Static members exist once PER SPECIALISATION, not once for the generic class.
```

The compiler generates a separate class for every distinct set of parameter values that the code uses, as if you had written each one by hand. This has three consequences that interviewers probe:

1. **Specialisations are unrelated types.** `stack #(int)` and `stack #(string)` have no common base class. A handle of one cannot be assigned to, or `$cast` to, the other.
2. **Static members are per specialisation.** A `static int instances` in the generic class becomes one variable in `stack #(int, 8)` and a different variable in `stack #(string, 2)`.
3. **Errors may appear only when a specialisation is created.** If the class body calls `item.print()`, the generic class compiles, `stack #(alu_txn)` compiles (the transaction has `print`), and `stack #(int)` fails, because an `int` has no methods.

Complete example

tb_param_class.sv

```
class stack #(type T = int, int DEPTH = 4);

  static int instances = 0;
  local  T   items [$];

  function new();
    instances++;
  endfunction

  function bit push(T item);
    if (items.size() >= DEPTH) return 0;      // full
    items.push_back(item);
    return 1;
  endfunction

  function bit pop(output T item);
    if (items.size() == 0) return 0;          // empty
    item = items.pop_back();
    return 1;
  endfunction

  function int size();
    return items.size();
  endfunction

endclass

module tb_param_class;

  typedef stack #(string, 2) str_stack_t;

  stack           int_stk;      // default specialisation: T = int, DEPTH = 4
  stack #(int, 4) int_stk2;     // the same type as int_stk
  str_stack_t     str_stk;

  int    i;
  string s;

  initial begin
    int_stk  = new();
    int_stk2 = new();
    str_stk  = new();

    for (int k = 1; k <= 5; k++)
      if (!int_stk.push(k * 10)) $display("1. push(%0d) rejected: stack is full", k * 10);

    void'(int_stk.pop(i));
    $display("2. popped %0d, size is now %0d", i, int_stk.size());

    void'(str_stk.push("alpha"));
    void'(str_stk.push("beta"));
    if (!str_stk.push("gamma")) $display("3. push(gamma) rejected: DEPTH is 2");
    void'(str_stk.pop(s));
    $display("4. popped %s", s);

    $display("5. stack #(int, 4)::instances = %0d", stack #(int, 4)::instances);
    $display("6. str_stack_t::instances     = %0d", str_stack_t::instances);

    int_stk2 = int_stk;         // legal: both handles have the same specialisation
    // str_stk = int_stk;       // ILLEGAL: different specialisations are different types
  end

endmodule
```

EXPECTED OUTPUT

```
1. push(50) rejected: stack is full
2. popped 40, size is now 3
3. push(gamma) rejected: DEPTH is 2
4. popped beta
5. stack #(int, 4)::instances = 2
6. str_stack_t::instances     = 1
```

Line by line

Code	Explanation
`class stack #(type T = int, int DEPTH = 4);`	One type parameter and one value parameter, both with defaults.
`static int instances = 0;`	A static counter. It exists once per specialisation.
`local T items [$];`	A queue whose element type is the parameter. `local` hides it from users of the class (M4).
`function bit push(T item);`	The argument type is the parameter. For `stack #(string)` this is `push(string item)`.
`if (items.size() >= DEPTH) return 0;`	The value parameter is used like a constant.
`function bit pop(output T item);`	Returns the status as the function value and the popped element through an `output` argument.
`typedef stack #(string, 2) str_stack_t;`	A name for the specialisation "stack of strings, depth 2".
`stack int_stk;` and `stack #(int, 4) int_stk2;`	Both are the specialisation `T = int, DEPTH = 4` (the second one just writes the defaults out), so they are the same type.
The `for` loop, line 1	Pushes 10, 20, 30, 40. The fifth push finds four items and `DEPTH` = 4, so it returns 0.
Line 2	A stack is last-in first-out: the last value pushed, 40, comes out. Three remain.
Lines 3 and 4	The string stack has `DEPTH` = 2, so the third push is rejected. `beta` was pushed last and is popped first.
Line 5	Two objects of the `(int, 4)` specialisation were constructed (`int_stk` and `int_stk2`), so its counter is 2.
Line 6	One object of the `(string, 2)` specialisation was constructed. Its counter is separate.
`int_stk2 = int_stk;`	Allowed because the types are identical.
`// str_stk = int_stk;`	Would be a compile error: the two classes are unrelated.

The unadorned-name rule for ::

Inside a declaration, the bare name `stack` means the default specialisation. With the scope resolution operator it does not: from outside the class, `stack::instances` is illegal, because the compiler cannot tell which specialisation is meant.

```
stack #()::instances              // legal: the default specialisation, written explicitly
stack #(int, 4)::instances        // legal
str_stack_t::instances            // legal: through a typedef
// stack::instances               // ILLEGAL outside the class
```

Parameterized classes and inheritance

```
class base_item;                                  // an ordinary class
endclass

// 1. A parameterized class derived from an ordinary class.
//    Every specialisation shares the base, so base_item handles can refer to any of them.
class typed_item #(type T = int) extends base_item;
  T value;
endclass

// 2. A parameterized class derived from a parameterized class; parameters are passed up.
class bounded_stack #(type T = int, int DEPTH = 8) extends stack #(T, DEPTH);
endclass

// 3. An ordinary class derived from one specialisation.
class int_stack extends stack #(int, 16);
endclass

// 4. The base class itself is a parameter (a "mixin").
class with_logging #(type BASE = base_item) extends BASE;
  function void log(string msg);  $display("[LOG] %s", msg);  endfunction
endclass
```

Pattern 1 is the standard answer to "specialisations are unrelated, so how do I store them in one array?": give them a common **non-parameterized base class** and declare the array with the base type. UVM uses this everywhere; for example every `uvm_sequencer #(REQ)` extends the non-parameterized `uvm_sequencer_base`.

Static utility methods

A parameterized class with only static methods acts as a generic function library. No object is ever created; the methods are called through the specialisation name.

```
class math_util #(type T = int);
  static function T max(T a, T b);
    return (a > b) ? a : b;
  endfunction
  static function T min(T a, T b);
    return (a < b) ? a : b;
  endfunction
endclass

// usage
int  m = math_util #(int)::max(3, 7);          // 7
real r = math_util #(real)::min(2.5, 1.5);     // 1.5
```

This is exactly the shape of UVM's `uvm_config_db #(T)::set(...)` and `uvm_config_db #(T)::get(...)` (Section 5.6): a parameterized class used only for its static methods.

Value parameter versus type parameter

Property	Value parameter	Type parameter
Declared as	`int WIDTH = 8`, `bit SIGNED = 0`	`type T = int`
Stands for	A constant value	A data type
Used inside the class as	A constant (array bounds, widths, limits)	A type (variables, arguments, return types, queue elements)
Typical use	Bus width, depth, number of channels	Transaction type, element type, base class

Property	Value parameter	Type parameter
Example specialisation	`bus_driver #(32)`	`mailbox #(alu_txn)`

Parameterized class versus related constructs

Construct	What varies	Decided	Example
Parameterized module	Widths, depths, types of hardware	Elaboration	`fifo #(.DEPTH(16)) u_fifo (...)`
Parameterized class	Types and constants of a class	Compile / elaboration	`stack #(alu_txn)`
Inheritance and polymorphism (M4)	Behaviour, through virtual methods	Run time (which object the handle refers to)	`base_txn h = new_derived;`
`typedef`	Nothing; it only names a type	Compile	`typedef stack #(alu_txn) txn_stack_t;`

Parameterization and polymorphism solve different problems. Parameterization gives **the same code for different types**, fixed at compile time. Polymorphism gives **different code behind the same handle type**, chosen at run time. Real libraries use both together.

Interview trap: "A parameterized class has `static int count`. `C #(int)` is constructed three times and `C #(bit)` twice. What is `count`?" There is no single `count`. `C #(int)::count` is 3 and `C #(bit)::count` is 2. If one shared counter is wanted, put it in a non-parameterized base class.

Interview trap: "Can a `stack #(int)` handle be assigned to a `stack #(byte)` handle with `$cast`?" No. They are different classes with no inheritance relationship, so the cast always fails. `$cast` only succeeds along an inheritance chain.

Common mistake: Forgetting the `#` and writing `stack (int) s;`, or writing `mailbox (alu_txn) m;`. The parameter list always starts with `#(`.

Common mistake: Writing a `typedef` in two places with different parameter spellings and expecting different types. `stack #(int, 4)` and `stack #(.DEPTH(4), .T(int))` and `stack` are all the same type; what counts is the final parameter values.

Verification application: The scoreboard, the mailboxes and the analysis FIFOs of a testbench are written once and parameterized by the transaction type, so the same code checks the ALU (`#(alu_txn)`), a FIFO (`#(fifo_txn)`) and a bus (`#(apb_txn)`). A driver for a bus that exists in 32-bit and 64-bit versions is parameterized by the address and data widths.

Interview questions

Q1. What is a parameterized class? A class declared with a parameter list `#(...)` containing value parameters and/or type parameters. It is a template from which the compiler generates a concrete class for each set of actual parameter values used.

Q2. What is the difference between a value parameter and a type parameter? A value parameter is a compile-time constant such as a width or depth. A type parameter, declared with the keyword `type`, represents a data type that is supplied when the class is used.

Q3. What is a specialisation? A parameterized class together with a specific set of parameter values, for example `stack #(string, 2)`. Each distinct set of values is a distinct class type.

Q4. Are `stack #(int)` and `stack #(byte)` compatible types? No. Different specialisations are separate, unrelated classes. Handles cannot be assigned between them, even with `$cast`.

Q5. How do static members behave in a parameterized class? Each specialisation gets its own copy of every static property. They are not shared between specialisations.

Q6. How would you keep objects of different specialisations in one queue? Derive the parameterized class from a common non-parameterized base class and declare the queue with the base class type. Virtual methods in the base give access to the specialised behaviour.

Q7. How do you access a static member of a parameterized class? Through an explicit specialisation, `stack #(int)::instances` or `stack #()::instances` for the default, or through a typedef of the specialisation. The bare generic name with `::` is not allowed outside the class.

Q8. Give examples of parameterized classes you have used. `mailbox #(alu_txn)` is the built-in example. In UVM: `uvm_driver #(REQ)`, `uvm_sequence #(REQ)`, `uvm_sequencer #(REQ)`, `uvm_analysis_port #(T)`, `uvm_tlm_analysis_fifo #(T)` and `uvm_config_db #(T)`.

Q9. Can the parameters of a class be changed at run time? No. They are elaboration-time constants. A run-time choice is made with polymorphism or with ordinary class properties, not with parameters.

2.2 Generic Reusable Classes

In plain English

"Generic" means "written without knowing the exact type it will work on". The stack in Section 2.1 is generic. This section builds two generic classes of the kind that appear in real testbenches, a FIFO and a scoreboard, and discusses what a generic class may assume about the type it is given.

What a generic class can do with a value of type T

Inside the class, the compiler knows nothing about `T` until a specialisation is made. The class may therefore only use operations that the eventual type supports:

Operation in the generic class	Works when T is	Fails when T is
Assignment `x = item;`, storing in a queue or array	Any type	
`==`, `!=`	Integral types, strings, unpacked structs and arrays (compares contents); class handles (compares **handles**, not contents)	
`<`, `>`, `+`	Integral and real types	Class handles, unpacked structs

Operation in the generic class	Works when T is	Fails when T is
`item.compare(other)`, `item.print()`, `item.copy()`	A class that has those methods	`int`, structs, any class without them
`$display("%p", item)`	Any type	

Two design styles follow from this table:

- **Value-type generic classes** use only assignment and `==`. They work for integers, strings and structs. They also compile for class handles, but then `==` compares handle identity, which is rarely what a scoreboard wants.
- **Object-type generic classes** require that `T` is a class with an agreed set of methods (`compare`, `copy`, `print`). This agreement is an informal interface: the generic class will not compile for a type that lacks the methods.

Generic FIFO

tb_generic_fifo.sv

```
class generic_fifo #(type T = int, int DEPTH = 4);

  protected T q [$];

  function bit is_full();   return (q.size() == DEPTH);  endfunction
  function bit is_empty();  return (q.size() == 0);      endfunction
  function int level();     return q.size();             endfunction

  function bit push(T item);
    if (is_full()) return 0;
    q.push_back(item);
    return 1;
  endfunction

  function bit pop(output T item);
    if (is_empty()) return 0;
    item = q.pop_front();
    return 1;
  endfunction

endclass

module tb_generic_fifo;

  typedef struct {
    bit [7:0] addr;
    bit [7:0] data;
  } bus_item_t;

  generic_fifo #(bit [7:0], 3) byte_fifo;
  generic_fifo #(bus_item_t)   item_fifo;

  bit [7:0]  b;
  bus_item_t it;

  initial begin
    byte_fifo = new();
    item_fifo = new();

    for (int i = 0; i < 4; i++) begin
      b = 8'hA0 + i;
      if (!byte_fifo.push(b)) $display("1. push of %h refused, FIFO full", b);
    end
    $display("2. level = %0d, full = %b", byte_fifo.level(), byte_fifo.is_full());

    while (byte_fifo.pop(b)) $display("3. popped %h", b);
    $display("4. empty = %b", byte_fifo.is_empty());
```

```
    void'(item_fifo.push('{addr: 8'h10, data: 8'hAA}));
    void'(item_fifo.push('{addr: 8'h14, data: 8'hBB}));
    void'(item_fifo.pop(it));
    $display("5. item: addr=%h data=%h, %0d left", it.addr, it.data, item_fifo.level());
  end

endmodule
```

EXPECTED OUTPUT

```
1. push of a3 refused, FIFO full
2. level = 3, full = 1
3. popped a0
3. popped a1
3. popped a2
4. empty = 1
5. item: addr=10 data=aa, 1 left
```

Line by line

Code	Explanation
`protected T q [$];`	The storage is a queue of `T`. `protected` lets a derived class reach it while hiding it from outside users.
`push` / `pop`	`push_back` and `pop_front` give first-in first-out order (the stack used `pop_back`).
`generic_fifo #(bit [7:0], 3) byte_fifo;`	A type parameter can be any type expression, including a vector type written in place.
`generic_fifo #(bus_item_t) item_fifo;`	The same code storing structs. `DEPTH` takes its default, 4.
The `for` loop, line 1	Three pushes succeed (`a0`, `a1`, `a2`). The fourth (`a3`) finds the FIFO full and `push` returns 0.
`while (byte_fifo.pop(b))`	Pops until the function returns 0. Values come out in the order they went in.
`'{addr: 8'h10, data: 8'hAA}`	An assignment pattern passed directly as the argument; the formal type `bus_item_t` gives it its context.

Generic scoreboard for value types

tb_generic_scoreboard.sv

```
class inorder_scoreboard #(type T = int);

  protected T  expected_q [$];
  int unsigned matches;
  int unsigned mismatches;
  string       name;

  function new(string name);
    this.name = name;
  endfunction

  function void add_expected(T item);
    expected_q.push_back(item);
  endfunction

  function void check_actual(T actual);
    T exp;
    if (expected_q.size() == 0) begin
      mismatches++;
      $display("[%s] FAIL: item %0d was not expected", name, matches + mismatches);
```

```
      return;
    end
    exp = expected_q.pop_front();
    if (actual == exp) matches++;
    else begin
      mismatches++;
      $display("[%s] FAIL: item %0d does not match", name, matches + mismatches);
    end
  endfunction

  function void report();
    $display("[%s] matches=%0d mismatches=%0d still expected=%0d",
             name, matches, mismatches, expected_q.size());
  endfunction

endclass

module tb_generic_scoreboard;

  typedef struct {
    bit [7:0] addr;
    bit [7:0] data;
  } bus_item_t;

  inorder_scoreboard #(int)        int_sb;
  inorder_scoreboard #(bus_item_t) bus_sb;
  bus_item_t item;

  initial begin
    int_sb = new("int_sb");
    bus_sb = new("bus_sb");

    int_sb.add_expected(5);
    int_sb.add_expected(6);
    int_sb.add_expected(7);
    int_sb.check_actual(5);
    int_sb.check_actual(9);
    int_sb.report();

    item = '{addr: 8'h10, data: 8'hAA};
    bus_sb.add_expected(item);
    bus_sb.check_actual(item);
    item.data = 8'hAB;
    bus_sb.check_actual(item);
    bus_sb.report();
  end

endmodule
```

EXPECTED OUTPUT

```
[int_sb] FAIL: item 2 does not match
[int_sb] matches=1 mismatches=1 still expected=1
[bus_sb] FAIL: item 2 was not expected
[bus_sb] matches=1 mismatches=1 still expected=0
```

Line by line

Code	Explanation
`class inorder_scoreboard #(type T = int);`	"In-order" means the actual items must arrive in the same order as the expected items.
`add_expected`	Called by whatever predicts the DUT output; appends to the queue.
`T exp;`	A local variable of the parameter type.
`if (expected_q.size() == 0)`	An actual item arrived with nothing expected: that is an error too.

Code	Explanation
`if (actual == exp) matches++; else ...`	The comparison is written so that the failing path is the `else`. If `T` is four-state and the actual value contains X, `==` gives X, the `if` is not taken, and the item is counted as a mismatch (see M1, Section 4.5).
`int_sb.check_actual(5);`	5 equals the first expected value: match.
`int_sb.check_actual(9);`	9 is compared with 6: mismatch on the second item. One expected value (7) is left in the queue.
`bus_sb`	The same class checking structs. `==` on unpacked structs compares member by member. The first check matches. The second arrives when the queue is empty, so it is reported as unexpected.

Generic scoreboard for transaction objects

For class types, `==` would compare handles. An object-type scoreboard calls the transaction's own `compare` method instead (M6 gave `alu_txn` the methods `print`, `copy` and `compare`):

```
class txn_scoreboard #(type T = alu_txn);

  mailbox #(T) exp_mbx;        // from the reference model
  mailbox #(T) act_mbx;        // from the monitor
  int unsigned n_pass, n_fail;

  function new(mailbox #(T) exp_mbx, mailbox #(T) act_mbx);
    this.exp_mbx = exp_mbx;
    this.act_mbx = act_mbx;
  endfunction

  task run();
    T exp, act;
    forever begin
      act_mbx.get(act);
      exp_mbx.get(exp);
      if (act.compare(exp)) n_pass++;        // requires: T has function bit compare(T rhs)
      else begin
        n_fail++;
        act.print("ACTUAL  ");               // requires: T has function void print(string)
        exp.print("EXPECTED");
      end
    end
  endtask

endclass
```

`txn_scoreboard #(alu_txn)` compiles because `alu_txn` provides `compare` and `print`. `txn_scoreboard #(int)` does not compile. The comments "requires:" document the contract.

A stricter way to state the contract is an abstract base class (M4): declare `virtual class base_txn` with `pure virtual compare` and `print`, make every transaction extend it, and write the scoreboard against `base_txn` handles. Then the compiler enforces the contract through inheritance instead of failing inside the generic class. UVM does this with `uvm_object`, whose `compare`, `copy`, `print` and `clone` every transaction inherits.

The parameterized mailbox (recap from M7)

Declaration	Meaning	Type checking
`mailbox mbx;`	Typeless mailbox: can carry values of any type	None at compile time. A `get` into a variable of the wrong type is a **run-time** error

Declaration	Meaning	Type checking
`mailbox #(alu_txn) mbx;`	The built-in parameterized class specialised for `alu_txn` handles	A `put` or `get` with the wrong type is a **compile** error
`typedef mailbox #(alu_txn) alu_txn_mbx_t;`	A reusable name for that specialisation	

`mailbox #(alu_txn)` and `mailbox #(fifo_txn)` are different types, exactly as for any parameterized class, so the handle a generator holds must be declared with the same specialisation as the handle the driver holds.

> **Common mistake:** Building a generic scoreboard with `==` and then specialising it with a class type. It compiles, and every comparison fails, because two different objects with identical contents have different handles. Use the object-type style with `compare()` for classes.

> **Common mistake:** Making every class parameterized "for reuse". Parameters spread: a parameterized transaction forces the driver, monitor, scoreboard, environment and test that use it to be parameterized or specialised too. Parameterize where types or widths really vary, and use `typedef`s to hide the specialisations.

> **Verification application:** Predictor-plus-comparator scoreboards, analysis FIFOs, register-model adapters and coverage collectors are all written generically and specialised by transaction type. Writing a small generic class on a whiteboard (a stack, a FIFO, a pair) is a common interview coding question.

Interview questions

Q1. What is a generic class? A class written in terms of type (and value) parameters instead of fixed types, so that the same code can be reused for many types by specialising it.

Q2. Write a generic class that swaps two values of any type. `class swapper #(type T = int); static function void swap(ref T a, ref T b); T tmp = a; a = b; b = tmp; endfunction endclass`, used as `swapper #(string)::swap(s1, s2);`. The `ref` arguments (M3) let the function modify the caller's variables.

Q3. A generic scoreboard uses `if (actual == expected)`. What happens when it is specialised with a transaction class? It compiles, but `==` on class handles compares whether both handles refer to the same object, not whether the contents are equal. Two separate objects with equal fields compare as different. The scoreboard should call a `compare()` method instead.

Q4. How can a generic class require that its type parameter has certain methods? Informally, by calling them: a specialisation with a type lacking the method fails to compile. Formally, by writing the class against an abstract base class with pure virtual methods that every usable type must extend.

Q5. What is the advantage of `mailbox #(alu_txn)` over a plain `mailbox`? Type safety at compile time. A plain mailbox accepts any type, and a type mismatch is only discovered at run time when `get` fails. The parameterized mailbox rejects a wrong type during compilation.

2.3 Virtual Interfaces: Advanced Recap

Module 8 introduced virtual interfaces in full. This section restates the essentials in the form an interviewer expects and adds the topics that belong with parameterization.

The essentials

Fact	Explanation
Why they exist	A class is created at run time; an interface instance is static hardware-like structure created at elaboration. A class cannot instantiate an interface or have ports. It needs a variable that can **refer to** an existing interface instance
What it is	`virtual alu_if vif;` declares a variable (a handle) that can refer to any instance of `alu_if`. The keyword `virtual` here has nothing to do with virtual methods
Default value	`null`. Using it before assignment is a run-time null-reference error
Assignment	In the top module, where the real instance exists: `env = new(alu_vif);` or `env.drv.vif = alu_vif;`
With a modport	`virtual alu_if.DRV vif;` restricts the class to the signals and directions of that modport
Access	`vif.rst_n`, `vif.drv_cb.a <= txn.a;`, `@(vif.drv_cb);`
Synthesizable	No. Testbench only

```
   tb_top (static, elaboration time)                class world (dynamic, run time)

   alu_if alu_vif (clk);  <------------------------  virtual alu_if vif;   (in alu_driver)
        |      ^                                      virtual alu_if vif;   (in alu_monitor)
        |      |   the handles refer to the SAME interface instance
        v      |
   alu dut (.a(alu_vif.a), ...);
```

Parameterized interfaces

If the interface has parameters, the virtual interface type must state them, and they must match the instance it is assigned from. A class that should work with any width must then be parameterized itself and pass its parameter on.

tb_param_vif.sv

```
interface bus_if #(parameter int W = 8) (input logic clk);
  logic [W-1:0] data;
  logic         valid;
endinterface

class bus_driver #(int W = 8);

  virtual bus_if #(W) vif;              // the class parameter selects the interface type

  function new(virtual bus_if #(W) vif);
    this.vif = vif;
  endfunction

  task send(logic [W-1:0] value);
    @(posedge vif.clk);
    vif.data  <= value;
    vif.valid <= 1'b1;
    @(posedge vif.clk);
    vif.valid <= 1'b0;
  endtask
```

```
endclass

module tb_param_vif;

  logic clk = 0;
  always #5 clk = ~clk;

  bus_if #(8)  if8  (clk);
  bus_if #(16) if16 (clk);

  bus_driver #(8)  drv8;
  bus_driver #(16) drv16;

  initial begin
    drv8  = new(if8);
    drv16 = new(if16);
    // drv8 = new(if16);    // ILLEGAL: virtual bus_if #(8) cannot refer to a bus_if #(16)

    fork
      drv8.send(8'hA5);
      drv16.send(16'hBEEF);
    join

    $display("[%0t] if8.data=%h (%0d bits)  if16.data=%h (%0d bits)",
             $time, if8.data, $bits(if8.data), if16.data, $bits(if16.data));
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[15] if8.data=a5 (8 bits)  if16.data=beef (16 bits)
```

Line by line

Code	Explanation
`interface bus_if #(parameter int W = 8) (input logic clk);`	A parameterized interface: the data width is a parameter.
`class bus_driver #(int W = 8);`	The class carries the same parameter.
`virtual bus_if #(W) vif;`	The virtual interface type includes the parameter. `virtual bus_if #(8)` and `virtual bus_if #(16)` are different types.
`bus_if #(8) if8 (clk); bus_if #(16) if16 (clk);`	Two real instances with different widths.
`drv8 = new(if8); drv16 = new(if16);`	Each driver specialisation receives the instance with the matching width.
`fork ... join`	Both `send` tasks run in parallel. Each waits for the rising edge at time 5, drives its data, waits for the edge at 15, and removes `valid`. Both finish at 15.
The `$display`	Shows that one class definition drove an 8-bit and a 16-bit interface.

Parameterized interfaces therefore push parameters into every class that touches them. Projects limit the spread in three ways:

Technique	Idea
Maximum-width interface	The interface is declared with the widest possible signals; narrower DUTs use the low bits. No class parameters are needed

Technique	Idea
`typedef` in the testbench package	`typedef virtual bus_if #(32) bus_vif_t;` used everywhere, so the parameter is written once
Abstract class / concrete class in the interface	A non-parameterized abstract class declares driving tasks as pure virtual methods; a concrete class defined *inside* the interface implements them and has direct access to the signals. The testbench only holds the abstract handle. (This pattern is beyond what a fresher is expected to code, but it is worth knowing it exists.)

Other points that are asked

- **Arrays of virtual interfaces:** `virtual alu_if vifs [4];` with `vifs[i] = if_array[i];` assigned in a `generate` loop or one by one. Used when a DUT has several identical ports.
- **Where a virtual interface may be declared:** in classes, tasks, functions and modules as a variable; it cannot be a port, and an interface instance cannot be created through it.
- **Comparison and null check:** `if (vif == null) $fatal(1, "vif not set");` is the standard first line of a driver's `run()`.
- **How UVM passes it:** the top module stores the handle in the configuration database, and each component fetches it in its build phase (Section 5.6). The mechanism replaces the constructor argument used in M9; the concept is unchanged.

> **Interview trap:** "Is a virtual interface a copy of the interface?" No. It is a reference. Every class holding the handle reads and drives the single real instance, which is how the driver's writes reach the DUT pins.

> **Common mistake:** Declaring `virtual bus_if vif;` (default parameter, width 8) in the class and assigning a `bus_if #(16)` instance to it. The types do not match and elaboration fails. The specialisation must be written in the virtual interface declaration.

Interview questions

Q1. Why can a class not use an interface directly? An interface instance is a static design element created at elaboration; a class object is created dynamically at run time and has no ports or instances. The class needs a variable that can be made to refer to an existing interface instance, which is what a virtual interface is.

Q2. What is the default value of a virtual interface, and what happens if it is used unassigned? `null`. Accessing a signal through it causes a run-time null-reference error.

Q3. How do you declare a virtual interface for a parameterized interface? With the parameter values in the type: `virtual bus_if #(16) vif;`. The values must match those of the instance assigned to it. A class meant to work with several widths is parameterized and passes its own parameter: `virtual bus_if #(W) vif;`.

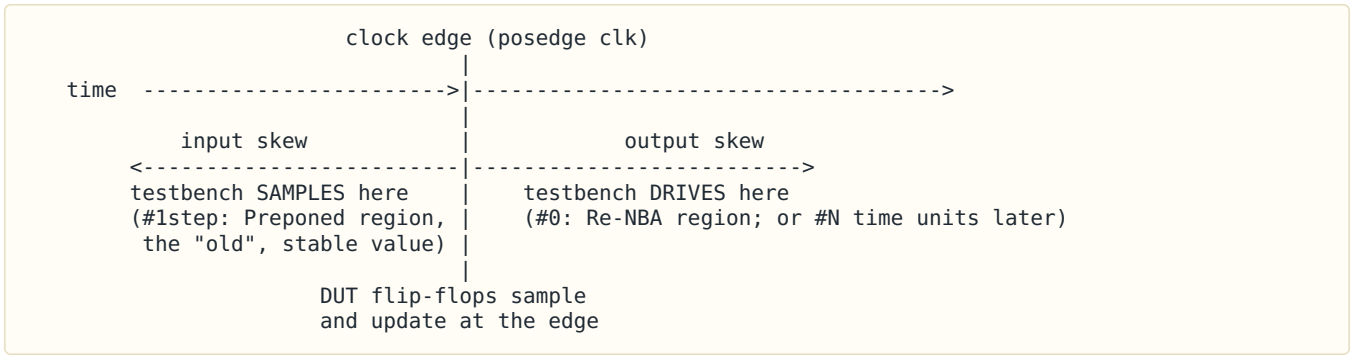
Q4. Can two classes share one interface instance? Yes. The driver and the monitor normally hold virtual interface handles that refer to the same instance.

2.4 Clocking Blocks: Advanced Recap

Module 8 covered clocking blocks. The table and diagram below collect what must be at your fingertips; the UVM driver in Section 5.8 uses them unchanged.

The essentials

Fact	Explanation
What it is	A block declared inside an interface (or module/program) that groups signals which are synchronous to one clock, and defines **when** the testbench samples them and **when** it drives them relative to that clock
Why it exists	To remove races between the testbench and the DUT at the clock edge. Without it, whether the testbench sees the old or the new DUT output depends on the simulator's process ordering
Input skew	How long **before** the clock event an `input` is sampled. Default `#1step`: the value in the Preponed region, that is, the value the signal had just before the edge
Output skew	How long **after** the clock event an `output` is driven. Default `#0`: driven in the Re-NBA region of the same time step, after the DUT's flip-flops have sampled
Direction is from the testbench's view	In `drv_cb`, the DUT's inputs are `output`; the DUT's outputs are `input`
Driving	Only with the synchronous drive `vif.drv_cb.a <= value;` (the `<=` is required)
Sampling	Read `vif.mon_cb.result`. The value is the one captured at the most recent clocking event
Waiting	`@(vif.drv_cb);` waits for the clocking event. `##3;` waits three cycles of the *default* clocking block



The ALU interface with its clocking blocks

This is the structure built in M8 and used by the M9 testbench, shown here in abridged form for reference:

```
interface alu_if (input logic clk);
  import alu_pkg::*;

  logic       rst_n;
  logic       valid_in;
  logic [7:0] a, b;
  alu_op_e    op;
  logic       valid_out;
  logic [8:0] result;

  clocking drv_cb @(posedge clk);           // the driver's view
    default input #1step output #1;
    output valid_in, a, b, op;
    input  valid_out, result;
  endclocking

  clocking mon_cb @(posedge clk);           // the monitor's view: everything is sampled
    default input #1step;
    input valid_in, a, b, op, valid_out, result;
  endclocking

  modport DUT (input  clk, rst_n, valid_in, a, b, op, output valid_out, result);
  modport DRV (clocking drv_cb, input clk, rst_n);
  modport MON (clocking mon_cb, input clk, rst_n);
endinterface
```

Points that interviewers add at this level

Question	Answer
What exactly is `#1step`?	The smallest step of simulation time (one unit of the global time precision). Sampling "one step before" the edge means sampling in the Preponed region of the edge's time step, which gives the value from before any activity at the edge
What does input skew `#0` do?	The input is sampled in the Observed region of the same time step, after the DUT's nonblocking assignments, so the testbench sees the **new** value
Can a signal be both sampled and driven?	Yes, by declaring it `inout` in the clocking block
Does reading `vif.mon_cb.result` right after `@(posedge vif.clk)` give the right value?	It can be stale: the clocking block may not have updated its sample yet. Synchronise with `@(vif.mon_cb)`, not with the raw clock, when you use the clocking block
What happens if two drives to the same clocking output occur in one cycle?	The last one wins, as for nonblocking assignments
Clocking block versus modport	A modport sets the **direction** of signals for one user of the interface. A clocking block sets the **timing** of sampling and driving. They are used together: the modport exposes the clocking block
Are clocking blocks synthesizable?	No

Common mistake: Mixing styles in one component, for example waiting with `@(posedge vif.clk)` and then reading `vif.mon_cb.result`, or driving `vif.a = value` directly while also driving through `vif.drv_cb`. Use the clocking block for both the waiting and the access.

Interview questions

Q1. What problem does a clocking block solve? Race conditions between the testbench and the DUT at clock edges. By sampling inputs slightly before the edge and driving outputs slightly after it, the testbench always reads stable values and never changes a DUT input at the instant the DUT samples it.

Q2. What are the default input and output skews? Input `#1step`, output `#0`.

Q3. What is the difference between a modport and a clocking block? A modport defines signal directions for a particular user of the interface. A clocking block defines sampling and driving timing with respect to a clock. A modport can include a clocking block so that the user gets both.

Q4. How is a signal driven through a clocking block? With a synchronous drive using `<=`: `vif.drv_cb.valid_in <= 1'b1;`. A blocking assignment `=` to a clocking block output is illegal.

Part 3: Process Control

3.1 Process Control

In plain English

A testbench is many activities running at the same time: the generator creates transactions, the driver wiggles pins, the monitor watches, a watchdog counts down. Each of these activities is a **process** (also called a thread). Module 7 showed how to start processes with `fork` and how to make them cooperate with events, mailboxes and semaphores.

Process control is about the other half of the job: **stopping, waiting for, pausing and inspecting** processes. Typical needs are:

- "Wait for the response, but give up after 1000 clock cycles."
- "Start three checkers; when the first one finishes, stop the other two."
- "Reset was asserted in the middle of a transfer: abandon whatever the driver was doing."
- "Is that background thread still running?"

Technical definition

A **process** is an independent thread of execution with its own point of execution and its own state. Processes are created by:

Source of a process	Example
Each `initial` block	`initial begin ... end`
Each `always`, `always_ff`, `always_comb`, `always_latch` block	`always @(posedge clk) ...`
Each `final` block	`final $display(...)`
Each statement inside a `fork`	`fork a(); b(); join` creates two processes
Each continuous assignment	`assign y = a & b;`
A dynamic process	A `fork ... join_none` or `join_any` branch that outlives the statement which created it

A task or function call does **not** create a process. A called task runs in the process of its caller. This fact matters several times in this part.

The process-control toolbox

Tool	What it does	Granularity	Introduced in
`fork ... join` / `join_any` / `join_none`	Starts processes and decides how long the parent waits		M7
`wait fork;`	Blocks until all child processes of the calling process have finished	All children of the caller	M7

Tool	What it does	Granularity	Introduced in
`disable fork;`	Kills all child processes of the calling process, and their descendants	All children of the caller	M7
`disable <block_or_task_name>;`	Terminates a named block or task, wherever it is executing	A named piece of code	Section 3.2
`process` class	A handle on one specific process: query its state, wait for it, kill, suspend or resume it	One process	Section 3.4
`return`, `break`, `continue`	Leave a task/function or a loop from inside	The current process only	M3

```
Coarse  <-------------------------------------------------------------->  Fine

$finish        disable fork         disable <name>         process handle
(everything)   (all my children)    (that named code,      (exactly this one
                                     every activation)      thread)
```

Process family relationships

```
initial begin                       // process P  (the parent)
  fork
    driver();                       // process C1 (child of P)
    begin                           // process C2 (child of P)
      fork
        checker_a();                // process G1 (child of C2, descendant of P)
        checker_b();                // process G2 (child of C2, descendant of P)
      join
    end
  join_none
  ...
end

From inside P:
  wait fork;       waits for C1 and C2 (its immediate children)
  disable fork;    kills C1 and C2 AND their descendants G1, G2
```

The rest of this part explains each tool in the toolbox that was not fully covered in M7, and the patterns built from them.

Interview questions

Q1. What is a process in SystemVerilog? An independent thread of execution. Every `initial` and `always` block, every branch of a `fork`, and every continuous assignment is a process.

Q2. Does calling a task create a new process? No. The task executes in the calling process. A new process is created only by `fork` (or by a procedural block).

Q3. Name the ways to terminate a process from outside. `disable fork` (all children of the caller), `disable` of a named block or task, and the `kill()` method of a `process` handle.

3.2 Named Blocks and the disable Statement

In plain English

Any `begin` ... `end` block or `fork` ... `join` block can be given a name. A name is useful for three things: the reader sees what the block is for, variables declared in it get a hierarchical name, and the block can be terminated from anywhere with `disable <name>`.

Technical definition

A **named block** is a sequential (`begin` ... `end`) or parallel (`fork` ... `join`) block with an identifier after the opening keyword:

```
begin : <name>
  ...
end : <name>          // the closing name is optional; if present it must match
```

The statement `disable <name>;` terminates the execution of the named block or task: every statement still executing inside it stops, and execution continues at the statement that follows the block. The same statement can be executed from inside the block (to leave it early) or from another process (to stop it from outside).

Why it is needed

Use	Example
Leave nested loops in one step	`disable search;` from the inner loop, where `break` would leave only the inner loop
Skip the rest of one loop iteration (old-style `continue`)	Name the loop **body** and disable it
Stop the losing threads of a race	`fork : race ... join_any disable race;`
Stop a specific background block	`disable tb_top.watchdog;`
Readable waveforms and messages	Variables and `%m` show the block name

Since SystemVerilog added `break` , `continue` and `return` , those are preferred for leaving code from the inside. `disable` remains the tool for stopping named code **from the outside**.

Complete example

tb_named_block.sv

```
module tb_named_block;

  int data [8] = '{3, 7, 0, 9, 4, 0, 1, 5};

  initial begin

    // ---- 1. leave a named sequential block early ----
    begin : search
      foreach (data[i]) begin
        if (data[i] == 0) begin
          $display("1. first zero found at index %0d", i);
          disable search;
        end
      end
      $display("   (not printed: the block was disabled before reaching this line)");
    end : search
    $display("2. execution continues after the named block");

    // ---- 2. name the loop BODY: disable acts like 'continue' ----
    for (int i = 0; i < 4; i++) begin : body
      if (i == 1) disable body;
      $display("3. i = %0d", i);
    end

    // ---- 3. a named fork block: stop the slower thread ----
    fork : race
      begin : thread_a
        #30 $display("[%0t] thread_a done", $time);
      end
      begin : thread_b
        #10 $display("[%0t] thread_b done", $time);
      end
    join_any
    disable race;
    $display("[%0t] 4. fork block 'race' disabled", $time);

    #50 $display("[%0t] 5. end of test", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
1. first zero found at index 2
2. execution continues after the named block
3. i = 0
3. i = 2
3. i = 3
[10] thread_b done
[10] 4. fork block 'race' disabled
[60] 5. end of test
```

Line by line

Code	Explanation
`begin : search ... end : search`	A named sequential block.
`disable search;`	Executed inside the block when the first zero is found (index 2). The whole block stops, including the `foreach`. The `$display` after the loop is never reached.
Line 2	Execution resumes at the statement following the disabled block.

Code	Explanation
`for (...) begin : body`	Here the name is on the loop's body, not around the loop.
`if (i == 1) disable body;`	Disabling the body ends only the current iteration; the `for` statement then performs `i++` and continues. So 0, 2 and 3 are printed. This is how `continue` was written before SystemVerilog.
`fork : race ... join_any`	A named fork with two threads. `join_any` returns when the first finishes: `thread_b`, at time 10.
`disable race;`	Terminates everything still running inside the fork block: `thread_a`, which was waiting for time 30. Its message never appears.
`#50 ...`	Proof that `thread_a` is gone: nothing is printed at time 30.

disable and tasks

`disable <task_name>;` terminates the task. Two properties make this dangerous:

1. It terminates **every** currently executing call of that task, in all processes, not only one.
2. The state it leaves behind is not cleaned up: output arguments are not assigned, nonblocking assignments already scheduled still happen, and a semaphore key the task was holding is not returned.

```
task automatic send_packet();
  sem.get(1);
  // ... many clock cycles ...
  sem.put(1);
endtask

// elsewhere
disable send_packet;    // every thread inside send_packet stops; the key is never put back
```

A function cannot be disabled, and a task cannot disable itself to return a value: use `return`.

What exactly does disable stop?

`disable` works on the **name in the hierarchy**, not on a thread. If the same named block is being executed by several threads at once, all of those executions stop. That happens when:

- the block is inside a task called from several processes;
- the block is inside a class method and several objects are running that method;
- the block is inside a module that is instantiated several times (here each instance has its own hierarchical name, so `disable inst1.blk` affects only that instance).

Statement	Stops	Typical problem
`disable <block>;`	All activations of that named block	With class methods running in several objects, it stops all of them
`disable <task>;`	All active calls of that task	Same; also leaves resources locked
`disable fork;`	All child processes of the calling process, with descendants	Kills children started earlier by the same process that you wanted to keep (Section 3.3)
`p.kill();`	Exactly the process `p`, with its descendants	None of the above; you must have obtained the handle

> **Interview trap:** "What is the difference between `disable fork` and `disable <label>`?" `disable fork` has no name: it kills the children of whichever process executes it. `disable <label>` kills the code with that name wherever it is running, regardless of who executes the statement. The first is relative to the caller, the second is absolute.

> **Common mistake:** Using `disable <label>` inside a class method that runs concurrently in several objects (for example every driver of a multi-port testbench). One object's `disable` terminates the block in all objects. In class-based code prefer `disable fork` with an isolating wrapper (Section 3.3) or process handles (Section 3.4).

> **Common mistake:** Expecting code after `disable` inside the block to run. `disable search;` is the last statement the block executes.

Interview questions

Q1. What is a named block and why would you name one? A `begin` ... `end` or `fork` ... `join` block with a label. Naming it lets it be terminated with `disable`, gives its local variables a hierarchical path, and documents its purpose.

Q2. How did Verilog implement `break` and `continue`? With `disable`: naming a block that encloses the loop and disabling it acts as `break`; naming the loop body and disabling it acts as `continue`.

Q3. What happens to a task that is disabled while it is running? It stops immediately. Its output and inout arguments are not updated, and anything it was holding (such as semaphore keys) is not released. All concurrent calls of the same task are terminated.

Q4. What is the difference between `disable`, `break` and `return`? `break` leaves the innermost loop. `return` leaves the current task or function, optionally with a value. Both act only on the code that executes them. `disable` terminates a named block or task, can be issued from a different process, and affects every activation of that name.

3.3 Named Fork Blocks, disable fork and wait fork

Recap from M7

Statement	Parent continues when	Children after that
`fork ... join`	All children have finished	None left
`fork ... join_any`	The first child has finished	The others keep running
`fork ... join_none`	Immediately	All keep running; they start only when the parent next blocks
`wait fork;`	All children of the calling process have finished	
`disable fork;`	Immediately	All children of the calling process, and their descendants, are killed

The disable fork trap

`disable fork` kills **all** children of the calling process, not just those of the nearest `fork` statement. Because a task call does not create a process, a `disable fork` inside a task kills children that the *caller* started earlier.

```
initial begin                          // process P
  fork
    background_monitor();              // child C1 of P   <-- must survive
  join_none

  wait_with_timeout();                 // runs in P (a task call is not a new process)
end

task wait_with_timeout();
  fork
    do_work();                         // child C2 of P
    #timeout;                          // child C3 of P
  join_any
  disable fork;                        // executed by P: kills C2, C3 ... and C1 !
endtask
```

The cure is to make the `disable fork` execute in a process whose **only** children are the ones to be killed. Wrapping the code in `fork begin ... end join` creates such a process:

```
task wait_with_timeout();
  fork                                 // creates process W (child of P); P waits for it
    begin
      fork
        do_work();                     // child of W
        #timeout;                      // child of W
      join_any
      disable fork;                    // executed by W: kills only W's children
    end
  join
endtask
```

This wrapper is called an **isolating fork** (or guard fork). It appears in almost every production testbench, and "why is the extra `fork begin ... end join` there?" is a favourite interview question.

Complete example

tb_disable_fork.sv

```
module tb_disable_fork;

  task automatic background(string name, int n);
    repeat (n) #20 $display("[%0t] %s tick", $time, name);
  endtask

  task automatic wait_with_timeout(int work_time, int timeout);
    fork                                  // isolating fork
      begin
        fork
          #work_time $display("[%0t] work done", $time);
          #timeout   $display("[%0t] TIMEOUT", $time);
        join_any
        disable fork;                     // kills only the loser of this race
      end
    join
  endtask

  initial begin
    fork
      background("monitor", 4);           // must keep running throughout
    join_none

    wait_with_timeout(30, 50);            // the work wins
    wait_with_timeout(70, 25);            // the timeout wins

    wait fork;                            // wait for the background thread
    $display("[%0t] all threads finished", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[20] monitor tick
[30] work done
[40] monitor tick
[55] TIMEOUT
[60] monitor tick
[80] monitor tick
[80] all threads finished
```

Execution flow

Time	What happens
0	The `initial` process forks the background thread with `join_none` and calls `wait_with_timeout(30, 50)`. The task forks the isolating process, which forks the two racing threads.
20	Background thread: first tick.
30	The work thread finishes first. `join_any` returns, `disable fork` kills the timeout thread (it would have fired at 50). The isolating process ends, `join` returns, the task returns. The second call starts: work due at 100, timeout due at 55.
40	Background tick. It was not affected by the `disable fork` at 30.
55	The timeout thread finishes first and prints `TIMEOUT`. `disable fork` kills the work thread. The task returns.
55	`wait fork` blocks: the background thread, a child of the `initial` process, is still running.
60, 80	Remaining ticks. After the fourth tick the background task ends.

Time	What happens
80	`wait fork` returns.

If the isolating `fork begin ... end join` were removed, the `disable fork` at time 30 would be executed by the `initial` process itself and would kill the background thread as well. The output would then contain a single tick at 20.

Named fork blocks

A fork block may be named like any other block, and its threads may be named individually:

```
fork : checks
  begin : check_valid   ...  end
  begin : check_result  ...  end
join_any
disable checks;                 // stop everything left in this fork block
// disable check_result;        // or stop one particular thread
```

Property	`disable fork;`	`disable <fork_name>;`
Identifies its targets by	Parenthood: children of the calling process	Name: the code inside the named fork block
Kills threads from earlier forks of the same process	Yes (the trap above)	No
Kills grandchildren	Yes	Yes, everything executing inside the named block
Safe in class methods that run in several objects	Yes (with an isolating fork)	No: all activations of that name are affected
Needs a label	No	Yes

Named forks are the simple solution in module-level testbench code. In class-based code the isolating fork with `disable fork`, or process handles, are the safe choices.

wait fork

`wait fork;` blocks the calling process until all of its **immediate** child processes have terminated. It is used at the end of a test or a task to make sure nothing started with `join_none` is still running:

```
for (int i = 0; i < 4; i++) begin
  fork
    automatic int ch = i;          // each thread gets its own copy (M7)
    send_on_channel(ch);
  join_none
end
wait fork;                         // all four sends have completed
```

Like `disable fork`, it refers to *all* children of the calling process. If a never-ending child (a monitor's `forever` loop) was started by the same process, `wait fork` never returns. The same isolating-fork idea fixes it.

> **Interview trap:** "`fork ... join_any` followed by `disable fork`: which threads die?" All children of the process executing `disable fork`, including children created by earlier `fork` statements of that process and their descendants, not merely the remaining threads of the fork just above.

> **Common mistake:** A `wait fork` at the end of `alu_env::run()` after the monitor and scoreboard were started with `join_none` and contain `forever` loops. The wait never ends and the test hangs. End-of-test must be based on something that actually completes (the generator finishing, a transaction count, a watchdog).

Interview questions

Q1. What does `disable fork` do? It terminates all active child processes of the process that executes it, together with all of their descendants.

Q2. Why is `fork begin ... end join` wrapped around a `join_any` and `disable fork`? To isolate the `disable fork`. The wrapper creates a new process whose only children are the threads forked inside it, so `disable fork` cannot kill unrelated threads that the outer process had started earlier.

Q3. What is the difference between `wait fork` and `join`? `join` belongs to one `fork` statement and waits for that statement's threads. `wait fork` is a separate statement that waits for all child processes of the calling process, whichever `fork` statements created them, including those started with `join_none` or left running after `join_any`.

Q4. How do you stop only one thread of a fork? Name that thread's block and `disable` the name, or capture the thread's `process` handle and call `kill()` on it.

3.4 The Process Class and Process Handles

In plain English

`disable fork` and `disable <name>` are blunt. Sometimes you want a handle on **one particular thread**, like a job number, so that you can ask "are you still running?", say "wait until that one has finished", or "stop that one and nothing else". The built-in `process` class provides that handle.

Technical definition

`process` is a built-in class declared in the `std` package. Each process in the simulation has an associated `process` object. A process obtains the handle to **itself** with the static method `process::self()`, and may store that handle in a variable so that other processes can control it.

```
// Prototype of the built-in class (you never write this; it is part of the language)
class process;
  typedef enum { FINISHED, RUNNING, WAITING, SUSPENDED, KILLED } state;

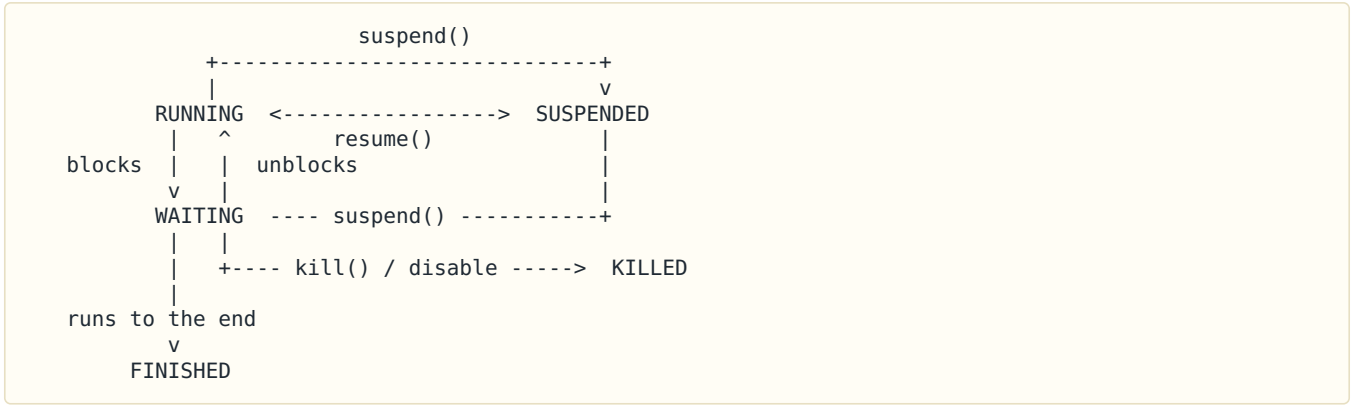
  static function process self();     // handle of the calling process
  function state status();            // current state of this process
  function void  kill();              // terminate this process and its sub-processes
  task           await();             // block until this process terminates
  function void  suspend();           // pause this process
  function void  resume();            // let a suspended process continue
  // also: srandom(), get_randstate(), set_randstate() for random stability (M5)
endclass
```

Special rules:

- You cannot create a process object with `new`. `process p = new();` is an error. The only source of handles is `process::self()`.
- The class cannot be extended.
- A handle remains valid after its process has ended; `status()` then reports `FINISHED` or `KILLED`.

The process states

State	Value	Meaning
`FINISHED`	0	The process terminated normally (ran to its end)
`RUNNING`	1	The process is currently executing (it is not blocked)
`WAITING`	2	The process is blocked in a blocking statement: a delay, `@`, `wait`, a mailbox `get`, and so on
`SUSPENDED`	3	The process was stopped by `suspend()` and is waiting for `resume()`
`KILLED`	4	The process was terminated by `kill()` or by a `disable`



Since only one process executes at any instant, a process that asks for the status of *another* process normally sees `WAITING`, `SUSPENDED`, `FINISHED` or `KILLED`. `RUNNING` is what a process sees when it asks about itself.

The methods

Method	Called on	Effect	Notes
`process::self()`	The class (static)	Returns the handle of the process that executes the call	A task called from process P returns P's handle
`p.status()`	Any handle	Returns the state enum	Use `process::FINISHED` etc. to compare
`p.kill()`	Any handle	Terminates `p` and all its descendants	A process may kill itself
`p.await()`	Another process's handle	Blocks the caller until `p` has terminated (finished or killed)	A process must not `await` itself: error
`p.suspend()`	Any handle	Stops `p` from executing until resumed	A process may suspend itself; another process must then resume it

Method	Called on	Effect	Notes
`p.resume()`	A suspended process's handle	Lets `p` continue	If `p` was suspended while waiting for a delay or event that has meanwhile occurred, it continues straight away

> **Note:** `self()`, `status()`, `kill()` and `await()` are supported by all major simulators and are used widely (UVM uses them internally). `suspend()` and `resume()` are less commonly used, and the exact time at which a resumed process continues has differed between tools. Avoid building testbench logic that depends on them.

How to obtain a handle to a forked thread

The thread itself must call `process::self()` and store the result. There is one subtlety: after `fork ... join_none`, the child has **not started yet** (it starts when the parent blocks), so the variable is still `null`. The parent must wait for the handle to appear.

```
parent                                  child
------                                  -----
fork
  begin
    p = process::self();    ....... (not executed yet)
    ...
  end
join_none
p.kill();        <-- WRONG: p is still null here -> null handle error

wait (p != null);           parent blocks -> child starts, sets p, blocks on a delay
p.kill();        <-- correct
```

Complete example: status, await and kill

tb_process_class.sv

```
module tb_process_class;

  process p_main, p_fast, p_slow;

  function automatic string st(process p);
    process::state s = p.status();
    return s.name();
  endfunction

  initial begin
    p_main = process::self();

    fork
      begin
        p_fast = process::self();
        #10 $display("[%0t] fast thread finished", $time);
      end
      begin
        p_slow = process::self();
        #100 $display("[%0t] slow thread finished", $time);
      end
    join_none

    wait (p_fast != null && p_slow != null);
    $display("[%0t] main=%s fast=%s slow=%s", $time, st(p_main), st(p_fast), st(p_slow));

    p_fast.await();
    $display("[%0t] after await : fast=%s slow=%s", $time, st(p_fast), st(p_slow));

    #5 p_slow.kill();
    $display("[%0t] after kill  : slow=%s", $time, st(p_slow));

    #200 $display("[%0t] end (the slow thread never printed)", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] main=RUNNING fast=WAITING slow=WAITING
[10] fast thread finished
[10] after await : fast=FINISHED slow=WAITING
[15] after kill  : slow=KILLED
[215] end (the slow thread never printed)
```

Line by line

Code	Explanation
`process p_main, p_fast, p_slow;`	Three handle variables. Like all class handles they start as `null`.
`function automatic string st(process p);`	A helper that returns the name of the state. `process::state` is the enum type declared inside the class, reached with `::`.
`p_main = process::self();`	The `initial` process stores its own handle.
`p_fast = process::self();` inside the fork	Executed by the first child, so it returns that child's handle.
`join_none`	The parent continues immediately. The children have not started.
`wait (p_fast != null && p_slow != null);`	The parent blocks. Both children now start, store their handles and block on their delays. The condition becomes true and the parent continues, still at time 0.

Code	Explanation
First `$display`	`p_main` is the process doing the printing: `RUNNING`. Both children are blocked on a delay: `WAITING`.
`p_fast.await();`	The parent blocks until the fast thread ends at time 10.
Second `$display`	The fast thread ran to completion: `FINISHED`. The slow one is still waiting for time 100.
`#5 p_slow.kill();`	At time 15 the slow thread is terminated. Its `$display` at time 100 never executes.
Third `$display`	A killed process reports `KILLED`, which is distinct from `FINISHED`.

Complete example: a pool of worker threads

Handles can be stored in a queue, which gives full control over a group of threads created in a loop.

tb_process_array.sv

```
module tb_process_array;

  process jobs [$];
  int     killed;

  initial begin
    for (int i = 0; i < 3; i++) begin
      fork
        automatic int id = i;
        begin
          jobs.push_back(process::self());
          #(10 * (id + 1));
          $display("[%0t] worker %0d done", $time, id);
        end
      join_none
    end

    wait (jobs.size() == 3);        // all three threads have started and registered

    #25;
    foreach (jobs[k]) begin
      if (jobs[k].status() != process::FINISHED) begin
        jobs[k].kill();
        killed++;
      end
    end
    $display("[%0t] killed %0d unfinished worker(s)", $time, killed);
  end

endmodule
```

EXPECTED OUTPUT

```
[10] worker 0 done
[20] worker 1 done
[25] killed 1 unfinished worker(s)
```

Line by line

Code	Explanation
`process jobs [$];`	A queue of process handles.
`automatic int id = i;`	Each forked thread gets its own copy of the loop variable (M7). Without it, every thread would see `i` = 3.
`jobs.push_back(process::self());`	Each worker registers itself. The order of registration within time 0 is not defined, and the example does not depend on it.

Code	Explanation
`#(10 * (id + 1));`	Workers 0, 1 and 2 need 10, 20 and 30 time units.
`wait (jobs.size() == 3);`	The parent waits until all three have registered.
`#25;` then the `foreach`	At time 25, workers 0 and 1 are `FINISHED`. Worker 2 is still `WAITING`; it is killed and counted. Nothing is printed at time 30.

Complete example: suspend and resume

`tb_process_suspend.sv`

```
module tb_process_suspend;

  process p_worker;

  initial begin
    fork
      begin
        p_worker = process::self();
        for (int i = 1; i <= 4; i++)
          #10 $display("[%0t] worker step %0d", $time, i);
      end
    join_none

    wait (p_worker != null);

    #15 p_worker.suspend();
    $display("[%0t] worker suspended", $time);

    #30 p_worker.resume();
    $display("[%0t] worker resumed", $time);

    p_worker.await();
    $display("[%0t] worker finished", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[10] worker step 1
[15] worker suspended
[45] worker resumed
[45] worker step 2
[55] worker step 3
[65] worker step 4
[65] worker finished
```

Explanation

- Step 1 is printed at time 10. The worker then waits for its next delay, which would expire at time 20.
- At time 15 the worker is suspended while it is blocked in that delay.
- Time 20 passes; the worker does not continue, because it is suspended.
- At time 45 it is resumed. The standard says that if the delay a process was waiting for has already elapsed, the resumed process is scheduled to continue immediately. So step 2 is printed at time 45, and the remaining steps follow at 10-unit intervals from there.
- The parent prints "worker resumed" before the worker prints "step 2", because `resume()` only schedules the worker; the parent keeps executing until it blocks on `await()`.

This is the behaviour defined by IEEE 1800. As noted above, it is the least uniformly implemented part of the class, so run this example on your simulator before relying on it.

Comparison of the ways to stop a thread

Property	`disable fork`	`disable <label>`	`process.kill()`
Targets	All children of the caller	All activations of a named block or task	One specific process (with its descendants)
Needs preparation	No	A label	The thread must store `process::self()`
Can query whether the thread is still alive	No	No	Yes, `status()`
Can wait for one specific thread	No (`wait fork` waits for all)	No	Yes, `await()`
Safe with many objects running the same method	Yes, with an isolating fork	No	Yes
Can pause and continue	No	No	Yes, `suspend()` / `resume()`

Interview trap: "What does `process::self()` return when it is called inside a task?" The handle of the process that **called** the task, because a task call does not create a process. To get a handle to a new thread, `self()` must be called inside a `fork` branch.

Interview trap: "The code does `fork begin p = process::self(); ... end join_none` and then immediately `p.kill();`. What goes wrong?" `p` is still `null`: a `join_none` child does not begin executing until the parent blocks. The result is a null-handle run-time error. Insert `wait (p != null);`.

Common mistake: `process p = new();`. Process objects cannot be constructed by the user.

Common mistake: Checking `p.status() == process::RUNNING` from another process to mean "still alive". A blocked process reports `WAITING`. Test for "not finished and not killed" instead.

Verification application: UVM is built on process handles: it uses them to kill sequences, to terminate phase tasks when a phase ends, and to keep each component's random stream stable. In hand-written testbenches they are used for per-transaction timeouts, for aborting a driver's current transfer on reset while leaving the rest of the testbench running, and for reporting which threads were still active when a test hung.

Interview questions

Q1. What is the `process` class? A built-in class in the `std` package that gives a handle on a process. Through the handle the process's status can be read and the process can be killed, awaited, suspended or resumed.

Q2. How do you get a process handle? The process itself calls the static method `process::self()` and assigns the result to a variable visible to whoever needs it. A process object cannot be created with `new`.

Q3. What are the process states? `FINISHED`, `RUNNING`, `WAITING`, `SUSPENDED` and `KILLED`.

Q4. What is the difference between `await()` and `wait fork`? `p.await()` waits for one specific process to terminate. `wait fork` waits for all child processes of the calling process.

Q5. What is the difference between `kill()` and `disable fork`? `kill()` terminates exactly the process referred to by the handle (and its descendants). `disable fork` terminates every child of the calling process. `kill()` is precise and works on any process you hold a handle to, not only on your own children.

Q6. What is the difference between `FINISHED` and `KILLED`? `FINISHED` means the process ran to its natural end. `KILLED` means it was terminated from outside by `kill()` or `disable`.

Q7. Can a process `await()` itself? Can it `kill()` or `suspend()` itself? It cannot await itself: that is an error (it would never return). It can kill itself and it can suspend itself; a suspended process needs another process to resume it.

3.5 Watchdog and Timeout Patterns

Every pattern below combines the tools of this part. They are the practical reason process control is taught, and they are frequent "write the code" interview questions.

Pattern 1: global test watchdog

The simplest protection against a hung simulation: a separate `initial` block in the top module that ends the test with an error after a maximum time.

```
module tb_top;
  ...
  initial begin : watchdog
    #1ms;
    $fatal(1, "[%0t] WATCHDOG: test did not finish in time", $time);
  end
endmodule
```

When the test finishes normally it calls `$finish`, which ends everything, including the watchdog. The limit is usually made configurable with `$value$plusargs` (Section 4.4).

Pattern 2: an operation with a timeout

"Wait for X, but not longer than N." A race between the operation and a timer, in an isolating fork:

```
task automatic wait_for_valid_out(virtual alu_if vif, int max_cycles, output bit ok);
  fork
    begin
      fork
        begin                                   // the awaited condition
          @(vif.mon_cb iff vif.mon_cb.valid_out === 1'b1);
          ok = 1;
        end
        begin                                   // the timer
          repeat (max_cycles) @(vif.mon_cb);
          ok = 0;
        end
      join_any
      disable fork;
    end
  join
endtask
```

Exactly one of the two branches completes; `disable fork` removes the other. The caller checks `ok` and reports an error on a timeout.

Pattern 3: first of N wins

Start several alternatives and continue with whichever completes first:

```
fork
  begin
    fork
      wait_for_response();
      wait_for_error();
      wait_for_reset();
    join_any
    disable fork;
  end
join
```

Pattern 4: abort the driver on reset

A driver must drop the transaction it is driving when reset is asserted, then start again. The driving loop is forked, a reset detector races with it, and the loop is restarted for ever.

```
task alu_driver::run();
  forever begin
    wait (vif.rst_n === 1'b1);                // stay idle while in reset
    fork
      begin
        fork
          drive_loop();                       // forever: get from gen2drv_mbx, drive
          @(negedge vif.rst_n);               // reset asserted
        join_any
        disable fork;                         // abandon the transfer in progress
      end
    join
    vif.drv_cb.valid_in <= 1'b0;              // return the pins to their idle values
  end
endtask
```

`drive_loop()` never returns, so `join_any` can only be satisfied by the reset branch.

Pattern 5: timeout using a process handle

The same idea as pattern 2 with explicit handles, which makes it possible to report what was killed:

```
task automatic run_with_timeout(int limit, output bit timed_out);
  process p_work;
  fork
    begin
      p_work = process::self();
      do_work();
    end
  join_none
  wait (p_work != null);

  fork
    begin
      fork
        p_work.await();
        #limit;
      join_any
      disable fork;
    end
  join

  timed_out = (p_work.status() != process::FINISHED);
  if (timed_out) p_work.kill();
endtask
```

A complete, runnable version of this pattern is Exercise 3 in Section 6.5.

Which pattern to use

Situation	Pattern
Whole simulation must not hang	1, global watchdog
One wait must not hang	2, race in an isolating fork
Several possible outcomes	3, first of N
Restartable activity (reset, link down)	4, loop around a race
Need to know or report what was still running	5, process handles

> **Common mistake:** Writing the timer as a second `initial` block that sets a flag, and polling the flag in a loop with no delay in it. A loop without any blocking statement never lets simulation time advance: the simulator hangs at one time step. Every waiting loop must contain a delay, an event control or a `wait`.

> **Common mistake:** A timeout measured in raw time units (`#1000`) inside reusable class code. The meaning depends on the time scale of the file. Count clock cycles through the clocking block, or use explicit units (`#1us`).

Interview questions

Q1. How do you implement a timeout for a blocking operation? Fork the operation and a timer in parallel with `join_any`, then `disable fork` to kill whichever did not finish, all inside an isolating `fork begin ... end join`. A flag or a process status tells the caller which branch won.

Q2. How does a testbench avoid hanging for ever when the DUT never responds? With a watchdog: a thread that waits for a maximum time (or cycle count) and then ends the simulation with `$fatal`. Local timeouts on individual waits give a more precise error message than the global watchdog.

Q3. How would you make a driver stop its current transaction when reset occurs? Run the driving code and a reset detector as a race in a `fork`...`join_any`, `disable fork` when reset wins, put the interface signals back to idle, wait for reset to be released, and start again, all inside a `forever` loop.

Part 4: Random Sequences, C Code and System Tasks

4.1 randcase

In plain English

A `case` statement chooses a branch by comparing a value. A `randcase` statement chooses a branch **at random**, and you say how likely each branch is by giving it a weight. It is the quickest way to write "60 percent of the time do a write, 30 percent a read, 10 percent stay idle" in procedural code.

Technical definition

`randcase` is a procedural statement that selects exactly one of its branches at random. Each branch is prefixed by a non-negative integer expression, its **weight**. The probability of a branch is its weight divided by the sum of all the weights.

Syntax

```
randcase
  <weight_1> : <statement_1>;
  <weight_2> : <statement_2>;
  ...
endcase
```

Rules

Rule	Detail
Weights are expressions	They may be constants or variables, evaluated each time the statement executes, so the mix can change at run time
Probability	weight / (sum of all weights). Weights 6, 3, 1 give 60 %, 30 %, 10 %
A weight of 0	That branch is never selected
All weights 0	No branch is taken; the simulator may issue a warning
Exactly one branch runs	There is no fall-through and no `default`
Random source	The random number generator of the current process (the same one `$urandom` uses), so the result is reproducible for a given seed and thread structure
Where it can be used	Any procedural code: tasks, functions, `initial` blocks, class methods. No class or `rand` variable is needed
Synthesizable	No

Complete example

tb_randcase.sv

```
module tb_randcase;

  int n_write, n_read, n_idle, n_never;

  initial begin
    repeat (1000) begin
      randcase
        6 : n_write++;
        3 : n_read++;
        1 : n_idle++;
        0 : n_never++;        // weight 0: never selected
      endcase
    end

    $display("write=%0d read=%0d idle=%0d never=%0d", n_write, n_read, n_idle, n_never);
    $display("total=%0d", n_write + n_read + n_idle + n_never);
  end

endmodule
```

EXPECTED OUTPUT

```
write=603 read=298 idle=99 never=0
total=1000
```

Sample output; differs per seed and simulator. What is guaranteed: `never` is 0, the total is exactly 1000 (one branch per execution), and the three counts are close to 600, 300 and 100.

Line by line

Code	Explanation
`repeat (1000)`	The `randcase` is executed 1000 times; each execution makes one independent random choice.
`6 : n_write++;`	Weight 6 out of a total of 10: selected about 60 % of the time.
`0 : n_never++;`	A zero weight removes the branch from the choice.
The total	Always 1000, which shows that exactly one branch executes each time.

An industry-style use

```
task alu_generator::run();
  alu_txn txn;
  repeat (num_txn) begin
    txn = new();
    randcase
      70 : assert (txn.randomize());                                    // fully random
      20 : assert (txn.randomize() with { a == 8'hFF; b == 8'hFF; });   // corner: maximum
      10 : assert (txn.randomize() with { op == ALU_SUB; a < b; });     // corner: borrow
    endcase
    gen2drv_mbx.put(txn);
  end
  -> gen_done;
endtask
```

The generator mixes plain random transactions with two families of corner cases in chosen proportions, without adding any constraint to the transaction class.

randcase compared with related constructs

Construct	Chooses	Weights	Where it is written	Works with constraints
`case`	The branch that matches a value	None	Procedural code	Not applicable
`randcase`	One **statement** at random	Per branch	Procedural code	No: it is not seen by the solver
`dist` in a constraint (M5)	A **value** for a `rand` variable	Per value or range	Inside a constraint	Yes: solved together with all other constraints
`randsequence` (Section 4.2)	A **sequence** of statements from a grammar	Per alternative	Procedural code	No

> **Interview trap:** "What happens if every weight in a `randcase` is zero?" No branch is executed (and a warning may be printed). It is not an error and it does not pick a branch arbitrarily.

> **Common mistake:** Assuming weights must add up to 100. They are relative. Weights 1 and 3 give 25 % and 75 %.

Interview questions

Q1. What is `randcase`? A procedural statement that randomly executes one of several branches, with the probability of each branch proportional to its weight.

Q2. What is the difference between `randcase` and a `dist` constraint? `randcase` selects which statement to execute in procedural code and is not part of constraint solving. `dist` assigns weighted probabilities to the values of a random variable inside a constraint and is solved together with the object's other constraints by `randomize()`.

Q3. In `randcase 1: a(); 2: b(); 5: c(); endcase`, what is the probability of `b()`? 2 / (1 + 2 + 5) = 25 %.

4.2 randsequence

In plain English

`randcase` makes one random choice. `randsequence` generates a whole random **sequence of actions** that follows a set of rules. You describe the legal structure of a scenario in the same way a grammar describes legal sentences: "a test is a reset, then some traffic, then a finish; traffic is several operations; an operation is a write, a read or an idle". The simulator then walks through these rules, making random choices wherever there are alternatives, and executes the code attached to each rule.

Technical definition

`randsequence` is a procedural statement that defines a set of **productions** (rules) in a notation similar to Backus-Naur Form (BNF). Execution starts at a named top production. A production expands into a list of other productions and code blocks, executed in order; where several alternatives are separated by `|`, one is chosen at random, optionally weighted with `:=`. Productions that contain only a code block are the terminals: they do the actual work.

Syntax

```
randsequence (<top_production>)
  <production> : <item> <item> ... ;                    // a sequence, run left to right
  <production> : <alt_1> | <alt_2> | <alt_3> ;          // alternatives: one is chosen
  <production> : <alt_1> := <w1> | <alt_2> := <w2> ;    // weighted alternatives
  <production> : { <statements> } ;                     // a code block (terminal)
endsequence
```

Further forms inside a production:

Form	Meaning
`repeat (n) prod`	Execute `prod` n times
`if (cond) prod_a else prod_b`	Conditional production
`case (expr) v1: prod_a; v2: prod_b; default: prod_c; endcase`	Selection by value
`rand join prod_a prod_b`	Interleave the items of the two productions in random order
`break;` inside a code block	End the whole `randsequence`
`return;` inside a code block	End the current production only
Productions with arguments and return values	`gen(int n) : ...`; a production may be declared with a return type

> **Note:** `randsequence` is part of the standard, but tool support is uneven, particularly for `rand join`, production arguments and return values. Basic sequences, alternatives, weights and `repeat` work in the major simulators. Check your tool before using the advanced forms, and expect many teams to use constraint-based sequences instead.

How it works internally

```
randsequence (main)
  main    : do_reset traffic do_done ;
  traffic : repeat (4) one_op ;
  one_op  : do_write := 3 | do_read := 2 | do_idle := 1 ;
  ...

Execution = a walk through the production tree:

                      main
          +-------------+--------------+
       do_reset       traffic        do_done
                         |
          +--------+-----+-----+--------+
        one_op   one_op      one_op   one_op          (repeat 4)
          |        |           |        |
        random   random      random   random          each picks write / read / idle
        choice   choice      choice   choice          with weights 3 : 2 : 1
```

Complete example

tb_randsequence.sv

```
module tb_randsequence;

  int n_ops;

  initial begin
    randsequence (main)
      main     : do_reset traffic do_done ;
      traffic  : repeat (4) one_op ;
      one_op   : do_write := 3 | do_read := 2 | do_idle := 1 ;
      do_reset : { $display("RESET"); } ;
      do_write : { $display("  WRITE"); n_ops++; } ;
      do_read  : { $display("  READ");  n_ops++; } ;
      do_idle  : { $display("  IDLE");  n_ops++; } ;
      do_done  : { $display("DONE after %0d operations", n_ops); } ;
    endsequence
  end

endmodule
```

EXPECTED OUTPUT

```
RESET
  WRITE
  READ
  WRITE
  IDLE
DONE after 4 operations
```

Sample output; differs per seed and simulator. What is guaranteed: the first line is `RESET`, the last line is `DONE after 4 operations`, and between them there are exactly four lines, each one of `WRITE`, `READ` or `IDLE`, with `WRITE` the most likely.

Line by line

Code	Explanation
`randsequence (main)`	Execution starts at the production named `main`.
`main : do_reset traffic do_done ;`	A sequence of three productions, executed in this order. The structure of the scenario is fixed here.
`traffic : repeat (4) one_op ;`	The production `one_op` is executed four times; each time a new random choice is made.
`one_op : do_write := 3	do_read := 2	do_idle := 1 ;`	Three alternatives with weights. `do_write` is chosen with probability 3/6, `do_read` 2/6, `do_idle` 1/6. Without `:=` all alternatives are equally likely.
`do_write : { ... } ;`	A terminal: its code block is ordinary procedural code, which may call tasks, drive an interface, or randomize and send a transaction.
`n_ops`	Counts the operations to demonstrate that exactly four are generated.

In a real testbench the code blocks would call tasks such as `send_write()` and `send_read()` that create a transaction and put it in `gen2drv_mbx`.

randcase versus randsequence versus constrained randomization

Property	`randcase`	`randsequence`	`randomize()` with constraints
Randomises	The choice of one statement	The structure and order of a series of statements	The values of variables
Describes	A weighted choice	A grammar of legal scenarios	A set of legal values
Hierarchical	No	Yes: productions refer to productions	Through object composition
Solver involved	No	No	Yes
Typical use	Mixing a few kinds of stimulus	Generating protocol-level scenarios, instruction streams	Transaction contents
Tool support	Universal	Uneven for advanced features	Universal
Reuse / override in derived tests	Poor (hard-coded in a task)	Poor	Good (constraints are inherited and overridden)

> **Common mistake:** Trying to make a `randsequence` choice depend on a constraint in a class. The two mechanisms are separate. Use `if` or `case` productions with ordinary variables to steer a `randsequence`.

> **Verification application:** Random instruction-stream generators for processors, and generators of legal command orders for a protocol ("configure, then enable, then any number of transfers, then disable"), are natural fits for a grammar. In UVM the same job is normally done with layered sequences, which is one reason `randsequence` is rare in modern testbenches. For interviews, know what it is, the meaning of `:=` and `|`, and be able to read a small one.

Interview questions

Q1. What is `randsequence`? A procedural construct that generates a random sequence of actions from a grammar of productions. Each production is a sequence of other productions, a weighted random choice between alternatives, or a code block.

Q2. What do `|` and `:=` mean inside a `randsequence`? `|` separates alternatives of which one is chosen at random. `:=` assigns a weight to an alternative.

Q3. When would you use `randsequence` instead of constraints? When the randomness is in the *order and structure* of operations (a scenario) and not in the *values* of fields: for example generating legal command sequences or instruction streams.

4.3 DPI Introduction

In plain English

A great deal of useful code already exists in C: golden reference models from the architecture team, CRC and encryption routines, the firmware that will run on the chip. Rewriting it in SystemVerilog would be slow and would create a second version that could differ from the first. The **Direct Programming Interface (DPI)** lets SystemVerilog call a C function as if it were a SystemVerilog function, and lets C call a SystemVerilog function or task.

Technical definition

DPI is the standard interface between SystemVerilog and a foreign programming language. The standard defines one foreign-language layer, for C, identified by the string `"DPI-C"`.

- An **import** declaration makes a C function callable from SystemVerilog.
- An **export** declaration makes a SystemVerilog function or task callable from C.

Arguments are passed directly, using a defined mapping between SystemVerilog types and C types. No simulator-specific API is needed for simple types.

Why it is needed

Use	Example
Reuse an existing C/C++ reference model in the scoreboard	The scoreboard calls `c_alu_model()` to obtain the expected result
Reuse algorithms	CRC, checksums, encryption, compression, floating-point maths
Performance	A heavy computation is faster in compiled C than in simulated SystemVerilog
Access to the operating system	Files, sockets, environment variables, shared memory
Hardware/software co-verification	Driver or firmware code written in C drives register reads and writes through exported SystemVerilog bus tasks

Before DPI, Verilog offered the PLI and VPI. They are general but verbose: values are passed through API calls, and every C function must be registered with the simulator. DPI replaced them for the common case of "call this function with these arguments".

Syntax

```
// Import: C function callable from SystemVerilog
import "DPI-C" function int  c_add(input int a, input int b);
import "DPI-C" pure    function real c_scale(input real x);
import "DPI-C" context function void c_notify(input string msg);
import "DPI-C" context task          c_run_firmware();       // may call exported tasks

// Import under a different SystemVerilog name:  c_name = sv_name
import "DPI-C" crc32_table = function int unsigned crc32(input int unsigned data);

// Export: SystemVerilog function or task callable from C
export "DPI-C" function sv_log;         // only the name: no arguments, no parentheses
export "DPI-C" task     sv_bus_write;
```

Imports may be placed in a module, an interface, a program, a package or the compilation unit scope. Placing them in a package gives every user access to the C functions.

Properties of an imported function

Qualifier	Meaning	Restrictions
(none)	An ordinary C function	Must not call exported SystemVerilog code or simulator APIs that need a scope

Qualifier	Meaning	Restrictions
`pure`	The result depends only on the input arguments and there are no side effects. The simulator may optimise calls away or reorder them	Must return a value (not `void`); no `output` or `inout` arguments; must not read or write anything outside itself (no files, no globals, no exported functions)
`context`	The C function needs to know the SystemVerilog scope it was called from, because it calls exported functions or tasks, or uses VPI/PLI	Slightly slower to call

Functions versus tasks follow the SystemVerilog rule from M3: an imported **function** returns immediately and consumes no simulation time. An imported **task** may consume time, which it can only do by calling an exported SystemVerilog task that contains timing controls; such an import must be `context`.

Data type mapping

SystemVerilog type	C type	Notes
`byte`	`char`	
`shortint`	`short int`	
`int`	`int`	The most convenient type for DPI
`longint`	`long long`	
`real`	`double`	
`shortreal`	`float`	
`string`	`const char*`	A C string; do not keep the pointer after the call returns
`chandle`	`void*`	An opaque C pointer stored on the SystemVerilog side
`bit`	`svBit` (`unsigned char`)	Value 0 or 1
`logic`, `reg`	`svLogic` (`unsigned char`)	Values 0, 1, Z, X encoded as 0, 1, 2, 3
`bit [N:0]` (packed two-state)	`svBitVecVal*`	Array of 32-bit words
`logic [N:0]` (packed four-state)	`svLogicVecVal*`	Array of `{aval, bval}` word pairs (the encoding from M1, Section 2.1)
Open array `[]` argument	`svOpenArrayHandle`	Accessed through functions in `svdpi.h`; support varies between tools

Argument directions:

SystemVerilog direction	Passed to C as
`input` of a small scalar type (`int`, `real`, `byte` ...)	By value
`input string`	`const char*`
`output`, `inout`	Pointer to the C type (`int*`, `double*`)
Packed vectors, any direction	Pointer to the canonical array

The types `svBit`, `svLogic`, `svBitVecVal` and the helper functions are declared in the header file `svdpi.h`, which every simulator supplies.

> **Key point:** Use two-state C-compatible types (`int` , `longint` , `byte` , `real` , `string`) for DPI arguments whenever possible. They map one-to-one onto C types. Four-state and wide vectors require the `svLogicVecVal` structures and are much more error-prone.

How it works internally

```
SystemVerilog side                                C side
------------------                                ------
import "DPI-C" function int c_alu_model(...);     int c_alu_model(int op, int a, int b)
                                                  { ... return r; }
exp = c_alu_model(op, a, b);   --- call --->      runs as normal compiled C code
                               <-- return --      (simulation time does not advance)

export "DPI-C" function sv_log;
function void sv_log(string msg); ...             extern void sv_log(const char* msg);
                               <--- call ---      sv_log("hello");

Build: the C file is compiled into an object file or shared library and linked with the
       simulation. The simulator matches the import name with the C function name.
```

While the C function runs, the simulator is stopped inside that call. A C function that loops for ever hangs the simulation, and a C function that crashes brings the simulator down with it.

Complete example

The SystemVerilog side:

tb_dpi.sv

```
module tb_dpi;

  // C functions callable from SystemVerilog
  import "DPI-C" pure    function int  c_alu_model(input int op, input int a, input int b);
  import "DPI-C"         function void c_minmax(input int a, input int b,
                                                output int lo, output int hi);
  import "DPI-C" context function void c_greet(input string name);

  // SystemVerilog function callable from C
  export "DPI-C" function sv_log;

  function void sv_log(input string msg);
    $display("[SV] %s", msg);
  endfunction

  int lo, hi;

  initial begin
    $display("1. c_alu_model(ADD, 200, 100) = %0d", c_alu_model(0, 200, 100));
    $display("2. c_alu_model(SUB, 5, 10)    = %0d", c_alu_model(1, 5, 10));

    c_minmax(42, 7, lo, hi);
    $display("3. lo = %0d, hi = %0d", lo, hi);

    c_greet("ALU testbench");
  end

endmodule
```

The C side:

dpi_model.c

```c
#include <stdio.h>
#include "svdpi.h"

/* Exported from SystemVerilog: declared here so that C can call it. */
extern void sv_log(const char* msg);

/* A reference model of the ALU (opcode numbering as in alu_pkg). 9-bit result. */
int c_alu_model(int op, int a, int b)
{
    int r;
    a &= 0xFF;
    b &= 0xFF;
    switch (op) {
        case 0:  r = a + b;         break;   /* ALU_ADD */
        case 1:  r = a - b;         break;   /* ALU_SUB */
        case 2:  r = a & b;         break;   /* ALU_AND */
        case 3:  r = a | b;         break;   /* ALU_OR  */
        case 4:  r = a ^ b;         break;   /* ALU_XOR */
        case 5:  r = (~a) & 0xFF;   break;   /* ALU_NOT */
        case 6:  r = a << 1;        break;   /* ALU_SHL */
        default: r = a >> 1;        break;   /* ALU_SHR */
    }
    return r & 0x1FF;
}

/* Output arguments arrive as pointers. */
void c_minmax(int a, int b, int* lo, int* hi)
{
    *lo = (a < b) ? a : b;
    *hi = (a < b) ? b : a;
}

/* A context function: it calls back into SystemVerilog. */
void c_greet(const char* name)
{
    char buf[128];
    snprintf(buf, sizeof(buf), "hello from C, %s", name);
    sv_log(buf);
}
```

EXPECTED OUTPUT

```
1. c_alu_model(ADD, 200, 100) = 300
2. c_alu_model(SUB, 5, 10)    = 507
3. lo = 7, hi = 42
[SV] hello from C, ALU testbench
```

How to run it. Both files are given to the simulator, which compiles the C file with its built-in or the system C compiler. The exact command differs per tool; typically the C file is simply listed with the SystemVerilog files (for example `vcs -sverilog tb_dpi.sv dpi_model.c`, `xrun tb_dpi.sv dpi_model.c`, or `vlog tb_dpi.sv dpi_model.c` followed by `vsim`). On EDA Playground, add the C file as an extra file and enable the tool's DPI option.

Line by line: SystemVerilog

Code	Explanation
`import "DPI-C" pure function int c_alu_model(...)`	Declares the C function. It is `pure`: the result depends only on its three inputs.
`import "DPI-C" function void c_minmax(..., output int lo, output int hi);`	A function with two `output` arguments. It cannot be `pure`.
`import "DPI-C" context function void c_greet(input string name);`	`context` because the C code calls the exported function `sv_log`.

Code	Explanation
`export "DPI-C" function sv_log;`	Makes the SystemVerilog function below callable from C. The export names the function only.
`c_alu_model(0, 200, 100)`	Called exactly like a SystemVerilog function. 200 + 100 = 300.
`c_alu_model(1, 5, 10)`	5 - 10 = -5; masked to 9 bits this is `1FB` = 507, the same value the DUT produces (M1).
`c_minmax(42, 7, lo, hi);`	The C function writes through the pointers; `lo` and `hi` receive 7 and 42.
`c_greet("ALU testbench");`	C builds a message and passes it back to SystemVerilog, which prints it.

Line by line: C

Code	Explanation
`#include "svdpi.h"`	The standard DPI header. Not strictly needed for `int` and `const char*`, but always included.
`extern void sv_log(const char* msg);`	The C declaration of the exported SystemVerilog function. `string` maps to `const char*`.
`int c_alu_model(int op, int a, int b)`	`input int` arguments arrive by value as C `int`.
`return r & 0x1FF;`	Keeps 9 bits so that the model matches the width of the DUT's `result`.
`void c_minmax(int a, int b, int* lo, int* hi)`	`output int` arguments arrive as `int*`.
`sv_log(buf);`	A call from C into SystemVerilog. This is why `c_greet` had to be imported as `context`.

Using the model in the scoreboard is then one line: `exp = c_alu_model(int'(txn.op), txn.a, txn.b);`.

DPI compared with PLI/VPI

Property	DPI	PLI / VPI
Call style	A normal function call with typed arguments	C routine registered with the simulator and invoked as a `$user_task`
Argument passing	Directly, by the type mapping	Through API calls that fetch each argument
Access to the design hierarchy from C	No (only what is passed in or exported)	Yes: can walk the hierarchy, read and force any signal
Speed	High	Lower
Effort	Small	Large
Typical use	Reference models, algorithms, firmware co-simulation	Tools: debuggers, waveform dumpers, coverage tools

> **Interview trap:** "Can an imported C function consume simulation time?" An imported **function** cannot, like any function. An imported **task** can, but only indirectly, by calling an exported SystemVerilog task that contains delays or event controls; it must be declared `context`.

> **Common mistake:** A mismatch between the import declaration and the C definition, for example `input int` on one side and `long` or `char` on the other, or an `output` argument that is not a pointer in C. There is no checking across the language boundary; the result is wrong values or a crash.

> **Common mistake:** Declaring a function `pure` when it prints, reads a file, uses a static variable or has an `output` argument. A simulator is allowed to remove or merge calls to a pure function.

> **Common mistake:** Using `"DPI"` instead of `"DPI-C"`. `"DPI"` is the obsolete 2005 form with a different, simulator-dependent argument-passing convention for packed types.

Interview questions

Q1. What is DPI? The Direct Programming Interface: a standard mechanism for calling C functions from SystemVerilog (import) and SystemVerilog functions and tasks from C (export), with arguments passed directly through a defined type mapping.

Q2. What is the difference between DPI import and export? An import declares a function implemented in C so that SystemVerilog code can call it. An export makes a function or task implemented in SystemVerilog callable from C.

Q3. What do `pure` and `context` mean? A `pure` imported function has no side effects and its result depends only on its inputs; it must return a value and have only input arguments. A `context` imported function or task needs the calling SystemVerilog scope, typically because it calls exported SystemVerilog functions or tasks or uses VPI.

Q4. How do SystemVerilog types map to C? `byte` to `char`, `shortint` to `short`, `int` to `int`, `longint` to `long long`, `real` to `double`, `shortreal` to `float`, `string` to `const char*`, `chandle` to `void*`. Single `bit` and `logic` map to `svBit` and `svLogic`; packed vectors map to arrays of `svBitVecVal` or `svLogicVecVal`. Output and inout arguments become pointers.

Q5. Why use DPI instead of PLI or VPI? It is far simpler and faster for calling functions: no registration, no API calls to fetch arguments. VPI is still needed when C code must explore or modify the design hierarchy.

Q6. Give a use of DPI in a verification environment. Calling a C reference model from the scoreboard to compute expected results; reusing a C CRC routine; letting C firmware perform register accesses through exported bus-functional tasks.

4.4 Simulator System Tasks and Functions

System tasks and functions are the built-in routines whose names start with `$`. They are part of the language, are executed by the simulator, and (with a few exceptions such as `$clog2` and `$bits` with constant arguments) are ignored or rejected by synthesis. This section is a reference for the ones a verification engineer uses constantly, with the behaviour differences that are asked in interviews.

Printing: $display, $write, $strobe, $monitor

Task	When it prints	Newline added	Values shown	Typical use
`$display`	Immediately, when the statement executes	Yes	Current values at that instant	General messages
`$write`	Immediately	No	Current values	Building one line from several calls
`$strobe`	At the **end of the current time step** (Postponed region)	Yes	Final, settled values of the time step, after nonblocking assignments	Printing the result of `<=` assignments
`$monitor`	At the end of every time step in which **any of its arguments changed**	Yes	Final values	Simple continuous tracing in small testbenches

Additional facts:

- Each has variants that set the default radix of arguments printed without a format: `$displayb`, `$displayo`, `$displayh` (and the same for `$write`, `$strobe`, `$monitor`).
- Only **one** `$monitor` is active at a time; a new call replaces the previous one. `$monitoroff` and `$monitoron` pause and resume it.
- `$monitor` does not react to changes of `$time` itself.
- The file versions take a file descriptor first: `$fdisplay`, `$fwrite`, `$fstrobe`, `$fmonitor`.

tb_display_strobe.sv

```
module tb_display_strobe;

  logic [3:0] a;

  initial begin
    a = 4'd1;
    $display("[%0t] $display : a = %0d", $time, a);
    $strobe ("[%0t] $strobe  : a = %0d", $time, a);
    a = 4'd2;
    a <= 4'd3;
    $display("[%0t] $display : a = %0d", $time, a);
    #10;
    $display("[%0t] $display : a = %0d", $time, a);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] $display : a = 1
[0] $display : a = 2
[0] $strobe  : a = 3
[10] $display : a = 3
```

Line of output	Explanation
`$display : a = 1`	`$display` prints immediately: `a` is 1 at that moment.
`$display : a = 2`	The blocking assignment `a = 2` has happened. The nonblocking `a <= 3` is only scheduled, so `a` is still 2.

Line of output	Explanation
`$strobe : a = 3`	Although `$strobe` was called second in the code, it prints at the end of time step 0, after the nonblocking assignment has updated `a` to 3.
`[10] ... a = 3`	The value persists.

tb_monitor.sv

```
module tb_monitor;

  int a, b;

  initial $monitor("[%0t] monitor: a=%0d b=%0d", $time, a, b);

  initial begin
    #5  a = 1;
    #5  b = 2;
        a = 3;
    #5  a = 3;          // no change: nothing is printed
    #5  $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[0] monitor: a=0 b=0
[5] monitor: a=1 b=0
[10] monitor: a=3 b=2
```

`$monitor` prints once when it is first called, then once per time step in which an argument changed. At time 10 two variables changed but only one line is printed, showing the final values. At time 15 the assignment does not change `a`, so nothing is printed.

Format specifiers

Specifier	Prints	Specifier	Prints
`%d` / `%0d`	Decimal / decimal without padding	`%s`	String
`%h` or `%x`	Hexadecimal	`%c`	One character
`%b`	Binary	`%t`	Time, formatted by `$timeformat`
`%o`	Octal	`%f`, `%e`, `%g`	Real: fixed, exponential, shorter of the two
`%p` / `%0p`	Any aggregate in assignment-pattern form (arrays, structs, enums by name) / compact form	`%m`	Hierarchical name of the current scope (takes no argument)
`%v`	Net value with strength	`%%`	A literal percent sign

A width may be given (`%8h`, `%-10s` for left-justified). A leading `0` in `%0d`, `%0h`, `%0t` means "minimum width, no padding".

Messages with severity

Task	Severity	Effect on simulation	Typical use
`$info("...")`	Information	Continues	Progress messages
`$warning("...")`	Warning	Continues	Suspicious but tolerated situations
`$error("...")`	Error	Continues; the simulator counts it	Check failures (scoreboard mismatch)
`$fatal(n, "...")`	Fatal	**Terminates** the simulation	Unrecoverable problems: null `vif`, randomization failure, timeout

The first argument of `$fatal` is the finish number (0, 1 or 2, controlling how much is printed), not a message: write `$fatal(1, "text")`. The severity tasks print the file name, line number, scope and time automatically, which `$display` does not. They are also the action tasks used in assertions (M10). Regression scripts decide pass or fail by searching the log for the error and fatal counts.

Ending and pausing simulation

Task	Effect
`$finish;` or `$finish(n);`	Ends the simulation. `n` = 0, 1 or 2 selects how much diagnostic information is printed (default 1)
`$stop;`	Suspends the simulation and returns control to the interactive prompt; it can be continued. In batch mode many tools treat it like `$finish`
`$exit;`	Ends a `program` block and waits for other programs; rarely used outside program-based testbenches
`$fatal(...)`	Reports a fatal error and ends the simulation

`final` blocks are executed when `$finish` is called; they run in zero time and are used to print summaries.

Time

Function / task	Returns or does
`$time`	Current time as a 64-bit integer in the time unit of the calling scope, rounded
`$realtime`	Current time as a `real`, including the fraction
`$stime`	Low 32 bits of the current time
`$timeformat(unit, precision, "suffix", min_width);`	Sets how `%t` prints. Call it once at the start of the test: `$timeformat(-9, 2, " ns", 10);`
`$printtimescale;`	Prints the time unit and precision of the current scope

Details are in M1, Section 2.7.

Random numbers

Function	Returns	Notes
`$random` or `$random(seed)`	**Signed** 32-bit integer	Legacy Verilog. `$random % 10` lies in -9 to 9. The idiom `{$random} % 10` forces it non-negative
`$urandom` or `$urandom(seed)`	**Unsigned** 32-bit integer	SystemVerilog. Uses the per-process random generator, so results are stable when unrelated code is added (random stability, M5)
`$urandom_range(max, min)`	Unsigned integer in the inclusive range	`min` defaults to 0. If `max` is less than `min` the two are swapped
`std::randomize(var) with {...}`	1 on success	Randomises ordinary variables with constraints (M5)
`$dist_uniform`, `$dist_normal`, `$dist_exponential`, `$dist_poisson`	Values with the named distribution	Take a seed variable as first argument

In new testbench code use `$urandom`, `$urandom_range` and `randomize()`. The simulation seed is set from the simulator command line (the option name is tool-specific) so that a failing run can be reproduced.

Command-line arguments: plusargs

Plusargs are options beginning with `+` on the simulator command line that the testbench can read at run time. They let one compiled testbench run many configurations without recompiling.

Function	Returns	Behaviour
`$test$plusargs("STRING")`	Non-zero if a plusarg **starting with** STRING exists	A prefix match: `$test$plusargs("DEBUG")` is true for `+DEBUG` and also for `+DEBUG_LEVEL=3`
`$value$plusargs("NAME=%d", variable)`	Non-zero if a matching plusarg was found	Converts the text after the matched prefix according to the format (`%d`, `%h`, `%s`, `%f` ...) and stores it in the variable. If not found, the variable is left unchanged

tb_plusargs.sv

```
module tb_plusargs;

  int    num_txn = 10;            // defaults, used when the plusarg is absent
  string test_name = "default_test";
  bit    verbose;

  initial begin
    if (!$value$plusargs("NUM_TXN=%d", num_txn))
      $display("NUM_TXN not given, using default %0d", num_txn);

    void'($value$plusargs("TEST=%s", test_name));

    verbose = $test$plusargs("VERBOSE") ? 1'b1 : 1'b0;

    $display("test=%s num_txn=%0d verbose=%b", test_name, num_txn, verbose);
  end

endmodule
```

Run without any plusargs:

EXPECTED OUTPUT

```
NUM_TXN not given, using default 10
test=default_test num_txn=10 verbose=0
```

Run with `+NUM_TXN=250 +TEST=corner_test +VERBOSE` added to the simulator's run command:

EXPECTED OUTPUT

```
test=corner_test num_txn=250 verbose=1
```

Code	Explanation
`int num_txn = 10;`	The variable is given its default first, because `$value$plusargs` leaves it untouched when the plusarg is missing.
`$value$plusargs("NUM_TXN=%d", num_txn)`	Searches the command line for `+NUM_TXN=`, converts what follows as a decimal number and stores it. The return value tells whether it was found.
`void'(...)`	The return value is deliberately discarded for `TEST`: a missing plusarg simply keeps the default.
`$test$plusargs("VERBOSE")`	A presence check for a flag with no value.

This is how a test name, a transaction count, a verbosity level or a watchdog limit is selected per run. UVM selects the test class in the same way, with `+UVM_TESTNAME=`.

Loading memories: $readmemh and $readmemb

Task	Reads
`$readmemh("file", mem);`	Hexadecimal values from a text file into an unpacked array
`$readmemb("file", mem);`	Binary values
`$readmemh("file", mem, start, end);`	Only into the given address range
`$writememh("file", mem);` / `$writememb`	Writes the array contents to a file

File format: values separated by white space or newlines; `@<hex address>` sets the address of the next value; `//` and `/* */` comments are allowed; `_` may separate digits; `x` and `z` are allowed.

```
// mem_init.hex
@0
de ad be ef       // addresses 0, 1, 2, 3
@10
55                // address 0x10
```

Typical uses: preloading a RAM or ROM model with a program, loading test vectors, initialising a reference memory. A runnable example is Exercise 6 in Section 6.5.

File input and output

Function / task	Purpose	Returns
`fd = $fopen("name", "mode");`	Opens a file. Modes: `"r"` read, `"w"` write (truncate), `"a"` append, also `"r+"`, `"w+"`, `"a+"`	A file descriptor (`int`); **0 if the open failed**
`$fclose(fd);`	Closes the file and flushes its buffer	

Function / task	Purpose	Returns
`$fdisplay(fd, ...)`, `$fwrite(fd, ...)`	Formatted output to the file	
`$fgets(str, fd)`	Reads one line into a string	Number of characters read; 0 at end of file
`$fscanf(fd, "format", args...)`	Formatted input	Number of items successfully read; -1 at end of file
`$sscanf(str, "format", args...)`	The same, reading from a string	Number of items read
`$feof(fd)`	End-of-file test	Non-zero at end of file
`$fflush(fd);`	Forces buffered output to disk	

tb_file_io.sv

```
module tb_file_io;

  int fd;
  int idx, a, b;

  initial begin
    // ---- write ----
    fd = $fopen("alu_vectors.txt", "w");
    if (fd == 0) $fatal(1, "cannot open alu_vectors.txt for writing");
    for (int i = 0; i < 3; i++)
      $fdisplay(fd, "%0d %0d %0d", i, i * 10, i * i);
    $fclose(fd);

    // ---- read back ----
    fd = $fopen("alu_vectors.txt", "r");
    if (fd == 0) $fatal(1, "cannot open alu_vectors.txt for reading");
    while ($fscanf(fd, "%d %d %d", idx, a, b) == 3)
      $display("vector %0d: a=%0d b=%0d", idx, a, b);
    $fclose(fd);
  end

endmodule
```

EXPECTED OUTPUT

```
vector 0: a=0 b=0
vector 1: a=10 b=1
vector 2: a=20 b=4
```

Code	Explanation
`fd = $fopen("alu_vectors.txt", "w");`	Opens (creates) the file for writing.
`if (fd == 0) $fatal(...)`	Always check the descriptor. Writing to descriptor 0 silently does nothing.
`$fdisplay(fd, ...)`	Like `$display`, but to the file. Three lines are written.
`$fclose(fd);`	Required before reading the file back, so that the data is flushed.
`while ($fscanf(...) == 3)`	Reads three numbers per iteration; the loop ends when fewer than three are read (end of file).

String formatting

Function / task	Use
`s = $sformatf("format", args...);`	A **function** that returns the formatted string. The most used of the three: `$sformatf("txn %0d: a=%0h", id, a)`
`$sformat(s, "format", args...);`	A **task** that writes the formatted text into its first argument
`$swrite(s, args...);`	Like `$write`, into a string
`$psprintf(...)`	A vendor extension equivalent to `$sformatf`; not in the standard. Use `$sformatf`

`$sformatf` is what builds the message strings of UVM's reporting macros, and what a transaction's `print` or `convert2string` method uses.

Bit and vector utility functions

Function	Returns	Example
`$bits(x)`	Number of bits of an expression or type	`$bits(int)` = 32, `$bits(logic [8:0])` = 9
`$clog2(n)`	Ceiling of log base 2 of `n`: the number of address bits needed for `n` locations	`$clog2(8)` = 3, `$clog2(9)` = 4, `$clog2(1)` = 0
`$countones(v)`	Number of bits that are 1	`$countones(8'b0010_1000)` = 2
`$countbits(v, '1, 'x)`	Number of bits having any of the listed values	Counts ones and X bits
`$onehot(v)`	1 if **exactly one** bit is 1	True for `0001_0000`
`$onehot0(v)`	1 if **at most one** bit is 1 (zero allowed)	True for `0000_0000` and `0001_0000`
`$isunknown(v)`	1 if any bit is X or Z	
`$size`, `$dimensions`, `$left`, `$right`, `$low`, `$high`	Array queries	M2
`$typename(x)`	A string describing the type of `x`	The exact text is tool-dependent; useful for debugging parameterized classes
`$signed(x)`, `$unsigned(x)`	Reinterpretation of signedness	M1
`$cast(dst, src)`	Checked dynamic cast	M1 (enums), M4 (class handles)

`$clog2` and `$bits` are constant functions when their arguments are constant, so they are used in synthesizable RTL for sizing: `localparam int AW = $clog2(DEPTH);`.

tb_utility_functions.sv

```
module tb_utility_functions;

  logic [7:0] v;
  string      s;
  int         r;

  initial begin
    $display("1. $clog2(1)=%0d  $clog2(8)=%0d  $clog2(9)=%0d  $clog2(1024)=%0d",
             $clog2(1), $clog2(8), $clog2(9), $clog2(1024));

    v = 8'b0010_1000;
    $display("2. %b : countones=%0d onehot=%b onehot0=%b",
             v, $countones(v), $onehot(v), $onehot0(v));
    v = 8'b0001_0000;
    $display("3. %b : countones=%0d onehot=%b onehot0=%b",
             v, $countones(v), $onehot(v), $onehot0(v));
    v = 8'b0000_0000;
    $display("4. %b : countones=%0d onehot=%b onehot0=%b",
             v, $countones(v), $onehot(v), $onehot0(v));
    v = 8'b0000_x001;
    $display("5. %b : isunknown=%b", v, $isunknown(v));

    s = $sformatf("txn %0d: a=%0h op=%s", 7, 8'hA5, "ALU_ADD");
    $display("6. %s (length %0d)", s, s.len());

    r = $urandom_range(10, 5);
    $display("7. $urandom_range(10, 5) is within 5..10: %b", r inside {[5:10]});
  end

endmodule
```

EXPECTED OUTPUT

```
1. $clog2(1)=0  $clog2(8)=3  $clog2(9)=4  $clog2(1024)=10
2. 00101000 : countones=2 onehot=0 onehot0=0
3. 00010000 : countones=1 onehot=1 onehot0=1
4. 00000000 : countones=0 onehot=0 onehot0=1
5. 0000x001 : isunknown=1
6. txn 7: a=a5 op=ALU_ADD (length 22)
7. $urandom_range(10, 5) is within 5..10: 1
```

Line 4 is the one to remember: for an all-zero vector `$onehot` is 0 and `$onehot0` is 1.

Waveform dumping

Task	Effect
`$dumpfile("waves.vcd");`	Names the VCD (Value Change Dump) file
`$dumpvars;`	Dumps every signal in the design
`$dumpvars(0, tb_top);`	Dumps `tb_top` and **all** levels below it
`$dumpvars(1, tb_top);`	Dumps only the signals directly in `tb_top`
`$dumpvars(2, tb_top.dut);`	Dumps `dut` and one level below it
`$dumpoff;` / `$dumpon;`	Pause and resume dumping, to limit file size
`$dumpall;`, `$dumpflush;`, `$dumplimit(bytes);`	Checkpoint, flush, and size limit

```
initial begin
  $dumpfile("alu.vcd");
  $dumpvars(0, tb_top);
end
```

VCD is the standard, portable format, and it is what EDA Playground displays. It records nets and variables of the static hierarchy only; class objects and dynamic arrays are not dumped. Each vendor also has a faster proprietary format with its own system tasks, which you will meet on the job.

Mathematical functions

`$ln`, `$log10`, `$exp`, `$sqrt`, `$pow(x, y)`, `$floor`, `$ceil`, `$sin`, `$cos`, `$tan`, `$asin`, `$acos`, `$atan`, `$atan2`, `$hypot`, `$sinh`, `$cosh`, `$tanh` take and return `real` values. They are used in real-number models and in checkers that compute expected analogue-like quantities.

Quick comparison of the most confused pairs

Pair	Difference
`$display` vs `$strobe`	Immediate values vs values at the end of the time step
`$display` vs `$write`	Newline vs no newline
`$display` vs `$monitor`	Prints once vs prints on every change
`$finish` vs `$stop`	Ends the simulation vs pauses it
`$error` vs `$fatal`	Reports and continues vs reports and terminates
`$random` vs `$urandom`	Signed, legacy vs unsigned, random-stable
`$time` vs `$realtime`	Rounded integer vs real with fraction
`$test$plusargs` vs `$value$plusargs`	Presence check (prefix match) vs presence check plus value extraction
`$sformatf` vs `$sformat`	Function returning a string vs task writing into a string argument
`$onehot` vs `$onehot0`	Exactly one bit set vs zero or one bit set
`$readmemh` vs `$fscanf`	Loads a whole array in a fixed format vs reads arbitrary formatted text item by item

Interview trap: "A variable is assigned with `<=` and printed with `$display` on the next line. Which value is printed?" The old value. The nonblocking update happens later in the time step. `$strobe` prints the new value.

Interview trap: "`$test$plusargs("TEST")` returns true although only `+TESTNAME=smoke` was given. Why?" It matches by prefix: any plusarg that begins with the string satisfies it.

Common mistake: `$fatal("message")`. The first argument must be the finish number: `$fatal(1, "message")`.

Common mistake: `x = $random % 8;` used as an array index. `$random` is signed, so the remainder can be negative. Use `$urandom_range(7)`.

Common mistake: Using a file descriptor without checking it, or never calling `$fclose`. A failed `$fopen` returns 0 and all later writes vanish; an unclosed file may be incomplete on disk if the simulation is killed.

Interview questions

Q1. What is the difference between `$display`, `$write`, `$strobe` and `$monitor`? `$display` prints immediately and appends a newline. `$write` prints immediately without a newline. `$strobe` prints once at the end of the current time step, showing final values. `$monitor` is set up once and then prints at the end of every time step in which one of its arguments changed.

Q2. What is the difference between `$finish` and `$stop`? `$finish` terminates the simulation. `$stop` suspends it and returns to the interactive prompt, from which it can be resumed.

Q3. What is the difference between `$random` and `$urandom`? `$random` returns a signed 32-bit value and comes from Verilog. `$urandom` returns an unsigned 32-bit value, uses the per-thread random number generator and therefore has random stability. `$urandom_range(max, min)` returns an unsigned value within an inclusive range.

Q4. How do you pass a value from the command line into a testbench? With a plusarg: add `+NAME=value` to the simulation command and read it with `$value$plusargs("NAME=%d", var)`. For a simple flag, `$test$plusargs("NAME")`.

Q5. How do you initialise a memory from a file? `$readmemh("file.hex", mem);` for hexadecimal data or `$readmemb` for binary, where `mem` is an unpacked array.

Q6. What does `$clog2` return and where is it used? The ceiling of the base-2 logarithm, which is the number of bits needed to address that many items. It is used to derive address or counter widths from a depth parameter: `localparam AW = $clog2(DEPTH);`.

Q7. What is the difference between `$onehot` and `$onehot0`? `$onehot` is true when exactly one bit is 1. `$onehot0` is true when at most one bit is 1, so it also accepts all zeros.

Q8. What is the difference between `$sformatf` and `$display`? `$display` prints to the log. `$sformatf` returns the formatted text as a string, which can be stored, passed to a function or included in another message.

Q9. Why use `$error` instead of `$display("ERROR ...")`? `$error` records the message with error severity: it adds the file, line, scope and time, increments the simulator's error count, and can be controlled by simulator options. A plain `$display` is just text that a script would have to search for.

Q10. How do you generate a waveform file? `$dumpfile("name.vcd");` followed by `$dumpvars(0, top_module);` in an `initial` block, or the vendor-specific equivalent for the tool's own format.

Part 5: Introduction to UVM

This part is an orientation, not a UVM course. Its purpose is to show that UVM contains no new language: it is a library of SystemVerilog classes that standardises the layered testbench you built in Module 9. If you can explain the M9 testbench, you can explain the outline of any UVM testbench, and that is the level expected of a fresher who lists "SystemVerilog" as the main skill.

5.1 What UVM Is and Why It Exists

In plain English

In Module 9 you made a number of design decisions: the generator talks to the driver through a mailbox; every component has a `run()` task; the environment constructs the components and connects them; the test decides how many transactions to send; an event tells the test when to stop. They were reasonable decisions, but they were *yours*. Another engineer would have named things differently, connected them differently, and ended the test differently. Your driver could not be dropped into that engineer's environment.

UVM (Universal Verification Methodology) is the industry's agreed set of answers to all of those decisions, delivered as a ready-made class library. Everybody's driver extends the same base class, receives transactions the same way, is constructed and connected at the same moments and reports errors with the same macros. Components written by different people, teams and companies therefore fit together.

Technical definition

UVM is a standardised verification methodology and an accompanying **SystemVerilog class library**. It was developed by Accellera (first released in 2011, derived mainly from OVM, with ideas from VMM) and is now also an IEEE standard, IEEE 1800.2. The library is delivered as:

- the package `uvm_pkg`, containing several hundred classes;
- the file `uvm_macros.svh`, containing macros that generate repetitive code.

A UVM testbench is ordinary SystemVerilog code that imports the package, includes the macros and extends the base classes.

What the library provides

Service	What it gives you	What you did by hand in M9
Base classes for every testbench role	`uvm_driver`, `uvm_monitor`, `uvm_sequencer`, `uvm_agent`, `uvm_scoreboard`, `uvm_env`, `uvm_test`	Wrote `alu_driver`, `alu_monitor`... from nothing
Phasing	All components are built, connected, run and reported in a fixed, automatic order	`alu_env` called constructors, then `run()` tasks inside a `fork`
Factory	Any class can be replaced by a derived class from the test, without editing the environment	Edited the environment or added a handle
Configuration database	Values and virtual interfaces are stored centrally and fetched by the components that need them	Passed `vif` through constructor arguments

Service	What it gives you	What you did by hand in M9
Sequences	Stimulus scenarios are objects, separate from the testbench structure, and can be layered and reused	A loop inside `alu_generator::run()`
TLM (transaction-level modelling) ports	Standard connections between components	Mailboxes
Objections	A standard way to decide when the test is over	The events `gen_done` and `drv_done`, counters
Reporting	Messages with ID, severity and verbosity, filterable and countable	`$display`, `$error`
Register layer (RAL)	A model of the DUT's registers with built-in access sequences	Not covered

Why the industry uses it

Benefit	Explanation
Reuse across projects (horizontal)	An agent for a standard protocol works unchanged in every testbench that has that protocol
Reuse from block to chip (vertical)	A block-level environment is instantiated inside the chip-level environment; its agents become passive monitors
Tests without changing the environment	Factory overrides and configuration let a test alter behaviour from outside
One vocabulary	Engineers moving between teams and companies recognise the structure immediately
Vendor independence	The same library runs on every major simulator

What UVM does not change

The DUT, the interface, the clocking blocks, the transaction's `rand` fields and constraints, covergroups, assertions, and the logic inside the driver's and monitor's main loops are all the same SystemVerilog as before. UVM changes the **framework around them**.

Interview questions

Q1. What is UVM? The Universal Verification Methodology: a standard methodology and SystemVerilog class library for building reusable, constrained-random, coverage-driven testbenches. It provides base classes for testbench components, phasing, a factory, a configuration database, sequences, TLM communication and reporting.

Q2. Is UVM a language? No. It is a library written in SystemVerilog. Using it requires only a SystemVerilog simulator plus the UVM package and macros.

Q3. Why is UVM used instead of a plain SystemVerilog layered testbench? For standardisation and reuse. A hand-written layered testbench works but follows private conventions. UVM fixes the conventions (structure, construction order, communication, end of test, reporting) so that components from different sources interoperate and tests can modify an environment without editing it.

5.2 How SystemVerilog Features Map to UVM

Every mechanism in UVM is built from a language feature covered in this course. This table is the core of the chapter: read each row as "UVM uses *this* to do *that*".

SystemVerilog feature (module)	How UVM uses it
Classes and objects (M4)	Everything in a UVM testbench is a class object: transactions, sequences, components, the test
Inheritance (M4)	User classes extend library base classes: `class alu_uvm_driver extends uvm_driver #(alu_seq_item);`
Virtual methods and polymorphism (M4)	The library calls methods such as `build_phase()` and `run_phase()` through base-class handles; your overriding versions execute. The factory returns a derived object through a base handle
`$cast` (M1, M4)	Used inside the factory and in `clone()`, which return base-type handles that are cast to the real type
Static properties and methods (M4)	The factory, the configuration database and the root of the component tree are reached through static methods: `uvm_config_db #(T)::set(...)`, `type_id::create(...)`
Parameterized classes (Section 2.1)	`uvm_driver #(REQ)`, `uvm_sequencer #(REQ)`, `uvm_sequence #(REQ)`, `uvm_analysis_port #(T)`, `uvm_config_db #(T)`
`typedef` (M1)	`typedef uvm_sequencer #(alu_seq_item) alu_sequencer;`
Packages and `::` (Part 1)	The whole library is `uvm_pkg`; `alu_seq_item::type_id::create()` is two scope resolutions and a static call
`rand`, constraints, `randomize()` (M5)	Sequence items are randomized by sequences; inline constraints shape scenarios
Transaction class with copy/compare/print (M6)	`uvm_sequence_item` provides `copy`, `clone`, `compare`, `print`, `convert2string` as standard methods to override or automate
Mailboxes (M7)	Replaced by TLM ports, exports and FIFOs, which are classes built on the same blocking `put`/`get` ideas
Events (M7)	`uvm_event`, and more importantly the objection mechanism for end of test
`fork`...`join` and process control (M7, Part 3)	The library forks every component's `run_phase` task in parallel and kills them when the phase ends; sequences run as processes that can be killed
Interfaces, virtual interfaces, clocking blocks (M8)	Used unchanged. The virtual interface is delivered through the configuration database
Layered testbench (M9)	Generator becomes sequence + sequencer; driver, monitor, scoreboard, environment and test keep their roles
Covergroups and assertions (M10)	Used unchanged. Covergroups usually live in a subscriber component connected to the monitor
Macros (`` `define ``)	`` `uvm_component_utils ``, `` `uvm_object_utils ``, `` `uvm_info ``, `` `uvm_error ``, `` `uvm_fatal `` generate boilerplate
Plusargs (Section 4.4)	`+UVM_TESTNAME=alu_base_test` selects the test; `+UVM_VERBOSITY=UVM_HIGH` sets message verbosity

> **How to say it in an interview:** "UVM is a SystemVerilog class library. Components are classes derived from `uvm_component`; transactions are classes derived from `uvm_sequence_item`. It relies on inheritance and virtual methods for phases and overrides, on parameterized classes for type-safe drivers, sequencers and ports, on static methods for the factory and the config database, and on virtual interfaces to reach the DUT. So the OOP features of SystemVerilog are the foundation that UVM is built on."

Interview questions

Q1. Which OOP concepts does UVM depend on? Inheritance (user classes extend UVM base classes), polymorphism through virtual methods (phase methods, factory overrides), parameterized classes (drivers, sequencers, ports, config database), static members (factory and config database access), and encapsulation.

Q2. Why must phase methods such as `run_phase` be virtual? The library holds every component through a `uvm_component` handle and calls `run_phase` on it. Only because the method is virtual does the call reach the overriding version in the user's derived class.

5.3 The Class Hierarchy: uvm_object and uvm_component

All UVM classes descend from a small tree. Knowing the two main branches is enough to place any class.

```
uvm_void
  |
uvm_object ------------------------------------------ data, things that come and go
  |
  +-- uvm_transaction
  |      +-- uvm_sequence_item ..................... alu_seq_item      (the transaction)
  |             +-- uvm_sequence #(REQ) ............ alu_sequence      (stream of items)
  |
  +-- uvm_report_object
         +-- uvm_component --------------------------- structure, things that stay
                |
                +-- uvm_test ....................... alu_base_test
                +-- uvm_env ........................ alu_uvm_env
                +-- uvm_agent ...................... alu_agent
                +-- uvm_driver #(REQ) .............. alu_uvm_driver
                +-- uvm_monitor .................... alu_uvm_monitor
                +-- uvm_sequencer #(REQ) ........... alu_sequencer
                +-- uvm_scoreboard ................. alu_uvm_scoreboard
                +-- uvm_subscriber #(T) ............ a coverage collector
```

(Intermediate library classes are omitted from the sketch.)

uvm_object versus uvm_component

Property	`uvm_object`	`uvm_component`
Represents	Data and behaviour that is created and discarded during the run: transactions, sequences, configuration objects	The static structure of the testbench: driver, monitor, agent, environment, test
Lifetime	Created whenever needed, at any time; garbage-collected when no longer referenced	Created once in the build phase at time 0; exists until the end of simulation
Place in a hierarchy	None	Has a parent and a full hierarchical name, for example `uvm_test_top.env.agent.drv`

Property	`uvm_object`	`uvm_component`
Phases	None	`build_phase`, `connect_phase`, `run_phase` and the others are called automatically
Constructor	`function new(string name = "alu_seq_item");`	`function new(string name, uvm_component parent);`
Registration macro	`` `uvm_object_utils(alu_seq_item) ``	`` `uvm_component_utils(alu_uvm_driver) ``
Created with	`alu_seq_item::type_id::create("req")`	`alu_uvm_driver::type_id::create("drv", this)`
In the M9 testbench	`alu_txn`	`alu_generator`, `alu_driver`, `alu_monitor`, `alu_scoreboard`, `alu_env`, `alu_test`

The two constructor signatures are worth memorising; a wrong one is the most common beginner compile error in UVM. A component's constructor takes its parent, which is how the component tree is formed.

Interview questions

Q1. What is the difference between `uvm_object` and `uvm_component`? `uvm_component` classes form the static testbench hierarchy: they are created once during the build phase, have a parent and a hierarchical name, and take part in phases. `uvm_object` classes are transient data such as transactions and sequences: they have no parent, no phases, and are created and destroyed throughout the simulation. `uvm_component` itself is derived from `uvm_object`.

Q2. From which class is a transaction derived? A driver? A transaction from `uvm_sequence_item`. A driver from `uvm_driver #(REQ)`, which is a `uvm_component`.

5.4 Phases

The problem phases solve

In M9 the environment's constructor created the components, then connected the mailboxes, then `run()` forked the components' tasks, and the test printed a report at the end. That order was coded by hand in `alu_env` and `alu_test`. In UVM the order is built into the library: every component implements the phase methods it needs, and the library calls them on **all** components at the right time.

The common phases

Phase	Kind	Order through the hierarchy	What you put in it	M9 equivalent
`build_phase`	function	Top-down (parent before children)	Create child components; fetch configuration and the virtual interface	Constructors: `gen = new(...)`, `drv = new(...)` in `alu_env::new`
`connect_phase`	function	Bottom-up	Connect TLM ports between components	Passing `gen2drv_mbx` and `mon2scb_mbx` to both ends

Phase	Kind	Order through the hierarchy	What you put in it	M9 equivalent
`end_of_elaboration_phase`	function	Bottom-up	Final adjustments; print the topology	
`start_of_simulation_phase`	function	Bottom-up	Print banners and configuration	
`run_phase`	**task**	All components in parallel	Everything that consumes time: driving, monitoring, starting sequences	Each component's `task run()` inside the `fork` of `alu_env::run()`
`extract_phase`	function	Bottom-up	Collect final values from components	
`check_phase`	function	Bottom-up	Check that nothing is left over (empty queues, all responses seen)	End-of-test checks in the scoreboard
`report_phase`	function	Bottom-up	Print the summary and pass/fail result	`alu_scoreboard::report()`
`final_phase`	function	Top-down	Close files	

```
time 0 (no time passes)           time advances            end of test (no time passes)

build -> connect -> end_of_elaboration -> start_of_simulation
                                  |
                                  v
                  run_phase  (every component's task, forked in parallel)
                                  |
                                  v
                  extract -> check -> report -> final

function phases                  the only task phase               function phases
```

Points to know:

- Only `run_phase` is a task. All other phases are functions and therefore take no simulation time (M3).
- `build_phase` is top-down because a parent must exist, and must have decided its configuration, before its children are created. `connect_phase` runs after the whole tree has been built, so every port that needs connecting already exists.
- The `run_phase` is internally divided into optional finer run-time phases (reset, configure, main, shutdown, each with pre- and post- phases) that run in parallel with it. Most testbenches use only `run_phase`.
- The phases start when the top module calls `run_test()`.

Interview questions

Q1. What are UVM phases and why are they needed? Phases are predefined steps that the library executes on every component in a fixed order: build, connect, end of elaboration, start of simulation, run, extract, check, report, final. They synchronise the construction, connection, execution and reporting of all components without the user coding that order.

Q2. Which phase is a task? `run_phase` (and its run-time sub-phases). All the others are functions.

Q3. Why is `build_phase` top-down and `connect_phase` bottom-up? Building is top-down because a parent creates its children and may configure them first. Connecting happens after everything exists; proceeding bottom-up ensures lower-level connections are made before the levels above them rely on them.

5.5 The Factory

In M9 the environment contained `drv = new(...)`, which fixes the class of the driver at that line. To run one test with an error-injecting driver you would have to edit the environment. The **factory** removes the need. It is a central table of all registered classes, plus a list of *overrides*. Components and objects are not constructed with `new` directly but by asking the factory: `alu_uvm_driver::type_id::create("drv", this)`. The factory normally returns an `alu_uvm_driver`. If a test has registered an override, it returns an object of the overriding **derived** class instead, through the same base-type handle. The environment's source code does not change.

Three language features make this work, all from earlier modules: the registration macro (`` `uvm_component_utils `` or `` `uvm_object_utils ``) adds a nested proxy type called `type_id` with a **static** `create` method; the object returned is a derived class assigned to a base-class handle (**upcasting**); and because the methods are **virtual**, the derived behaviour runs. Typical uses are replacing a transaction with a version that has extra constraints, or a driver with one that injects protocol errors, for one test only.

```
// In the environment (never changes):
drv = alu_uvm_driver::type_id::create("drv", this);

// In one particular test's build_phase: use the error-injecting driver everywhere
alu_uvm_driver::type_id::set_type_override(alu_err_driver::get_type());
```

5.6 The Configuration Database

In M9 the virtual interface travelled from `tb_top` to the driver through a chain of constructor arguments: top to test, test to environment, environment to driver. Every level had to know about it. The **configuration database**, `uvm_config_db`, is a central store that replaces the chain. One piece of code deposits a value under a name and says which components may see it; any of those components fetches it by the same name.

`uvm_config_db #(T)` is a parameterized class (Section 2.1) used only through two static methods. `set(context, instance_path, field_name, value)` stores a value; `get(context, instance_path, field_name, variable)` retrieves it and returns 1 if it was found. The type parameter `T` must be the same in the `set` and the `get`, and so must the field name string. Its most common use is passing the virtual interface from the top module to the driver and the monitor; it is also used for configuration objects (number of transactions, active or passive agent, and similar settings).

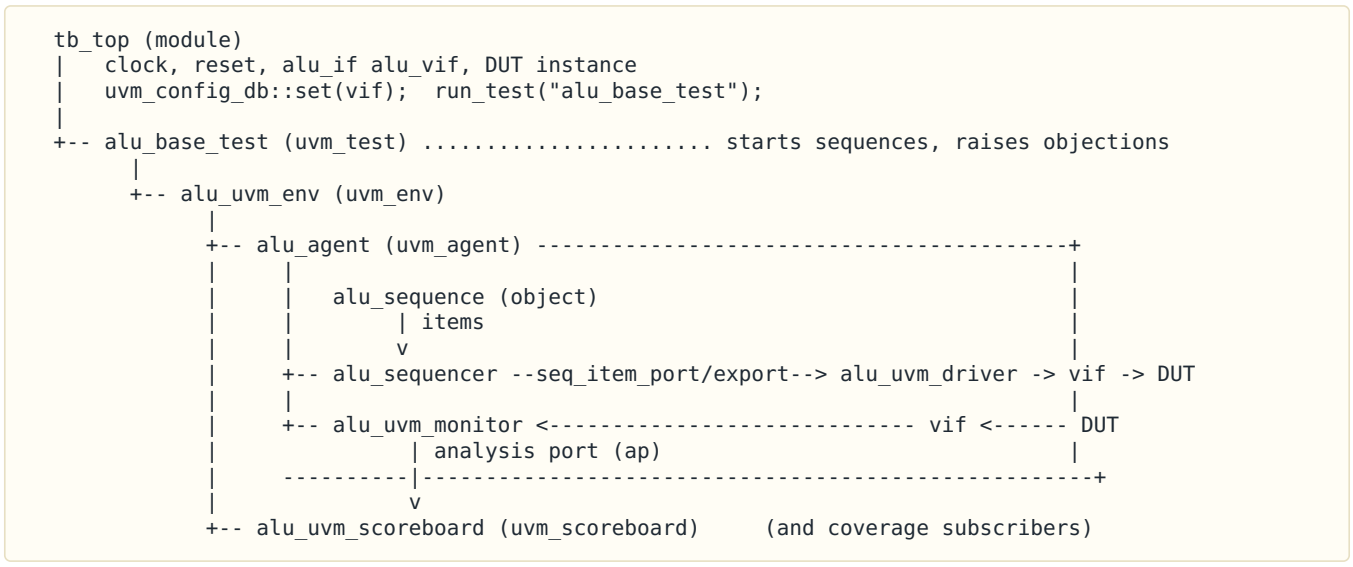
```
// In tb_top, before run_test(): visible to every component ("*")
uvm_config_db #(virtual alu_if)::set(null, "*", "vif", alu_vif);

// In the driver's build_phase
if (!uvm_config_db #(virtual alu_if)::get(this, "", "vif", vif))
  `uvm_fatal("NOVIF", "virtual interface was not set for the driver")
```

> **Interview trap:** "The driver's `get` from the config database fails although `set` was called. Why?" The usual causes are a mismatch in the type parameter (for example `virtual alu_if` in one place and `virtual alu_if.DRV` in the other), a different field name string, an instance path in `set` that does not cover the driver, or calling `set` after `run_test()` has already started the build phase.

5.7 UVM Components at a High Level

The structure of a UVM testbench



Each element in one paragraph

Sequence item (`alu_seq_item`, extends `uvm_sequence_item`). The transaction: the same `rand` fields and constraints as `alu_txn`, plus the standard object methods. It is what flows from the sequence to the driver and from the monitor to the scoreboard.

Sequence (`alu_sequence`, extends `uvm_sequence #(alu_seq_item)`). An object whose `body()` task creates items, randomizes them and sends them. It is the stimulus *scenario*. It replaces the loop inside `alu_generator::run()`, but being a separate object it can be swapped per test, layered inside other sequences, and reused. Sequences are not components: they are created and started by the test and disappear when finished.

Sequencer (`alu_sequencer`, a `uvm_sequencer #(alu_seq_item)`). A component that sits between sequences and the driver. It hands the driver one item at a time when the driver asks, and arbitrates if several sequences run at once. In M9 the mailbox `gen2drv_mbx` did the simple version of this job. Most sequencers are just a `typedef` of the library class.

Driver (`alu_uvm_driver`, extends `uvm_driver #(alu_seq_item)`). Converts items into pin activity through the virtual interface, exactly like `alu_driver`. The loop is: `seq_item_port.get_next_item(req);` drive the item; `seq_item_port.item_done();`. `get_next_item` blocks until a sequence supplies an item (compare with `gen2drv_mbx.get(txn)`); `item_done` tells the sequence that the item has been driven, which unblocks it (the handshake that `drv_done` provided in M9).

Monitor (`alu_uvm_monitor`, extends `uvm_monitor`). Passive. Samples the interface through `mon_cb`, rebuilds items, and publishes each one by calling `ap.write(item)` on its **analysis port**. It does not know or care who is listening.

Agent (`alu_agent`, extends `uvm_agent`). A container for one interface's sequencer, driver and monitor. It has a mode: **active** (all three; it drives the DUT) or **passive** (monitor only; it just observes). The same agent is active at block level and passive at chip level where real logic drives that interface. The M9 testbench had no agent level because it had only one interface.

Scoreboard (`alu_uvm_scoreboard`, extends `uvm_scoreboard`). Receives items from the monitor through an analysis connection, computes the expected result, compares, counts, and reports in `report_phase`. The checking logic is the same as in `alu_scoreboard`.

Environment (`alu_uvm_env`, extends `uvm_env`). Creates the agent(s), the scoreboard and coverage collectors in `build_phase` and connects them in `connect_phase`. Same role as `alu_env`.

Test (`alu_base_test`, extends `uvm_test`). The top-level component. Creates the environment, applies configuration and factory overrides, starts the sequences in `run_phase`, and controls the end of test with objections. A project has many tests, usually derived from one base test, all using the same environment. The test to run is chosen by name with `run_test("alu_base_test")` or `+UVM_TESTNAME=...`.

Top module (`tb_top`). Still a module: it instantiates the DUT and the interface, generates clock and reset, puts the virtual interface into the configuration database and calls `run_test()`.

Side-by-side: the M9 layered testbench and UVM

Role	M9 layered testbench	UVM equivalent	What changed
Transaction	`alu_txn`	`alu_seq_item` extends `uvm_sequence_item`	Inherits standard copy/ compare/print; created by the factory
Stimulus scenario	Loop in `alu_generator::run()`	`alu_sequence::body()`	Separated from structure; selectable per test
Passing stimulus to the driver	`gen2drv_mbx` (`mailbox #(alu_txn)`)	`alu_sequencer` with `seq_item_port` / `seq_item_export`	A pull model with a built-in handshake
Driver	`alu_driver` with `task run()`	`alu_uvm_driver` with `run_phase`	`get_next_item` / `item_done` instead of `mailbox.get`
Monitor	`alu_monitor` puts into `mon2scb_mbx`	`alu_uvm_monitor` calls `ap.write(item)`	Broadcast to any number of subscribers
Grouping per interface	(none)	`alu_agent`	New level, enabling active/ passive reuse
Checker	`alu_scoreboard` gets from `mon2scb_mbx`	`alu_uvm_scoreboard` with `write()`	Receives through an analysis connection
Container	`alu_env`	`alu_uvm_env`	Build and connect split into phases
Test	`alu_test`	`alu_base_test` extends `uvm_test`	Chosen by name at run time

Role	M9 layered testbench	UVM equivalent	What changed
Top	`tb_top` creates `alu_test` and calls `run()`	`tb_top` calls `run_test()`	The library creates the test
Getting `vif`	Constructor arguments	`uvm_config_db #(virtual alu_if)`	No chain of arguments
Construction	`new()`	`type_id::create()`	Overridable through the factory
Starting threads	`fork gen.run(); drv.run(); ... join_any`	All `run_phase` tasks forked by the library	Automatic
End of test	Events `gen_done`, `drv_done`, counters	Objections	Standard, component-independent
Messages	`$display`, `$error`	`` `uvm_info ``, `` `uvm_error ``, `` `uvm_fatal ``	Verbosity control, counts, IDs

TLM ports versus mailboxes

TLM stands for transaction-level modelling: components exchange whole transactions through method calls instead of signals.

Property	Mailbox (M7, M9)	UVM TLM connection
What the component holds	A handle to a shared mailbox object	A **port** object of its own
Who knows about the other end	Both ends share the same mailbox, created by the environment	Neither component knows the other; the environment connects port to export in `connect_phase`
Blocking transfer	`mbx.put(t)`, `mbx.get(t)`	`port.put(t)`, `port.get(t)`; for sequences, `get_next_item` / `item_done`
One-to-many broadcast	Not possible: one `get` removes the item; you need one mailbox per receiver	**Analysis port**: `ap.write(t)` delivers the same item to every connected subscriber (scoreboard, coverage, logger)
Does `write` block?	`put` blocks on a full bounded mailbox	`write` is a function: never blocks, takes no time
Buffering	The mailbox is the buffer	None in a port; add a `uvm_tlm_analysis_fifo #(T)` when the receiver needs to `get` at its own pace
Type safety	`mailbox #(alu_txn)`	`uvm_analysis_port #(alu_seq_item)`

The idea is the same: a typed channel that decouples the sender from the receiver. The TLM port adds that the two ends are developed without sharing an object, and the analysis port adds broadcast. Internally a `uvm_tlm_fifo` is itself implemented with a mailbox.

Objections versus the hand-written end of test

`run_phase` tasks such as a driver's `forever` loop never return, so the library needs to be told when the test is over. A component that still has work to do **raises an objection**; when it is done it **drops** it. The run phase ends when every raised objection has been dropped, and the library then kills the remaining `run_phase` processes and moves on to the reporting phases.

	M9 layered testbench	UVM
"I have work to do"	Implicit: the test waits for specific events	`phase.raise_objection(this);`
"I am done"	`-> gen_done;`, a transaction counter reaching its target	`phase.drop_objection(this);`
Who decides the test is over	Code in `alu_test` or `alu_env` that knows every component's event	The library: all objections dropped
Extra time for the last transaction to be checked	A fixed delay or waiting on the scoreboard count	A drain time on the objection, or a delay before dropping
Adding a new component that also has work	Edit the end-of-test code	The new component raises its own objection

Normally only the test (or the top sequence) raises an objection, around the sequences it starts. If no one ever raises an objection, the run phase ends at time 0 and nothing is simulated, which is a classic first-day mistake.

Interview questions

Q1. Name the components of a UVM testbench. Sequence item, sequence, sequencer, driver, monitor, agent, scoreboard, environment and test, with a top module that holds the DUT and interface. Coverage collectors (subscribers) are usually present too.

Q2. What is the difference between a sequence and a sequencer? A sequence is an object that generates a stream of items in its `body()` task. A sequencer is a component that routes items from sequences to the driver, one at a time, on request.

Q3. What is an agent? What is the difference between active and passive? An agent packages the sequencer, driver and monitor of one interface. An active agent contains all three and drives the DUT. A passive agent contains only the monitor and just observes; it is used when the interface is driven by real design logic, for example at chip level.

Q4. How does the driver get a transaction in UVM? Through its `seq_item_port`, which is connected to the sequencer's `seq_item_export`. The driver calls `get_next_item(req)`, drives the item, and calls `item_done()`.

Q5. What is an analysis port? A TLM port with a single non-blocking function `write(item)` that broadcasts the item to all connected subscribers. Monitors use it to send observed transactions to scoreboards and coverage collectors.

Q6. How does a UVM test end? By the objection mechanism. The run phase ends when all raised objections have been dropped.

Q7. How does the M9 mailbox between generator and driver relate to UVM? It corresponds to the sequencer-driver connection. The generator's role is taken by a sequence; the mailbox's role by the sequencer and the `seq_item_port`; the `get` becomes `get_next_item`, and the done-event becomes `item_done`.

5.8 A Minimal UVM Skeleton for the ALU

The code below is a complete but minimal UVM testbench for the course DUT. Read it side by side with the M9 testbench. **It cannot run by itself:** it needs a simulator with the UVM library enabled, and the files `alu_pkg.sv`, `alu.sv` (M1) and `alu_if.sv` (M8, summarised in Section 2.4) compiled before it. Treat it as an annotated reading exercise.

alu_uvm_skeleton.sv

```
// Requires: the UVM library, alu_pkg.sv, alu.sv and alu_if.sv.

package alu_uvm_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import alu_pkg::*;

  // ---------------------------------------------------------------------- transaction
  class alu_seq_item extends uvm_sequence_item;
    rand bit [7:0] a, b;
    rand alu_op_e  op;
    logic [8:0]    result;

    `uvm_object_utils(alu_seq_item)

    function new(string name = "alu_seq_item");
      super.new(name);
    endfunction

    function string convert2string();
      return $sformatf("op=%s a=%0d b=%0d result=%0d", op.name(), a, b, result);
    endfunction
  endclass

  // ---------------------------------------------------------------------- sequence
  class alu_sequence extends uvm_sequence #(alu_seq_item);
    `uvm_object_utils(alu_sequence)

    int unsigned num_items = 20;

    function new(string name = "alu_sequence");
      super.new(name);
    endfunction

    task body();
      repeat (num_items) begin
        req = alu_seq_item::type_id::create("req");
        start_item(req);                    // wait until the driver asks for an item
        if (!req.randomize()) `uvm_error("SEQ", "randomization failed")
        finish_item(req);                   // hand over; returns after item_done()
      end
    endtask
  endclass

  // ---------------------------------------------------------------------- sequencer
  typedef uvm_sequencer #(alu_seq_item) alu_sequencer;

  // ---------------------------------------------------------------------- driver
  class alu_uvm_driver extends uvm_driver #(alu_seq_item);
    `uvm_component_utils(alu_uvm_driver)

    virtual alu_if vif;

    function new(string name, uvm_component parent);
      super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      if (!uvm_config_db #(virtual alu_if)::get(this, "", "vif", vif))
        `uvm_fatal("NOVIF", "virtual interface not set for the driver")
    endfunction

    task run_phase(uvm_phase phase);
      vif.drv_cb.valid_in <= 1'b0;
      wait (vif.rst_n === 1'b1);
      forever begin
        seq_item_port.get_next_item(req);     // blocks until a sequence provides an item
        @(vif.drv_cb);
        vif.drv_cb.a        <= req.a;
        vif.drv_cb.b        <= req.b;
```

```
      vif.drv_cb.op       <= req.op;
      vif.drv_cb.valid_in <= 1'b1;
      @(vif.drv_cb);
      vif.drv_cb.valid_in <= 1'b0;
      seq_item_port.item_done();            // tell the sequence the item was driven
    end
  endtask
endclass

// ---------------------------------------------------------------- monitor
class alu_uvm_monitor extends uvm_monitor;
  `uvm_component_utils(alu_uvm_monitor)

  virtual alu_if                    vif;
  uvm_analysis_port #(alu_seq_item) ap;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    ap = new("ap", this);
    if (!uvm_config_db #(virtual alu_if)::get(this, "", "vif", vif))
      `uvm_fatal("NOVIF", "virtual interface not set for the monitor")
  endfunction

  task run_phase(uvm_phase phase);
    alu_seq_item item;
    forever begin
      @(vif.mon_cb);
      if (vif.mon_cb.valid_in === 1'b1) begin
        item    = alu_seq_item::type_id::create("item");
        item.a  = vif.mon_cb.a;
        item.b  = vif.mon_cb.b;
        item.op = vif.mon_cb.op;
        @(vif.mon_cb);                      // the result is registered: one cycle later
        item.result = vif.mon_cb.result;
        ap.write(item);                     // broadcast to all subscribers
      end
    end
  endtask
endclass

// ---------------------------------------------------------------- agent
class alu_agent extends uvm_agent;
  `uvm_component_utils(alu_agent)

  alu_sequencer   sqr;
  alu_uvm_driver  drv;
  alu_uvm_monitor mon;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon = alu_uvm_monitor::type_id::create("mon", this);
    if (get_is_active() == UVM_ACTIVE) begin
      sqr = alu_sequencer::type_id::create("sqr", this);
      drv = alu_uvm_driver::type_id::create("drv", this);
    end
  endfunction

  function void connect_phase(uvm_phase phase);
    if (get_is_active() == UVM_ACTIVE)
      drv.seq_item_port.connect(sqr.seq_item_export);
  endfunction
endclass

// ---------------------------------------------------------------- scoreboard
class alu_uvm_scoreboard extends uvm_scoreboard;
```

```
  `uvm_component_utils(alu_uvm_scoreboard)

  uvm_analysis_imp #(alu_seq_item, alu_uvm_scoreboard) item_export;
  int unsigned n_pass, n_fail;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    item_export = new("item_export", this);
  endfunction

  // Called by the monitor's ap.write(item)
  function void write(alu_seq_item item);
    logic [8:0] exp;
    case (item.op)
      ALU_ADD: exp = item.a + item.b;
      ALU_SUB: exp = item.a - item.b;
      ALU_AND: exp = {1'b0, item.a & item.b};
      ALU_OR : exp = {1'b0, item.a | item.b};
      ALU_XOR: exp = {1'b0, item.a ^ item.b};
      ALU_NOT: exp = {1'b0, ~item.a};
      ALU_SHL: exp = {item.a, 1'b0};
      ALU_SHR: exp = {2'b00, item.a[7:1]};
    endcase
    if (item.result !== exp) begin
      n_fail++;
      `uvm_error("SCB", $sformatf("MISMATCH %s expected=%0d", item.convert2string(), exp))
    end
    else n_pass++;
  endfunction

  function void report_phase(uvm_phase phase);
    `uvm_info("SCB", $sformatf("pass=%0d fail=%0d", n_pass, n_fail), UVM_LOW)
  endfunction
endclass

// ---------------------------------------------------------------------- environment
class alu_uvm_env extends uvm_env;
  `uvm_component_utils(alu_uvm_env)

  alu_agent          agent;
  alu_uvm_scoreboard scb;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agent = alu_agent::type_id::create("agent", this);
    scb   = alu_uvm_scoreboard::type_id::create("scb", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    agent.mon.ap.connect(scb.item_export);
  endfunction
endclass

// ---------------------------------------------------------------------- test
class alu_base_test extends uvm_test;
  `uvm_component_utils(alu_base_test)

  alu_uvm_env env;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
```

```
      env = alu_uvm_env::type_id::create("env", this);
    endfunction

    task run_phase(uvm_phase phase);
      alu_sequence seq;
      seq = alu_sequence::type_id::create("seq");
      phase.raise_objection(this);            // the test has work to do
      seq.start(env.agent.sqr);               // blocks until the sequence has finished
      #100;                                   // let the last result reach the scoreboard
      phase.drop_objection(this);             // the run phase may now end
    endtask
  endclass

endpackage : alu_uvm_pkg

// ---------------------------------------------------------------------- top module
module tb_top_uvm;
  import uvm_pkg::*;
  import alu_uvm_pkg::*;

  logic clk = 0;
  always #5 clk = ~clk;

  alu_if alu_vif (clk);

  alu dut (
    .clk       (clk),
    .rst_n     (alu_vif.rst_n),
    .valid_in  (alu_vif.valid_in),
    .a         (alu_vif.a),
    .b         (alu_vif.b),
    .op        (alu_vif.op),
    .valid_out (alu_vif.valid_out),
    .result    (alu_vif.result)
  );

  initial begin
    alu_vif.rst_n = 1'b0;
    repeat (2) @(posedge clk);
    alu_vif.rst_n = 1'b1;
  end

  initial begin
    uvm_config_db #(virtual alu_if)::set(null, "*", "vif", alu_vif);
    run_test("alu_base_test");
  end
endmodule
```

Reading guide

Element in the skeleton	SystemVerilog feature at work	Where you learned it
`class alu_seq_item extends uvm_sequence_item;`	Inheritance	M4
`rand bit [7:0] a, b; rand alu_op_e op;`	Random variables, enum	M5, M1
`` `uvm_object_utils(alu_seq_item) ``	A macro that registers the class with the factory and adds the nested `type_id` type	Section 5.5
`function new(string name = "alu_seq_item"); super.new(name);`	Constructor with a default argument, call to the base constructor	M4, M3
`class alu_sequence extends uvm_sequence #(alu_seq_item);`	Extending a specialisation of a parameterized class	Section 2.1

Element in the skeleton	SystemVerilog feature at work	Where you learned it
`req = alu_seq_item::type_id::create("req");`	Scope resolution twice, static method; `req` is a handle inherited from the base class	Section 1.3
`start_item(req); ... finish_item(req);`	Blocking tasks that perform the handshake with the driver	M7 (blocking communication)
`typedef uvm_sequencer #(alu_seq_item) alu_sequencer;`	`typedef` of a specialisation	Section 2.1
`function new(string name, uvm_component parent);`	The component constructor signature	Section 5.3
`function void build_phase(uvm_phase phase); super.build_phase(phase);`	Overriding a virtual method and calling the base version	M4
`uvm_config_db #(virtual alu_if)::get(this, "", "vif", vif)`	Static method of a parameterized class; virtual interface	Sections 2.1, 2.3
`task run_phase(uvm_phase phase);`	A virtual task, forked by the library for every component	M7
`vif.drv_cb.a <= req.a; @(vif.drv_cb);`	Clocking block drive and synchronisation, unchanged from M8/M9	M8
`seq_item_port.get_next_item(req); ... item_done();`	TLM replacing `gen2drv_mbx.get()` and the `drv_done` event	Section 5.7
`uvm_analysis_port #(alu_seq_item) ap; ap.write(item);`	Parameterized class; broadcast replacing `mon2scb_mbx.put()`	Section 5.7
`get_is_active() == UVM_ACTIVE`	A component property deciding whether the agent drives or only observes	Section 5.7
`drv.seq_item_port.connect(sqr.seq_item_export);`	Connecting two components in `connect_phase`	Section 5.4
`function void write(alu_seq_item item);` in the scoreboard	The function that the analysis connection calls; the reference model is the same `case` as in M9	M9
`item.result !== exp`	Case inequality so that X is detected	M1
`phase.raise_objection(this); ... phase.drop_objection(this);`	End-of-test control	Section 5.7
`seq.start(env.agent.sqr);`	Runs the sequence's `body()` on the given sequencer	Section 5.7
`run_test("alu_base_test");`	The library creates the test by name through the factory and starts the phases	Sections 5.4, 5.5

The monitor in this skeleton relies on the driver leaving one idle cycle between transactions. A production monitor handles back-to-back transactions with a small pipeline; that refinement is the same in UVM and in plain SystemVerilog.

5.9 SystemVerilog versus UVM

Aspect	SystemVerilog	UVM
What it is	A language (IEEE 1800)	A methodology and class library (IEEE 1800.2) written in that language
Provides	Data types, classes, randomization, coverage, assertions, interfaces, IPC	Base classes, phases, factory, configuration database, sequences, TLM, reporting, register layer
Testbench structure	Whatever the engineer designs (M9 is one design)	Prescribed: test, environment, agents, sequencer, driver, monitor, scoreboard
Component creation	`new()`	Factory `type_id::create()`
Component communication	Mailboxes, events, semaphores	TLM ports, exports, analysis ports, FIFOs
Stimulus	Generator class or procedural code	Sequences running on sequencers
Start-up and ordering	Coded by hand with `fork`	Automatic phases
End of test	Events, counters, `$finish`	Objections
Passing the virtual interface	Constructor or direct assignment	`uvm_config_db`
Messaging	`$display`, `$error`, `$fatal`	`` `uvm_info ``, `` `uvm_warning ``, `` `uvm_error ``, `` `uvm_fatal `` with verbosity and counts
Changing behaviour per test	Edit or extend the environment	Factory overrides, configuration, different sequences
Reuse between projects	Depends on private conventions	High, because conventions are standard
Learning effort	The language itself	The language first, then the library
Best suited to	Learning, small blocks, quick directed checks	Any testbench that must be maintained, shared or scaled

Key point: UVM does not replace SystemVerilog knowledge; it assumes it. Almost every UVM debugging session ends at a SystemVerilog concept: a null handle, a `$cast` that failed, a type-parameter mismatch, a blocking call that never returns, a race at a clock edge, a constraint conflict. That is why interviews for junior verification roles concentrate on SystemVerilog.

Common mistake: Memorising UVM class names without being able to say what each one does in plain terms. An interviewer who hears "sequencer" will ask "what would you use if UVM did not exist?" The answer is "a mailbox between a generator and a driver", and Module 9 is where you built it.

Interview questions

Q1. What is the difference between SystemVerilog and UVM? SystemVerilog is the language. UVM is a methodology and a class library written in SystemVerilog that standardises how a testbench is structured, built, connected, run and reported.

Q2. Can you build a testbench without UVM? Yes. A layered class-based testbench with a generator, driver, monitor, scoreboard and environment communicating through mailboxes does the same job. UVM adds standardisation, reuse, the factory, configuration, sequences and phasing.

Q3. What does UVM give you that mailboxes and `fork`...`join` do not? A common architecture that makes components interoperable; automatic phasing; replacement of classes from the test through the factory; hierarchical configuration; stimulus as reusable sequences; one-to-many analysis connections; a standard end-of-test mechanism; and uniform reporting.

Q4. Name three UVM mechanisms and the SystemVerilog feature behind each. The factory (static methods, inheritance, polymorphism and `$cast`); `uvm_config_db` (a parameterized class with static methods); phases (virtual methods called by the library on every component, with `run_phase` tasks forked as parallel processes).

Part 6: Module 11 Revision and Practice

6.1 Revision Summary

Packages and scope

- A package is a named scope for shared declarations: types, parameters, functions, tasks, classes, variables. It is compiled once, exists once and is never instantiated.
- Not allowed in a package: module or interface definitions and instances, procedural blocks, continuous assignments, hierarchical references to the outside.
- Three ways to use an item: `pkg::name`, `import pkg::name;` (explicit), `import pkg::*;` (wildcard).
- An explicit import is a local declaration: it collides with a local name. A wildcard import only offers candidates: a local name wins, and two packages offering the same name are a problem only if the name is used.
- An explicit import of an enum type does not import its labels.
- Imports are not transitive; `export` passes names on.
- Put the import in the module header when ports need the types.
- `` `include `` pastes text; `import` gives visibility. A class file is included in exactly one package.
- `$unit` is the anonymous compilation-unit scope. Avoid declaring or importing there.
- `::` selects a name from a scope (package, class, `$unit`, `std`, `local`); `.` selects a member of an instance.

Parameterized classes

- `class C #(type T = int, int N = 8);` Value parameters are constants; type parameters are types.
- Each distinct set of parameter values is a separate, unrelated class (a specialisation). No assignment or `$cast` between specialisations.
- Static members exist once per specialisation.
- `C` alone is the default specialisation in a declaration; with `::` write `C #()::member`.
- A common non-parameterized base class lets different specialisations share a handle type.
- A generic class may only use operations its type parameter supports. `==` on class handles compares handles; use `compare()` for objects.
- `mailbox #(T)` is the built-in parameterized class.
- A virtual interface to a parameterized interface carries the parameters: `virtual bus_if #(W) vif;`.

Process control

Tool	Stops or waits for
`wait fork`	All children of the calling process
`disable fork`	All children of the calling process, with descendants
`disable <name>`	Every activation of a named block or task
`p.kill()`	One process (with descendants)
`p.await()`	One process

Tool	Stops or waits for
`p.status()`	`FINISHED`, `RUNNING`, `WAITING`, `SUSPENDED`, `KILLED`

- A task call does not create a process; `process::self()` in a task returns the caller's handle.
- After `join_none` the child has not started: `wait (p != null)` before using its handle.
- Isolating fork: `fork begin fork ... join_any disable fork; end join` limits `disable fork` to the threads inside.
- A `process` cannot be created with `new`.

randcase, randsequence, DPI

- `randcase`: one branch chosen with probability weight / sum of weights. Weight 0 never; all zero takes no branch.
- `randsequence`: productions; `|` separates alternatives, `:=` gives a weight, `repeat (n)` repeats, `{ }` is a code block.
- DPI: `import "DPI-C" function ...` (SystemVerilog calls C), `export "DPI-C" function name;` (C calls SystemVerilog). `pure`: no side effects, no outputs, returns a value. `context`: calls back into SystemVerilog.
- Type mapping: `int` ↔ `int`, `byte` ↔ `char`, `shortint` ↔ `short`, `longint` ↔ `long long`, `real` ↔ `double`, `string` ↔ `const char*`, `chandle` ↔ `void*`; outputs are pointers.

System tasks

Remember	Detail
`$display` / `$strobe` / `$monitor`	Now / end of time step / on every change
`$error` / `$fatal(1, ...)`	Continue / terminate
`$finish` / `$stop`	End / pause
`$random` / `$urandom` / `$urandom_range(max, min)`	Signed legacy / unsigned stable / inclusive range
`$test$plusargs` / `$value$plusargs`	Prefix presence test / value extraction, returns found flag
`$fopen` returns 0 on failure	Always check, always `$fclose`
`$sformatf`	Returns a formatted string
`$clog2(n)`	Bits needed to address `n` items; `$clog2(1)` = 0
`$onehot` / `$onehot0`	Exactly one / at most one bit set
`$dumpfile`, `$dumpvars(0, top)`	VCD waveforms of the whole hierarchy

UVM in ten lines

1. UVM is a SystemVerilog class library (`uvm_pkg`) plus macros; a methodology, not a language.
2. `uvm_component`: permanent structure with a parent and phases. `uvm_object`: transient data.
3. Phases: build (top-down), connect, end of elaboration, start of simulation, **run (task)**, extract, check, report, final.
4. Factory: create with `type_id::create()` so that a test can override the class.
5. `uvm_config_db #(T)::set/get`: central store, mostly for the virtual interface.
6. Sequence item = transaction; sequence = stimulus scenario; sequencer = router to the driver.

7. Driver: `get_next_item(req)`, drive, `item_done()`.
8. Monitor: sample, `ap.write(item)` to an analysis port (broadcast).
9. Agent = sequencer + driver + monitor; active or passive. Environment contains agents and scoreboard. Test selects sequences and configuration.
10. End of test by objections: `raise_objection` / `drop_objection`.

6.2 Interview Questions and Answers

The questions inside each section cover the direct definitions. These are additional.

Basic level

Q1. What is the purpose of a package? To hold declarations that several design or testbench units share, in one named scope, so that they are written once and name clashes are avoided.

Q2. Write the three ways to use the type `alu_op_e` from `alu_pkg` in a module. `alu_pkg::alu_op_e op;` or `import alu_pkg::alu_op_e;` then `alu_op_e op;` or `import alu_pkg::*;` then `alu_op_e op;`.

Q3. Can a package contain a class? A module? An `always` block? A class yes. A module no. An `always` block no.

Q4. What does `#(type T = int)` mean in a class declaration? The class has a type parameter named `T`, with `int` as the default. Inside the class `T` is used as a data type, and each use of the class can supply a different type.

Q5. What is `mailbox #(alu_txn)`? The built-in mailbox class specialised so that it carries only `alu_txn` handles, with compile-time type checking.

Q6. What does `process::self()` return? A handle to the process that is currently executing the call.

Q7. What does `disable fork` do? Kills all child processes of the calling process, and their descendants.

Q8. What is the difference between `$display` and `$strobe`? `$display` prints immediately with the values at that instant. `$strobe` prints at the end of the current time step, after all assignments (including nonblocking ones) of that time step have taken effect.

Q9. What is `$urandom_range(15, 4)`? A random unsigned integer between 4 and 15 inclusive.

Q10. What is DPI used for? Calling C code from SystemVerilog and SystemVerilog from C, typically to reuse C reference models and algorithms.

Q11. What does UVM stand for and what is it? Universal Verification Methodology: a standard SystemVerilog class library and methodology for building reusable testbenches.

Q12. Name the UVM components that correspond to a generator and a mailbox in a layered testbench. A sequence (the generator's role) and a sequencer with the driver's `seq_item_port` (the mailbox's role).

Intermediate level

Q13. A module imports two packages with `::*`, and both define `MAX`. When does the compiler complain? Only when `MAX` is referenced without a package prefix. The unused collision is legal. Use `pkg_a::MAX`.

Q14. Why should classes be put in a package instead of being compiled as loose files? Loose class declarations go into the compilation-unit scope, whose extent depends on the tool's compile mode, so visibility is not portable. In a package the classes have a fixed scope, are compiled once, and are visible exactly where the package is imported.

Q15. A function in a package is used from RTL. Why should it be `automatic`? A static function has one shared copy of its arguments and local variables. If it is called from several places concurrently they interfere, and synthesis tools require automatic functions so that each call is independent logic.

Q16. What is the difference between a parameterized class and inheritance? A parameterized class generates several independent classes from one template, differing in types or constants fixed at compile time. Inheritance creates a related class that extends or overrides behaviour, and allows a base handle to refer to a derived object at run time.

Q17. How do you declare a queue that can hold `typed_item #(int)` and `typed_item #(string)` objects together? Make `typed_item` extend a non-parameterized base class, and declare the queue with the base type: `item_base q [$];`.

Q18. What is the difference between `fork ... join_any` followed by `disable fork`, and using a `process` handle? `disable fork` kills every child of the current process, which is simple but coarse and needs an isolating fork to be safe. A process handle refers to one thread, so that thread alone can be killed or awaited, and its status can be queried.

Q19. What states can `status()` return, and which will a parent usually see for a child that is waiting on a mailbox `get`? `FINISHED`, `RUNNING`, `WAITING`, `SUSPENDED`, `KILLED`. A child blocked on a mailbox is `WAITING`.

Q20. What is the difference between `randcase` and `if (($urandom % 10) < 6) ... else ...`? They can produce the same distribution, but `randcase` states the weights directly, scales to many branches, and lets the weights be expressions. The hand-written form is error-prone as branches are added.

Q21. In a DPI import, how does an `output int` argument appear in the C function? As a pointer, `int*`. The C function writes the result through it.

Q22. What is the difference between `$value$plusargs("N=%d", n)` returning 0 and returning 1? 0: no plusarg starting with `N=` was given, and `n` is unchanged. 1: it was found and `n` holds the converted value.

Q23. Why is the virtual interface passed through `uvm_config_db` and not through a constructor in UVM? Components are created by the factory with a fixed constructor signature `(name, parent)`, so extra constructor arguments are not possible. The configuration database also removes the need for every hierarchical level to pass the handle on.

Q24. What is the difference between `uvm_object_utils` and `uvm_component_utils`? Both register a class with the factory. `uvm_object_utils` is for classes derived from `uvm_object` (constructor with a name only); `uvm_component_utils` is for classes derived from `uvm_component` (constructor with name and parent).

Advanced and scenario-based

Q25. Two teams deliver packages that both `` `include "common_txn.sv" ``. Your environment imports both and gets "type mismatch" when connecting them. Explain and fix. The class was compiled into two packages, producing two unrelated classes with the same name. A handle of one cannot be passed where the other is expected. The class must be compiled in one package only (a shared package), which both teams import.

Q26. A timeout task containing `fork ... join_any disable fork;` works in a unit test, but in the full testbench the monitor stops working after the first timeout. Why? The task runs in the caller's process. That

process had earlier started the monitor with `join_none`, so the monitor is also its child, and `disable fork` killed it. Wrap the race in an isolating `fork begin ... end join`.

Q27. A test hangs. How would you find out which threads are still running? If the threads registered their `process` handles, print each handle's `status()` from a watchdog before calling `$fatal`: threads that are `WAITING` are the suspects. Otherwise use the simulator's thread view, or add a watchdog and progressively narrow down with messages. Typical causes are a `get` on a mailbox that nobody fills, an event triggered before the `@` started waiting, and `wait fork` with a never-ending child.

Q28. Why does `uvm_driver` have a type parameter? So that the `req` handle and the `seq_item_port` are typed with the user's transaction class. The driver code then accesses `req.a` directly, and connecting it to a sequencer of a different item type is a compile error. It is the same reason `mailbox #(alu_txn)` is preferred over a typeless mailbox.

Q29. The generic scoreboard works for `int` but reports every transaction object as a mismatch. Why? It compares with `==`. For class handles that compares identity, and the expected and actual transactions are different objects. Call the transaction's `compare()` method, or write the scoreboard against a base class that declares `compare` as a virtual method.

Q30. You must check an ALU against an existing C model. Outline the solution. Import the C function with DPI: `import "DPI-C" pure function int c_alu_model(input int op, input int a, input int b);`. In the scoreboard, call it with the fields of each observed transaction, mask the result to 9 bits, and compare with `!==` against the DUT result. Compile the C file together with the testbench.

Q31. A regression must run the same compiled testbench with different numbers of transactions and different tests. How? Use plusargs. Read `+NUM_TXN=` with `$value$plusargs` into a variable that has a default, and select the test by a `+TEST=` string (in UVM, `+UVM_TESTNAME=`). Set the random seed with the simulator's seed option.

Q32. Why must the parent wait before using a process handle set inside `fork ... join_none`? A thread created by `join_none` is only scheduled; it starts executing when the parent blocks or finishes. Until then the assignment `p = process::self()` has not happened and `p` is `null`.

Q33. What would break if `run_phase` were a function? A function cannot contain timing controls, so a driver could not wait for clock edges and a monitor could not sample over time. `run_phase` is the one phase that must consume simulation time, so it is a task; the library forks one process per component for it.

Q34. Explain, in terms of language features only, what happens in `drv = alu_uvm_driver::type_id::create("drv", this);`. `alu_uvm_driver::type_id` is a type declared inside the class by the registration macro. `create` is its static method. It asks the factory singleton for an object of the registered type, or of an overriding derived type, constructs it with `new(name, parent)`, and returns a base-type handle that is cast with `$cast` to the requested type and assigned to `drv`.

6.3 Output-Prediction Questions

Work each one out on paper first.

P1.

```
package cnt_pkg;
  int hits = 0;
  function automatic void hit();
    hits++;
  endfunction
endpackage

module m_a;
  initial begin
    #1 cnt_pkg::hit();
       cnt_pkg::hit();
  end
endmodule

module m_b;
  import cnt_pkg::*;
  initial begin
    #2 hit();
       $display("%0d", hits);
  end
endmodule

module p1;
  m_a u_a ();
  m_b u_b ();
endmodule
```

Answer: `3`. There is one `hits` in the simulation. Module `m_a` increments it twice at time 1 and `m_b` once at time 2.

P2.

```
package p_pkg;
  int v = 5;
  int w = 6;
endpackage

module p2;
  import p_pkg::*;
  int v = 50;
  initial $display("%0d %0d %0d", v, w, p_pkg::v);
endmodule
```

Answer: `50 6 5`. The local `v` takes precedence over the wildcard candidate. `w` comes from the package. The prefix still reaches the package's `v`.

P3.

```
class cnt #(type T = int);
  static int n;
  function new();
    n++;
  endfunction
endclass

module p3;
  cnt #(int) a, b;
  cnt #(bit) c;
  cnt        d;
  initial begin
    a = new();  b = new();  c = new();  d = new();
    $display("%0d %0d", cnt #(int)::n, cnt #(bit)::n);
  end
endmodule
```

Answer: `3 1`. `cnt` without parameters is the default specialisation `cnt #(int)`, so `a`, `b` and `d` share one counter. `cnt #(bit)` has its own.

P4.

```
module p4;
  process p_init, p_task, p_fork;

  task automatic grab();
    p_task = process::self();
  endtask

  initial begin
    p_init = process::self();
    grab();
    fork
      p_fork = process::self();
    join
    $display("%b %b", p_task == p_init, p_fork == p_init);
  end
endmodule
```

Answer: `1 0`. A task runs in its caller's process, so `p_task` is the `initial` process. A `fork` branch is a new process.

P5.

```
module p5;
  initial begin
    fork
      #50 $display("A");
    join_none

    fork
      #10 $display("B");
      #20 $display("C");
    join_any
    disable fork;

    #100 $display("D");
  end
endmodule
```

Answer: `B` then `D`. `join_any` returns at time 10. `disable fork` kills **all** children of the `initial` process: the `C` thread and also the `A` thread from the earlier fork.

P6.

```
module p6;
  int x;
  initial begin
    x = 1;
    x <= 5;
    $strobe ("S %0d", x);
    $display("D %0d", x);
    x = 2;
    #1 $display("E %0d", x);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
D 1
S 5
E 5
```

`$display` prints at once: 1. The blocking `x = 2` then executes, and at the end of the time step the nonblocking assignment sets `x` to 5 (its right-hand side was evaluated when the statement executed). `$strobe` prints after that: 5.

P7.

```
module p7;
  initial $display("%0d %0d %0d %b %b",
                   $clog2(5), $clog2(16), $clog2(17), $onehot(4'b0000), $onehot0(4'b0000));
endmodule
```

Answer: `3 4 5 0 1`. Five items need 3 address bits, 16 need 4, 17 need 5. All zeros is not one-hot, but it satisfies "at most one bit set".

P8.

```
module p8;
  initial begin
    for (int i = 0; i < 3; i++) begin : outer
      for (int j = 0; j < 3; j++) begin
        if (j == 1) disable outer;
        $display("%0d%0d", i, j);
      end
    end
    $display("end");
  end
endmodule
```

Answer: `00`, `10`, `20`, `end`. `outer` names the **body** of the outer loop. Disabling it abandons the rest of that iteration (including the inner loop) and the outer `for` continues with the next `i`.

P9.

```
module p9;
  int n = 7;
  initial begin
    if ($value$plusargs("N=%d", n)) $display("given %0d", n);
    else                            $display("default %0d", n);
  end
endmodule
```

Answer: with no plusarg, `default 7`. With `+N=12` on the command line, `given 12`. When the plusarg is absent the function returns 0 and does not modify the variable.

P10.

```
module p10;
  process        p;
  process::state s;
  initial begin
    fork
      begin
        p = process::self();
        #10;
      end
    join_none
    #1  s = p.status();  $display("%s", s.name());
    #20 s = p.status();  $display("%s", s.name());
  end
endmodule
```

Answer: `WAITING` then `FINISHED`. At time 1 the child is blocked in its delay. At time 21 it has run to completion. (The handle is valid at time 1 because the child started when the parent blocked on `#1`.)

6.4 Find-the-Error and Debugging Questions

Compile-time errors

E1.

```
package bad_pkg;
  logic clk;
  initial clk = 0;
  always #5 clk = ~clk;
endpackage
```

Procedural blocks are not allowed in a package. Generate the clock in a module or interface.

E2.

```
module m;
  import alu_pkg::alu_op_e;
  alu_op_e op;
  initial op = ALU_SUB;
endmodule
```

The explicit import brings in the type name only. `ALU_SUB` is not visible. Write `alu_pkg::ALU_SUB`, import the label too, or use `import alu_pkg::*;`.

E3.

```
package a_pkg;  int size = 4;   endpackage
package b_pkg;  int size = 8;   endpackage

module m;
  import a_pkg::*;
  import b_pkg::*;
  initial $display("%0d", size);
endmodule
```

`size` is offered by two wildcard imports and is used without a prefix: ambiguous. Write `a_pkg::size` or `b_pkg::size`.

E4.

```
module m;
  int count;
  import cfg_pkg::count;
endmodule
```

An explicit import declares `count` in the module scope, where `count` already exists. Either remove the local declaration, use a wildcard import (the local name then wins), or refer to `cfg_pkg::count`.

E5.

```
module dut (input alu_op_e op);
  import alu_pkg::*;
endmodule
```

The import is in the body, after the port list has already used the type. Move it into the header: `module dut import alu_pkg::*; (input alu_op_e op);`.

E6.

```
class stack #(type T = int);
  static int instances;
endclass

module m;
  initial $display("%0d", stack::instances);
endmodule
```

A parameterized class name cannot be used unadorned with `::` outside the class. Write `stack #()::instances` or `stack #(int)::instances`.

E7.

```
stack #(int)    s_int = new();
stack #(string) s_str;
initial s_str = s_int;
```

Different specialisations are different, unrelated types. The assignment is illegal, and `$cast` would fail too.

E8.

```
process p = new();
```

A process object cannot be constructed. Obtain a handle with `process::self()` from inside the process.

E9.

```
import "DPI-C" pure function void c_fill(output int value);
```

A `pure` function must return a value and may not have `output` or `inout` arguments. Remove `pure`, or return the value as the function result.

E10.

```
package tb_pkg;
  function automatic void wait_clk();
    @(posedge tb_top.clk);
  endfunction
endpackage
```

Two errors. A package may not contain a hierarchical reference to something outside it (`tb_top.clk`), and a function may not contain a timing control. Use a task that receives a virtual interface as an argument.

Run-time bugs (the code compiles)

D1.

```
process p;
initial begin
  fork
    begin  p = process::self();  forever #10;  end
  join_none
  p.kill();
end
```

Null handle error. The child has not started when `p.kill()` executes. Insert `wait (p != null);` before using the handle.

D2.

```
task run();
  fork  mon.run();  join_none
  repeat (10) begin
    fork
      drv.send();
      #1000;
    join_any
    disable fork;
  end
endtask
```

The first `disable fork` also kills `mon.run()`, because it is a child of the same process. Wrap the race in `fork begin ... end join`.

D3.

```
if ($test$plusargs("TEST")) run_special_test();
```

Run with `+TESTNAME=smoke`, the special test executes unexpectedly: `$test$plusargs` matches any plusarg that **starts with** the string. Use a more specific string or `$value$plusargs`.

D4.

```
int idx;
initial idx = $random % 16;      // used as mem[idx]
```

`$random` is signed, so `idx` can be anywhere from -15 to 15. Use `$urandom_range(15)`.

D5.

```
int fd;
initial begin
  fd = $fopen("/no/such/dir/log.txt", "w");
  $fdisplay(fd, "start of test");
end
```

The open fails and returns 0; the write is silently lost. Check `if (fd == 0) $fatal(1, ...)`, and close the file at the end.

D6.

```
int w_rd = 0, w_wr = 0;
initial repeat (100)
  randcase
    w_rd : do_read();
    w_wr : do_write();
  endcase
```

Both weights are 0, so no branch is ever taken: the loop does nothing, possibly with a warning. Initialise the weights to non-zero values.

D7.

```
int seed;
initial begin
  $value$plusargs("SEED=%d", seed);
  $display("using seed %0d", seed);
end
```

When `+SEED=` is not given, `seed` stays at its default 0 and the log misleadingly reports a seed. Check the return value and handle the missing case (and note that a return value ignored in this way also draws a compiler warning; use `void'(...)` if it is intended).

D8.

```
class port_driver;
  task run();
    forever begin : drive_one
      get_item();
      if (abort) disable drive_one;
      drive_item();
    end
  endtask
endclass
```

Four `port_driver` objects run this method in parallel. `disable drive_one` in one object terminates the block in **all** of them, since `disable` works on the name. Use `continue` here: `if (abort) continue;`.

D9.

```
// in tb_top
initial begin
  run_test("alu_base_test");
  uvm_config_db #(virtual alu_if)::set(null, "*", "vif", alu_vif);
end
```

`run_test()` starts the phases and does not return until the test is over, so the `set` happens too late (in practice never). The driver's `get` fails in `build_phase` and reports the missing interface. Call `set` before `run_test()`.

D10.

```
task run_phase(uvm_phase phase);
  alu_sequence seq = alu_sequence::type_id::create("seq");
  seq.start(env.agent.sqr);
endtask
```

No objection is raised, so the run phase ends at time 0 and the sequence is killed before it sends anything. Add `phase.raise_objection(this);` before and `phase.drop_objection(this);` after.

6.5 Coding Exercises with Solutions

Exercise 1: a design package

Write a package `bus_pkg` containing address and data width parameters, a direction enum, a packed struct for a bus request, a function that builds a request (data defaulting to zero) and a function that returns its parity. Use it from a module.

ex01_bus_pkg.sv

```
package bus_pkg;

  parameter int ADDR_W = 8;
  parameter int DATA_W = 16;

  typedef enum bit {BUS_READ, BUS_WRITE} bus_dir_e;

  typedef struct packed {
    bus_dir_e          dir;
    logic [ADDR_W-1:0] addr;
    logic [DATA_W-1:0] data;
  } bus_req_t;

  function automatic bus_req_t make_req(bus_dir_e          dir,
                                        logic [ADDR_W-1:0] addr,
                                        logic [DATA_W-1:0] data = '0);
    bus_req_t r;
    r.dir  = dir;
    r.addr = addr;
    r.data = data;
    return r;
  endfunction

  function automatic bit parity(bus_req_t r);
    return ^r;
  endfunction

endpackage : bus_pkg

module ex01_bus_pkg;
  import bus_pkg::*;

  bus_req_t req;

  initial begin
    req = make_req(BUS_WRITE, 8'h3C, 16'hBEEF);
    $display("dir=%s addr=%h data=%h bits=%0d",
             req.dir.name(), req.addr, req.data, $bits(req));
    $display("raw=%h parity=%b", req, parity(req));

    req = make_req(BUS_READ, 8'h10);
    $display("dir=%s addr=%h data=%h", req.dir.name(), req.addr, req.data);
  end
endmodule
```

EXPECTED OUTPUT

```
dir=BUS_WRITE addr=3c data=beef bits=25
raw=13cbeef parity=0
dir=BUS_READ addr=10 data=0000
```

The struct is 1 + 8 + 16 = 25 bits. As a vector it is `1_0011_1100_1011_1110_1110_1111`, printed in hexadecimal as `13cbeef`. It contains 18 ones (1 in `dir`, 4 in `3C`, 13 in `BEEF`), an even number, so the reduction XOR is 0. The second call omits `data`, which takes its default.

Exercise 2: a generic statistics class

Write a parameterized class with static functions that return the maximum and the minimum of a queue of any numeric type. Use it with `int` and with `real`.

ex02_generic_stats.sv

```
class stats #(type T = int);

  static function T max_of(T q [$]);
    T m = q[0];
    foreach (q[i]) if (q[i] > m) m = q[i];
    return m;
  endfunction

  static function T min_of(T q [$]);
    T m = q[0];
    foreach (q[i]) if (q[i] < m) m = q[i];
    return m;
  endfunction

endclass

module ex02_generic_stats;

  int  iq [$] = '{3, 9, -2, 7};
  real rq [$] = '{1.5, 0.25, 7.75};

  initial begin
    $display("int : max=%0d min=%0d", stats #(int)::max_of(iq), stats #(int)::min_of(iq));
    $display("real: max=%0.2f min=%0.2f",
             stats #(real)::max_of(rq), stats #(real)::min_of(rq));
  end

endmodule
```

EXPECTED OUTPUT

```
int : max=9 min=-2
real: max=7.75 min=0.25
```

No object is created: the functions are static and are called through the specialisation name. The class would not compile for a type without `<` and `>`, such as a class handle.

Exercise 3: a task with a timeout, using a process handle

Write `run_with_timeout(duration, limit, timed_out)`: start a piece of work that takes `duration` time units; if it has not finished after `limit` time units, kill it and report a timeout.

ex03_timeout.sv

```
module ex03_timeout;

  task automatic do_work(int duration);
    #duration;
  endtask

  task automatic run_with_timeout(int duration, int limit, output bit timed_out);
    process p_work;

    fork                                  // start the work as its own process
      begin
        p_work = process::self();
        do_work(duration);
      end
    join_none
    wait (p_work != null);

    fork                                  // isolating fork around the race
      begin
        fork
          p_work.await();
          #limit;
        join_any
        disable fork;
      end
    join

    timed_out = (p_work.status() != process::FINISHED);
    if (timed_out) p_work.kill();
  endtask

  bit expired;

  initial begin
    run_with_timeout(30, 50, expired);
    $display("[%0t] work=30 limit=50 : timed_out=%b", $time, expired);

    run_with_timeout(80, 50, expired);
    $display("[%0t] work=80 limit=50 : timed_out=%b", $time, expired);

    #100 $display("[%0t] end", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[30] work=30 limit=50 : timed_out=0
[80] work=80 limit=50 : timed_out=1
[180] end
```

First call: the work ends at time 30, `await()` returns, `join_any` completes, `disable fork` removes the timer, and the status is `FINISHED`. Second call (starting at 30): the timer expires at 80 while the work is still `WAITING`, so a timeout is reported and the work process is killed. The isolating fork guarantees that `disable fork` touches only the awaiting thread and the timer; the work itself is stopped explicitly through its handle.

Exercise 4: a weighted stimulus mix with a self-check

Use `randcase` to choose among three kinds of operation with weights 8, 1 and 1 for 2000 iterations. Check that exactly one branch executed each time and that the heavily weighted branch dominates.

ex04_randcase_mix.sv

```
module ex04_randcase_mix;

  int n_add, n_sub, n_shift;

  initial begin
    repeat (2000) begin
      randcase
        8 : n_add++;
        1 : n_sub++;
        1 : n_shift++;
      endcase
    end

    $display("add=%0d sub=%0d shift=%0d", n_add, n_sub, n_shift);

    if (n_add + n_sub + n_shift != 2000)        $display("FAIL: total is wrong");
    else if (n_add < n_sub || n_add < n_shift)  $display("FAIL: weights not respected");
    else                                        $display("PASS");
  end

endmodule
```

EXPECTED OUTPUT

```
add=1611 sub=197 shift=192
PASS
```

Sample output; the three counts differ per seed and simulator. The total is always 2000, and with weights 8:1:1 the first count is about 1600, so the check prints `PASS`.

Exercise 5: several specialisations in one queue

Store objects of `typed_item #(int)`, `typed_item #(byte)` and `typed_item #(bit [11:0])` in a single queue and print a description of each.

ex05_common_base.sv

```
virtual class item_base;
  pure virtual function string describe();
endclass

class typed_item #(type T = int) extends item_base;
  T value;

  function new(T value);
    this.value = value;
  endfunction

  virtual function string describe();
    return $sformatf("%0d-bit item with value %0d", $bits(value), value);
  endfunction
endclass

module ex05_common_base;

  item_base               q [$];
  typed_item #(int)       i_item;
  typed_item #(byte)      b_item;
  typed_item #(bit [11:0]) v_item;

  initial begin
    i_item = new(100);
    b_item = new(-5);
    v_item = new(12'hABC);

    q.push_back(i_item);        // derived handle stored in a base-class queue (upcast)
    q.push_back(b_item);
    q.push_back(v_item);

    foreach (q[k]) $display("%0d: %s", k, q[k].describe());
  end

endmodule
```

EXPECTED OUTPUT

```
0: 32-bit item with value 100
1: 8-bit item with value -5
2: 12-bit item with value 2748
```

The three specialisations are unrelated to each other, but each extends the non-parameterized `item_base`. The queue is declared with the base type, and the virtual method `describe` reaches the right specialisation at run time. `12'hABC` is 2748 in decimal and prints as a positive number because `bit [11:0]` is unsigned.

Exercise 6: write and load a memory image

Create a hexadecimal memory file from the testbench, load it into an 8-entry memory with `$readmemh`, and print the memory.

ex06_readmem.sv

```
module ex06_readmem;

  logic [7:0] mem [0:7];
  int         fd;

  initial begin
    fd = $fopen("mem_init.hex", "w");
    if (fd == 0) $fatal(1, "cannot create mem_init.hex");
    $fdisplay(fd, "// test image");
    $fdisplay(fd, "@0");
    $fdisplay(fd, "de ad be ef");
    $fdisplay(fd, "@6");
    $fdisplay(fd, "55 aa");
    $fclose(fd);

    $readmemh("mem_init.hex", mem);
    foreach (mem[i]) $display("mem[%0d] = %h", i, mem[i]);
  end

endmodule
```

EXPECTED OUTPUT

```
mem[0] = de
mem[1] = ad
mem[2] = be
mem[3] = ef
mem[4] = xx
mem[5] = xx
mem[6] = 55
mem[7] = aa
```

`@0` and `@6` set the load address. Locations 4 and 5 are not mentioned in the file, so they keep the default value of `logic`, X. (Some simulators print a warning that not every location was initialised.)

Further practice (no solutions given)

1. Split the M9 testbench into `alu_pkg`, `alu_tb_pkg` (with one `` `include `` per class file) and `tb_top`. Write the compile order.
2. Write a generic class `pair #(type A = int, type B = string)` with a constructor and a `swap_into(output pair #(B, A) p)` method. Why is the type of the result a different specialisation?
3. Rewrite `inorder_scoreboard` as an out-of-order scoreboard using an associative array keyed by `id`.
4. Start five threads in a loop, each with a random delay from `$urandom_range(10, 50)`. Using process handles, print which thread finished first and kill the others.
5. Write a `randsequence` that generates: one `CONFIG`, then between 2 and 5 transfers (each `WRITE` twice as likely as `READ`), then one `DISABLE`.
6. Write a C function `c_crc8(const char* s)` and call it through DPI for three strings. Declare it `pure`. Explain why that is legitimate.
7. Add `+TIMEOUT=<cycles>` to the M9 testbench: read it with `$value$plusargs`, default 10000, and end the test with `$fatal` when it expires.
8. For each class in the UVM skeleton of Section 5.8, write one sentence naming the M9 class it corresponds to and one difference.

6.6 Concepts You Must Understand Before Moving Forward

Packages and scope

- [ ] I can list what a package may and may not contain, and say why.
- [ ] I can explain explicit import, wildcard import and `pkg::name`, and the collision rules of each.
- [ ] I can explain why an explicit import of an enum type does not give me its labels.
- [ ] I can explain `` `include `` versus `import`, and why a class file belongs in exactly one package.
- [ ] I can list at least six uses of `::` and distinguish it from `.`.

Generic code

- [ ] I can write a parameterized class with a type parameter and a value parameter.
- [ ] I can explain why specialisations are unrelated types and how static members behave.
- [ ] I can explain when a generic class should use `==` and when `compare()`.
- [ ] I can declare a virtual interface to a parameterized interface.
- [ ] I can state the default clocking-block skews and what they prevent.

Process control

- [ ] I can say what creates a process and what does not.
- [ ] I can explain the `disable fork` trap and write the isolating fork from memory.
- [ ] I can obtain a process handle correctly and name the five states.
- [ ] I can choose between `disable fork`, `disable <label>` and `kill()`.
- [ ] I can write a timeout around a blocking operation.

randcase, randsequence, DPI, system tasks

- [ ] I can compute branch probabilities of a `randcase` and say what happens with zero weights.
- [ ] I can read a small `randsequence`.
- [ ] I can write a DPI import with input and output arguments and the matching C prototype, and explain `pure` and `context`.
- [ ] I can explain `$display` / `$strobe` / `$monitor`, `$error` / `$fatal`, `$random` / `$urandom`, the two plusargs functions, `$readmemh`, `$fopen`, `$sformatf`, `$clog2`, `$onehot` / `$onehot0`.

UVM

- [ ] I can say what UVM is in two sentences.
- [ ] I can map every M9 component to its UVM counterpart.
- [ ] I can explain `uvm_object` versus `uvm_component`, the phases, the factory, `uvm_config_db`, analysis ports and objections at the level of "what problem does it solve".
- [ ] For each of those, I can name the SystemVerilog feature it is built on.

6.7 What Comes Next

This was the last teaching module. The final part of the course is the **interview preparation section**: more than 150 questions with answers covering all eleven modules, grouped as basic, intermediate and advanced, and

including output-prediction, code-debugging, find-the-error, write-the-code, constraint-writing, OOP, IPC, layered-testbench and scenario-based questions, with coding exercises and solutions. Use the checklists at the end of each module to decide which areas to revise before attempting it.